

Discrete Time Signal Processing

Discrete Fourier Transform (DFT) & Fast Fourier Transform (FFT)

Fast Fourier Transform



Discrete Fourier Transform

Discrete Fourier Transform

- DFT of discrete-time signal $x[n]$ of length N

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \quad k = 0, 1, \dots, N-1$$

- Let, $k=0$

- Assuming $x[n]$ be complex
 - **Complex multiplications: N per DFT value**
 - **Complex addition: $(N-1)$ per DFT value**

Discrete Fourier Transform

- DFT of discrete-time signal $x[n]$ of length N

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j \frac{2\pi}{N} k n} \quad k = 0, 1, \dots, N-1$$

- Let, $k = 0$

- Assuming $x[n]$ be complex
 - Complex multiplications: N per DFT value**
 - Complex addition: (N-1) per DFT value**

Discrete Fourier Transform

- DFT of discrete-time signal $x[n]$ of length N

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \quad k = 0, 1, \dots, N-1$$

- Let, $k=0$

$$X[0] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{4}0n}$$

- Assuming $x[n]$ be complex
 - **Complex multiplications: N per DFT value**
 - **Complex addition: $(N-1)$ per DFT value**

Discrete Fourier Transform

- DFT of discrete-time signal $x[n]$ of length N

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \quad k = 0, 1, \dots, N-1$$

- Let, $k=0$

$$\begin{aligned} X[0] &= \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{4}0n} \\ &= x[0] e^{-j\frac{2\pi}{4}00} + x[1] e^{-j\frac{2\pi}{4}01} + \dots + x[N-1] e^{-j\frac{2\pi}{4}0(N-1)} \end{aligned}$$

- Assuming $x[n]$ be complex
 - **Complex multiplications: N per DFT value**
 - **Complex addition: $(N-1)$ per DFT value**

Discrete Fourier Transform

- DFT of discrete-time signal $x[n]$ of length N

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \quad k = 0, 1, \dots, N-1$$

- Let, $k=0$

$$\begin{aligned} X[0] &= \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{4}0n} \\ &= x[0] e^{-j\frac{2\pi}{4}00} + x[1] e^{-j\frac{2\pi}{4}01} + \dots + x[N-1] e^{-j\frac{2\pi}{4}0(N-1)} \end{aligned}$$

- Assuming $x[n]$ be complex
 - **Complex multiplications: N per DFT value**
 - **Complex addition: $(N-1)$ per DFT value**

Discrete Fourier Transform

- DFT of discrete-time signal $x[n]$ of length N

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \quad k = 0, 1, \dots, N-1$$

- Let, $k=0$

$$\begin{aligned} X[0] &= \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{4}0n} \\ &= x[0] e^{-j\frac{2\pi}{4}00} + x[1] e^{-j\frac{2\pi}{4}01} + \dots + x[N-1] e^{-j\frac{2\pi}{4}0(N-1)} \end{aligned}$$

- Assuming $x[n]$ be complex
 - Complex multiplications: N per DFT value
 - Complex addition: $(N-1)$ per DFT value
- For N -point DFT N^2 complex multiplications
 $N(N-1)$ complex additions

Discrete Fourier Transform

Discrete Fourier Transform

- Separating real and imaginary parts
- Assuming $x[n]$ be complex
 - **Real multiplications: $4N$ per DFT value**
 - **Real addition: $(4N-2)$ per DFT value**

Discrete Fourier Transform

- Separating real and imaginary parts

$$X[k] = \sum_{n=0}^{N-1} \operatorname{Re}\{x[n]\} \cdot \cos\left(\frac{2\pi}{4} k n\right) - \operatorname{Im}\{x[n]\} \cdot \sin\left(\frac{2\pi}{4} k n\right) \\ + j \left(\operatorname{Re}\{x[n]\} \cdot \sin\left(\frac{2\pi}{4} k n\right) - \operatorname{Im}\{x[n]\} \cdot \cos\left(\frac{2\pi}{4} k n\right) \right)$$

- Assuming $x[n]$ be complex
 - **Real multiplications: 4N per DFT value**
 - **Real addition: (4N-2) per DFT value**

Discrete Fourier Transform

- Separating real and imaginary parts

$$X[k] = \sum_{n=0}^{N-1} \operatorname{Re}\{x[n]\} \cdot \cos\left(\frac{2\pi}{4} k n\right) - \operatorname{Im}\{x[n]\} \cdot \sin\left(\frac{2\pi}{4} k n\right) \\ + j \left(\operatorname{Re}\{x[n]\} \cdot \sin\left(\frac{2\pi}{4} k n\right) - \operatorname{Im}\{x[n]\} \cdot \cos\left(\frac{2\pi}{4} k n\right) \right)$$

- Assuming $x[n]$ be complex
 - **Real multiplications: 4N per DFT value**
 - **Real addition: (4N-2) per DFT value**

Discrete Fourier Transform

- Separating real and imaginary parts

$$X[k] = \sum_{n=0}^{N-1} \text{Re}\{x[n]\} \cdot \cos\left(\frac{2\pi}{4} k n\right) - \text{Im}\{x[n]\} \cdot \sin\left(\frac{2\pi}{4} k n\right) \\ + j \left(\text{Re}\{x[n]\} \cdot \sin\left(\frac{2\pi}{4} k n\right) - \text{Im}\{x[n]\} \cdot \cos\left(\frac{2\pi}{4} k n\right) \right)$$

- Assuming $x[n]$ be complex
 - Real multiplications: $4N$ per DFT value
 - Real addition: $(4N-2)$ per DFT value
- For N-point DFT $4N^2$ real multiplications
 $N(4N - 2)$ real additions

Fast Fourier Transform

Fast Fourier Transform

- **Fast Fourier Transform (FFT) is an algorithm** that efficiently computes the Discrete Fourier Transform (DFT).
- Computational algorithms that exploit both the **symmetry and the periodicity of the sequence W_N^{nk}** .
- Runge (1905) and later Danielson and Lanczos (1942) described algorithms for which computation was roughly proportional to **$N \log N$ rather than N^2** .
 - Distinction was **not of great importance for the small values of N** .
- **Cooley and Tukey (1965)** published an algorithm for the computation of the discrete Fourier transform that is applicable when **N is a composite number**, i.e., the product of two or more integers.
 - James w. Cooley, Peter A. W. Lewis, and Peter D. WELCH, “Historical Notes on the Fast Fourier Transform”, *Proceedings IEEE. vol. 55. no. 10., October 1967.*

Fast Fourier Transform

- **Fast Fourier Transform (FFT) is an algorithm** that efficiently computes the Discrete Fourier Transform (DFT).
- Computational algorithms that exploit both the **symmetry and the periodicity of the sequence W_N^{nk}** .
- Runge (1905) and later Danielson and Lanczos (1942) described algorithms for which computation was roughly proportional to **$N \log N$ rather than N^2** .
 - Distinction was **not of great importance for the small values of N** .
- **Cooley and Tukey (1965)** published an algorithm for the computation of the discrete Fourier transform that is applicable when **N is a composite number**, i.e., the product of two or more integers.
 - James w. Cooley, Peter A. W. Lewis, and Peter D. WELCH, “Historical Notes on the Fast Fourier Transform”, *Proceedings IEEE. vol. 55. no. 10., October 1967.*

Fast Fourier Transform

- **Fast Fourier Transform (FFT)** is an **algorithm** that efficiently computes the Discrete Fourier Transform (DFT).
- Computational algorithms that exploit both the **symmetry and the periodicity of the sequence W_N^{nk}** .
- Runge (1905) and later Danielson and Lanczos (1942) described algorithms for which computation was roughly proportional to **$N \log N$ rather than N^2** .
 - Distinction was **not of great importance for the small values of N** .
- **Cooley and Tukey (1965)** published an algorithm for the computation of the discrete Fourier transform that is applicable when **N is a composite number**, i.e., the product of two or more integers.
 - James w. Cooley, Peter A. W. Lewis, and Peter D. WELCH, “Historical Notes on the Fast Fourier Transform”, *Proceedings IEEE. vol. 55. no. 10., October 1967.*

Fast Fourier Transform

- **Fast Fourier Transform (FFT) is an algorithm** that efficiently computes the Discrete Fourier Transform (DFT).
- Computational algorithms that exploit both the **symmetry and the periodicity of the sequence W_N^{nk}** .
- Runge (1905) and later Danielson and Lanczos (1942) described algorithms for which computation was roughly proportional to **$N \log N$ rather than N^2** .
 - Distinction was **not of great importance for the small values of N** .
- **Cooley and Tukey (1965)** published an algorithm for the computation of the discrete Fourier transform that is applicable when **N is a composite number**, i.e., the product of two or more integers.
 - James w. Cooley, Peter A. W. Lewis, and Peter D. WELCH, “Historical Notes on the Fast Fourier Transform”, *Proceedings IEEE. vol. 55. no. 10., October 1967.*

Fast Fourier Transform

- **Fast Fourier Transform (FFT) is an algorithm** that efficiently computes the Discrete Fourier Transform (DFT).
- Computational algorithms that exploit both the **symmetry and the periodicity of the sequence W_N^{nk}** .
- Runge (1905) and later Danielson and Lanczos (1942) described algorithms for which computation was roughly proportional to **$N \log N$ rather than N^2** .
 - Distinction was **not of great importance for the small values of N** .
- **Cooley and Tukey (1965)** published an algorithm for the computation of the discrete Fourier transform that is applicable when **N is a composite number**, i.e., the product of two or more integers.
 - James w. Cooley, Peter A. W. Lewis, and Peter D. WELCH, “Historical Notes on the Fast Fourier Transform”, *Proceedings IEEE. vol. 55. no. 10., October 1967.*

Complexity of FFT algorithms

Complexity of FFT algorithms

- The number of **arithmetic multiplications and additions** as a measure of computational complexity.
- Simple to apply and **directly related to the computational speed** when on general-purpose digital computers or special-purpose microprocessors
- In custom VLSI implementations, the **area of the chip and power requirements** are important considerations.
- The efficiency of FFT algorithms is so high, that in many cases the **most efficient** procedure for implementing a **convolution is through DFT and IDFT process**.

Complexity of FFT algorithms

- The number of **arithmetic multiplications and additions** as a measure of computational complexity.
- Simple to apply and **directly related to the computational speed** when on general-purpose digital computers or special-purpose microprocessors
- In custom VLSI implementations, the **area of the chip and power requirements** are important considerations.
- The efficiency of FFT algorithms is so high, that in many cases the **most efficient** procedure for implementing a **convolution is through DFT and IDFT process**.

Complexity of FFT algorithms

- The number of **arithmetic multiplications and additions** as a measure of computational complexity.
- Simple to apply and **directly related to the computational speed** when on general-purpose digital computers or special-purpose microprocessors
- In custom VLSI implementations, the **area of the chip and power requirements** are important considerations.
- The efficiency of FFT algorithms is so high, that in many cases the **most efficient** procedure for implementing **a convolution is through DFT and IDFT process**.

Complexity of FFT algorithms

- The number of **arithmetic multiplications and additions** as a measure of computational complexity.
- Simple to apply and **directly related to the computational speed** when on general-purpose digital computers or special-purpose microprocessors
- In custom VLSI implementations, the **area of the chip and power requirements** are important considerations.
- The efficiency of FFT algorithms is so high, that in many cases the **most efficient** procedure for implementing a **convolution is through DFT and IDFT process**.

FFT: Cooley-Tuckey algorithm



FFT Butterfly

FFT Butterfly

- Let $x[n] = \{x_0, x_1\}$ and corresponding DFT $X(k) = \{X_0, X_1\}$

FFT Butterfly

- Let $x[n] = \{x_0, x_1\}$ and corresponding DFT $X(k) = \{X_0, X_1\}$

$$X[k] = \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}kn} \quad \dots(\text{Taking DFT})$$

FFT Butterfly

- Let $x[n] = \{x_0, x_1\}$ and corresponding DFT $X(k) = \{X_0, X_1\}$

$$X[k] = \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}kn} \quad \dots(\text{Taking DFT})$$

$$X[0] = \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}0n}$$

FFT Butterfly

- Let $x[n] = \{x_0, x_1\}$ and corresponding DFT $X(k) = \{X_0, X_1\}$

$$X[k] = \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}kn} \quad \dots(\text{Taking DFT})$$

$$X[0] = \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}0n} = x[0] e^{-j\frac{2\pi}{2}00} + x[1] e^{-j\frac{2\pi}{2}01}$$

FFT Butterfly

- Let $x[n] = \{x_0, x_1\}$ and corresponding DFT $X(k) = \{X_0, X_1\}$

$$X[k] = \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}kn} \quad \dots (\text{Taking DFT})$$

$$\begin{aligned} X[0] &= \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}0n} = x[0] e^{-j\frac{2\pi}{2}00} + x[1] e^{-j\frac{2\pi}{2}01} \\ &= X_0 + X_1 \end{aligned}$$

FFT Butterfly

- Let $x[n] = \{x_0, x_1\}$ and corresponding DFT $X(k) = \{X_0, X_1\}$

$$X[k] = \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}kn} \quad \dots(\text{Taking DFT})$$

$$\begin{aligned} X[0] &= \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}0n} = x[0] e^{-j\frac{2\pi}{2}00} + x[1] e^{-j\frac{2\pi}{2}01} \\ &= X_0 + X_1 \end{aligned}$$

$$X[1] = \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}1n}$$

FFT Butterfly

- Let $x[n] = \{x_0, x_1\}$ and corresponding DFT $X(k) = \{X_0, X_1\}$

$$X[k] = \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}kn} \quad \dots(\text{Taking DFT})$$

$$\begin{aligned} X[0] &= \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}0n} = x[0] e^{-j\frac{2\pi}{2}00} + x[1] e^{-j\frac{2\pi}{2}01} \\ &= X_0 + X_1 \end{aligned}$$

$$X[1] = \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}1n} = x[0] e^{-j\frac{2\pi}{2}10} + x[1] e^{-j\frac{2\pi}{2}11}$$

FFT Butterfly

- Let $x[n] = \{x_0, x_1\}$ and corresponding DFT $X(k) = \{X_0, X_1\}$

$$X[k] = \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}kn} \quad \dots(\text{Taking DFT})$$

$$\begin{aligned} X[0] &= \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}0n} = x[0] e^{-j\frac{2\pi}{2}00} + x[1] e^{-j\frac{2\pi}{2}01} \\ &= X_0 + X_1 \end{aligned}$$

$$\begin{aligned} X[1] &= \sum_{n=0}^1 x[n] e^{-j\frac{2\pi}{2}1n} = x[0] e^{-j\frac{2\pi}{2}10} + x[1] e^{-j\frac{2\pi}{2}11} \\ &= X_0 - X_1 \end{aligned}$$

FFT Butterfly

FFT Butterfly

$$X[k] = \left\{ x_0 + x_1, \quad x_0 - x_1 \right\}$$

FFT Butterfly

$$X[k] = \left\{ x_0 + x_1, \quad x_0 - x_1 \right\}$$

x_0

x_1

FFT Butterfly

$$X[k] = \left\{ x_0 + x_1, \quad x_0 - x_1 \right\}$$

x_0

X_0

x_1

X_1

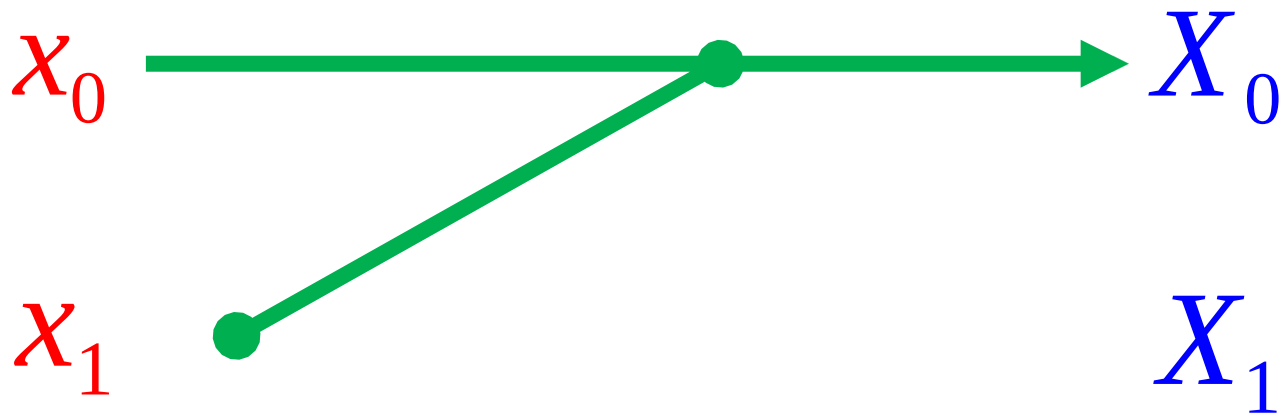
FFT Butterfly

$$X[k] = \left\{ x_0 + x_1, \quad x_0 - x_1 \right\}$$



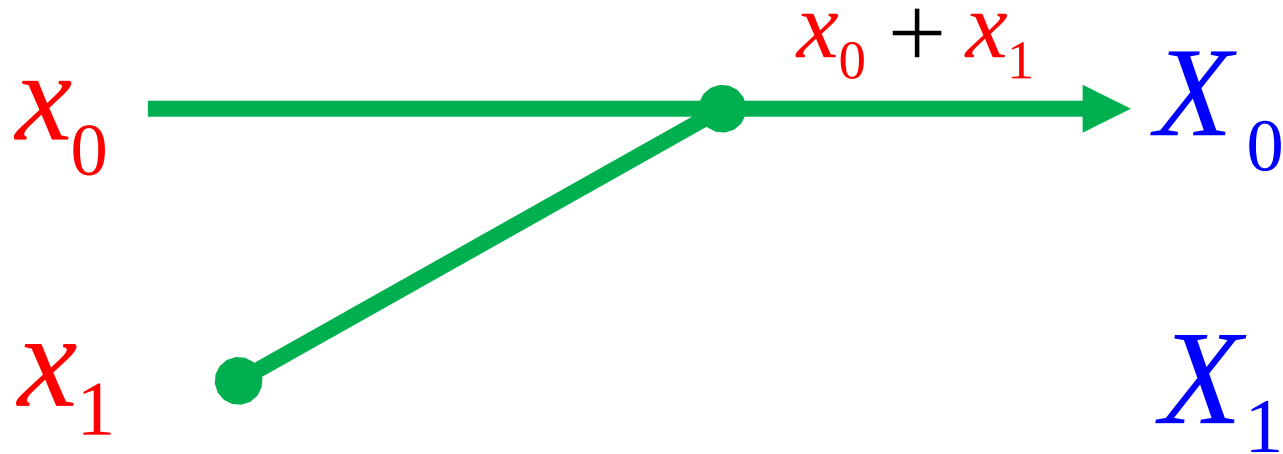
FFT Butterfly

$$X[k] = \{x_0 + x_1, \quad x_0 - x_1\}$$



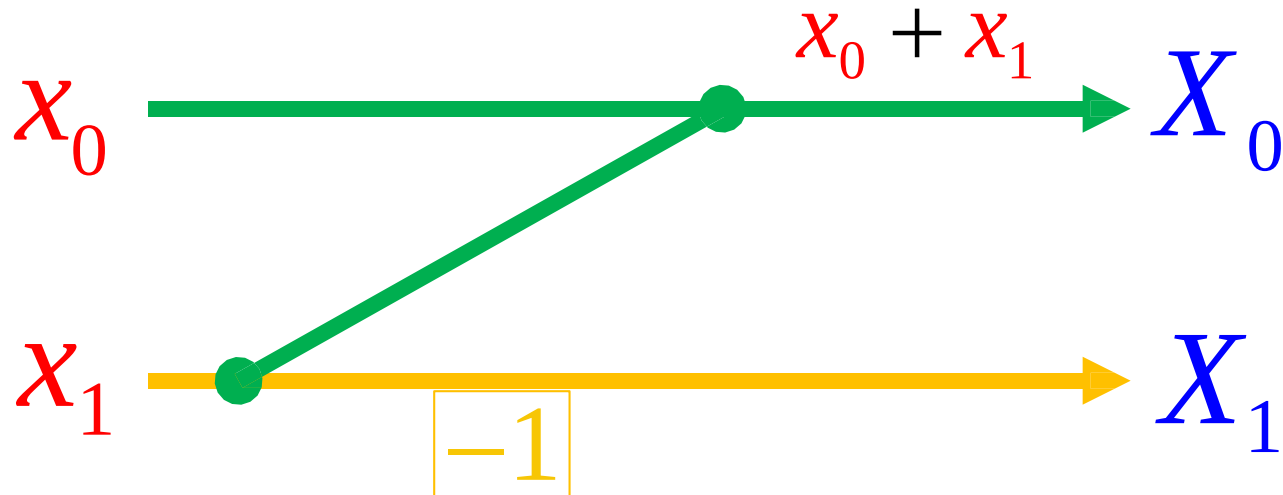
FFT Butterfly

$$X[k] = \{x_0 + x_1, \quad x_0 - x_1\}$$



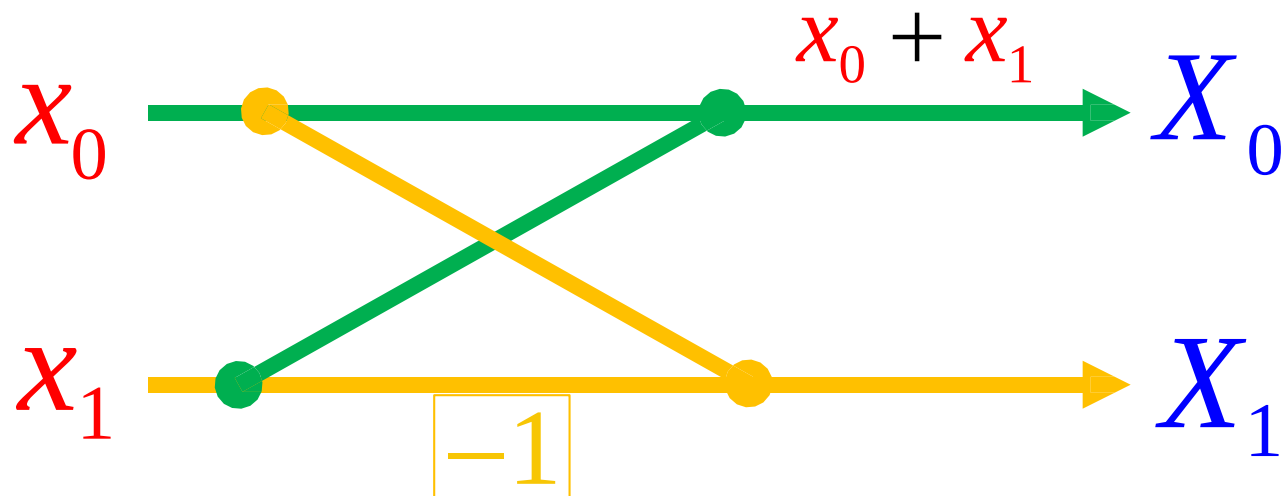
FFT Butterfly

$$X[k] = \{x_0 + x_1, \quad x_0 - x_1\}$$



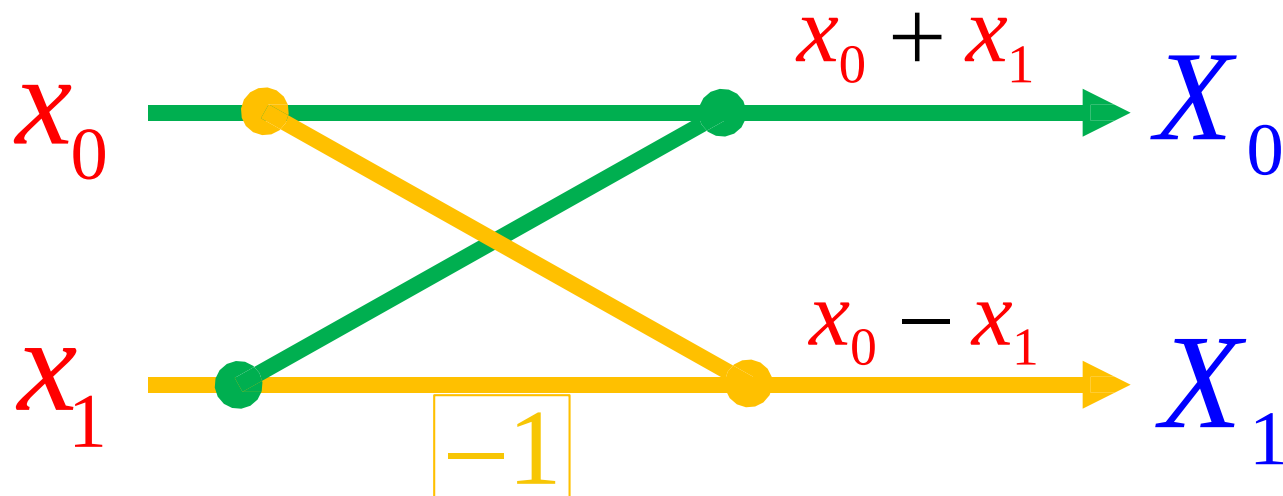
FFT Butterfly

$$X[k] = \{x_0 + x_1, \quad x_0 - x_1\}$$



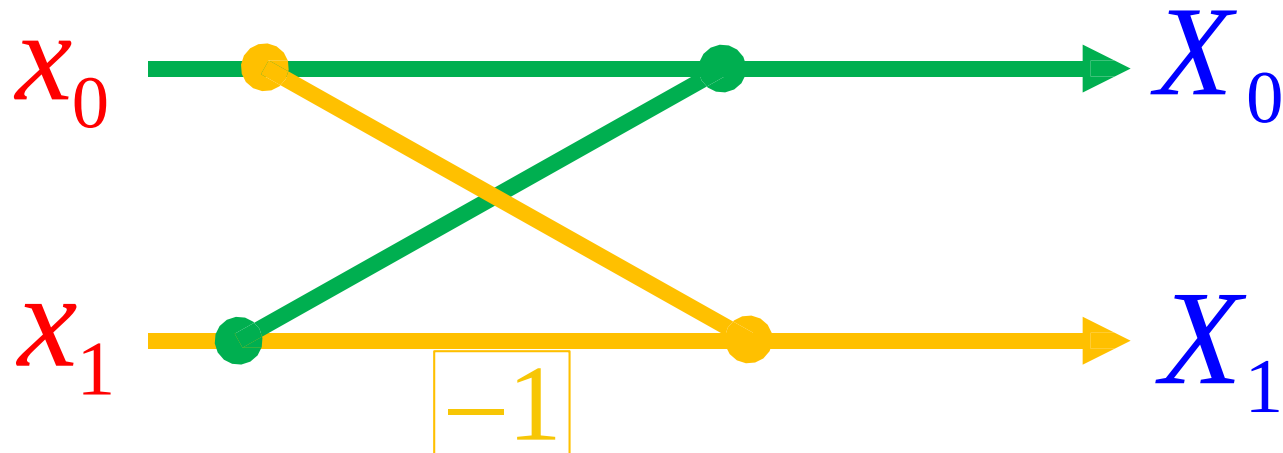
FFT Butterfly

$$X[k] = \{x_0 + x_1, \quad x_0 - x_1\}$$



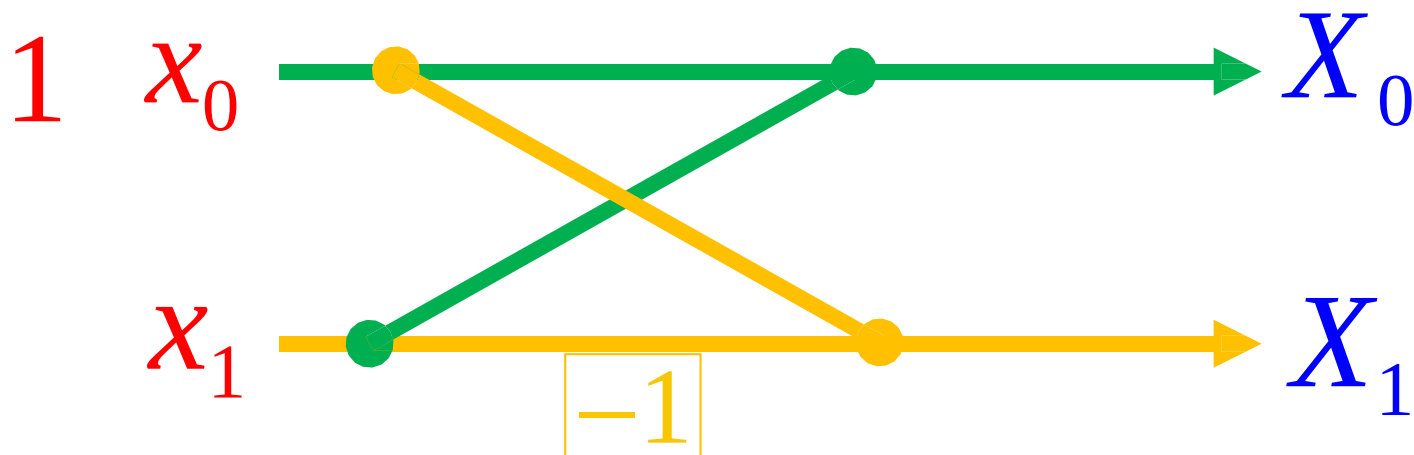
Example-8

Compute DFT of $x[n] = \{1, 2\}$ using FFT algorithm



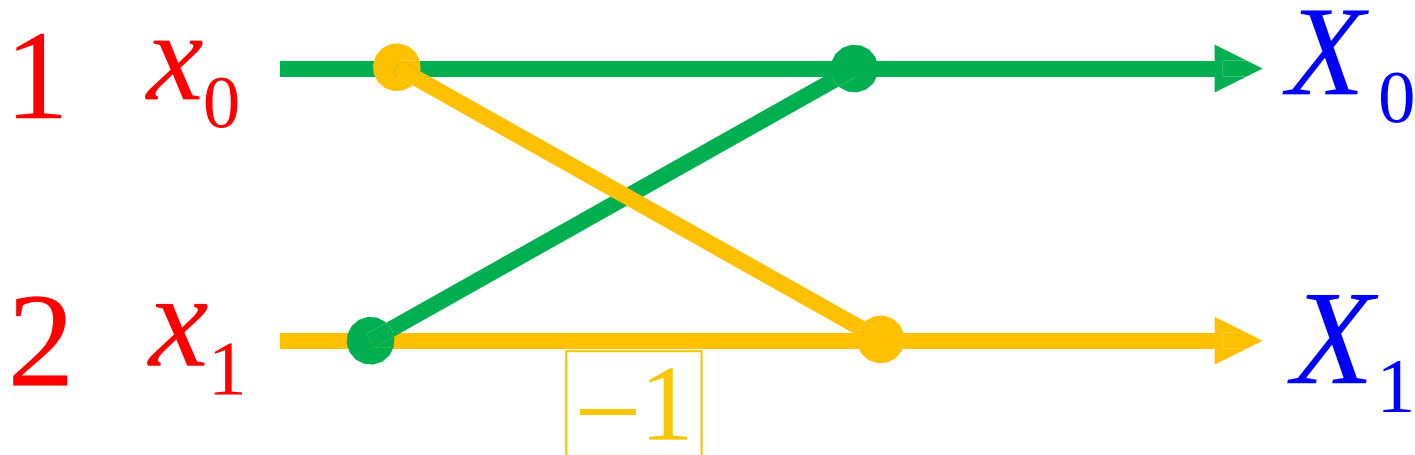
Example-8

Compute DFT of $x[n] = \{1, 2\}$ using FFT algorithm



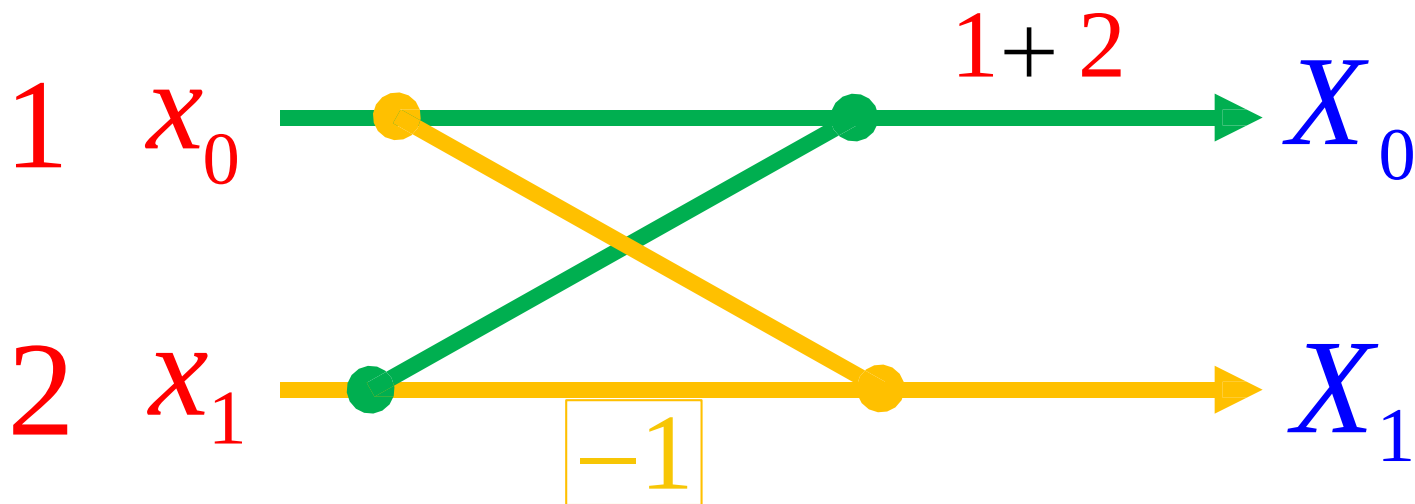
Example-8

Compute DFT of $x[n] = \{1, 2\}$ using FFT algorithm



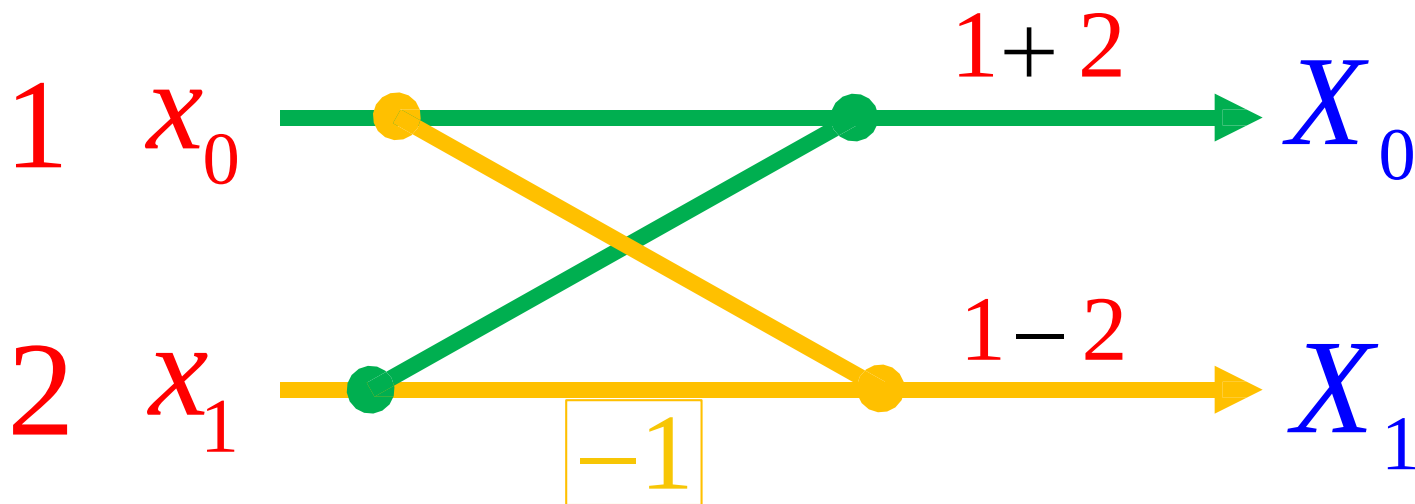
Example-8

Compute DFT of $x[n] = \{1, 2\}$ using FFT algorithm



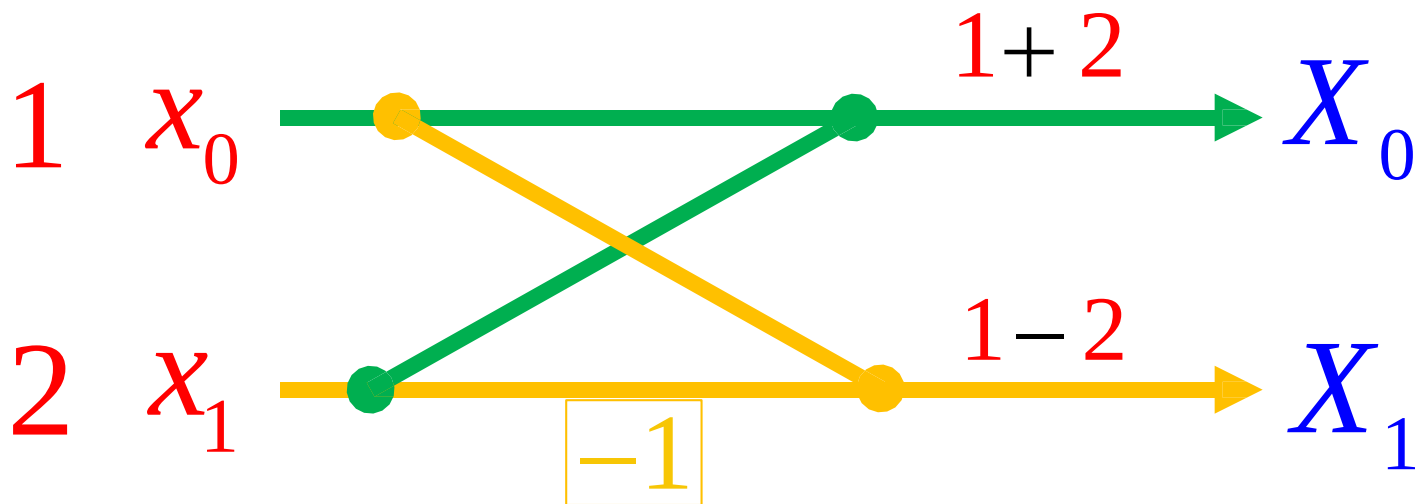
Example-8

Compute DFT of $x[n] = \{1, 2\}$ using FFT algorithm



Example-8

Compute DFT of $x[n] = \{1, 2\}$ using FFT algorithm



$$X[k] = \{3, -1\}$$

Fast Fourier Transform

Fast Fourier Transform

- Fundamental principle of **decomposing** the computation of N-point DFT into **successively smaller DFTs**.
- The manner in which this principle is implemented leads to a variety of different algorithms.
 - **Decimation in Time (DIT)**

The sequence $x[n]$ (generally thought of as a time sequence) is decomposed into successively smaller subsequences.
 - **Decimation in Frequency (DIF)**

The sequence of DFT coefficients $X[k]$ is decomposed into smaller subsequences

Fast Fourier Transform

- Fundamental principle of **decomposing** the computation of N-point DFT into **successively smaller DFTs**.
- The manner in which this principle is implemented leads to a variety of different algorithms.
 - **Decimation in Time (DIT)**

The sequence $x[n]$ (generally thought of as a time sequence) is decomposed into successively smaller subsequences.
 - **Decimation in Frequency (DIF)**

The sequence of DFT coefficients $X[k]$ is decomposed into smaller subsequences

Fast Fourier Transform

- Fundamental principle of **decomposing** the computation of N-point DFT into **successively smaller DFTs**.
- The manner in which this principle is implemented leads to a variety of different algorithms.
 - **Decimation in Time (DIT)**

The sequence $x[n]$ (generally thought of as a time sequence) is decomposed into successively smaller subsequences.
 - **Decimation in Frequency (DIF)**

The sequence of DFT coefficients $X[k]$ is decomposed into smaller subsequences

Fast Fourier Transform

- Fundamental principle of **decomposing** the computation of N-point DFT into **successively smaller DFTs**.
- The manner in which this principle is implemented leads to a variety of different algorithms.
 - **Decimation in Time (DIT)**

The sequence $x[n]$ (generally thought of as a time sequence) is decomposed into successively smaller subsequences.
 - **Decimation in Frequency (DIF)**

The sequence of DFT coefficients $X[k]$ is decomposed into smaller subsequences

Decimation In Time FFT (DIT-FFT)



Decimation-In-Time FFT Algorithm

Decimation-In-Time FFT Algorithm

- Sequence $x[n]$ is decomposed into successively smaller subsequences.
- Considering the special case of **N an integer power of 2**, i.e., $N = 2^v$.
- Separating $x[n]$ into two $(N/2)$ -point sequences consisting of the even-indexed points in $x[n]$ and the odd-indexed points in $x[n]$.
- Even indexed $x[2m]$ and odd indexed $x[2m + 1]$ each $N/2$ long

Decimation-In-Time FFT Algorithm

- Sequence $x[n]$ is decomposed into successively smaller subsequences.
- Considering the special case of **N an integer power of 2**, i.e., $N = 2^v$.
- Separating $x[n]$ into two $(N/2)$ -point sequences consisting of the even-indexed points in $x[n]$ and the odd-indexed points in $x[n]$.
- Even indexed $x[2m]$ and odd indexed $x[2m + 1]$ each $N/2$ long

Decimation-In-Time FFT Algorithm

- Sequence $x[n]$ is decomposed into successively smaller subsequences.
- Considering the special case of **N an integer power of 2**, i.e., $N = 2^v$.

$$X[k] = \sum_{n=0}^{N-1} x[n] w_N^{kn} \quad k, n = 0, 1, \dots, N-1$$

- Separating $x[n]$ into two $(N/2)$ -point sequences consisting of the even-indexed points in $x[n]$ and the odd-indexed points in $x[n]$.
- Even indexed $x[2m]$ and odd indexed $x[2m+1]$ each $N/2$ long

Decimation-In-Time FFT Algorithm

- Sequence $x[n]$ is decomposed into successively smaller subsequences.
- Considering the special case of **N an integer power of 2**, i.e., $N = 2^v$.

$$X[k] = \sum_{n=0}^{N-1} x[n] w_N^{kn} \quad k, n = 0, 1, \dots, N-1$$

- Separating $x[n]$ into two $(N/2)$ -point sequences consisting of the even-indexed points in $x[n]$ and the odd-indexed points in $x[n]$.
- Even indexed $x[2m]$ and odd indexed $x[2m+1]$ each $N/2$ long

Decimation-In-Time FFT Algorithm

- Sequence $x[n]$ is decomposed into successively smaller subsequences.
- Considering the special case of **N an integer power of 2**, i.e., $N = 2^v$.

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn} \quad k, n = 0, 1, \dots, N-1$$

- Separating $x[n]$ into two $(N/2)$ -point sequences consisting of the even-indexed points in $x[n]$ and the odd-indexed points in $x[n]$.

$$X[k] = \sum_{n \text{ even}} x[n] W_N^{kn} + \sum_{n \text{ odd}} x[n] W_N^{kn}$$

- Even indexed $x[2m]$ and odd indexed $x[2m+1]$ each $N/2$ long

Decimation-In-Time FFT Algorithm

- Sequence $x[n]$ is decomposed into successively smaller subsequences.
- Considering the special case of **N an integer power of 2**, i.e., $N = 2^v$.

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn} \quad k, n = 0, 1, \dots, N-1$$

- Separating $x[n]$ into two $(N/2)$ -point sequences consisting of the even-indexed points in $x[n]$ and the odd-indexed points in $x[n]$.

$$X[k] = \sum_{n \text{ even}} x[n] W_N^{kn} + \sum_{n \text{ odd}} x[n] W_N^{kn}$$

- Even indexed $x[2m]$ and odd indexed $x[2m+1]$ each $N/2$ long

Decimation-In-Time FFT Algorithm

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] W_N^{k2m} + \sum_{m=0}^{N/2-1} x[2m+1] W_N^{k(2m+1)}$$

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] W_N^{k2m} + \sum_{m=0}^{N/2-1} x[2m+1] W_N^{k(2m+1)}$$

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] \left(W_N^2\right)^{km} + \sum_{m=0}^{N/2-1} x[2m+1] \left(W_N^2\right)^{km} W_N^k$$

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] W_N^{k2m} + \sum_{m=0}^{N/2-1} x[2m+1] W_N^{k(2m+1)}$$

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] \left(W_N^2\right)^{km} + \sum_{m=0}^{N/2-1} x[2m+1] \left(W_N^2\right)^{km} W_N^k$$

$$\left(W_N^2\right)^{km}$$

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] W_N^{k2m} + \sum_{m=0}^{N/2-1} x[2m+1] W_N^{k(2m+1)}$$

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] \left(W_N^2\right)^{km} + \sum_{m=0}^{N/2-1} x[2m+1] \left(W_N^2\right)^{km} W_N^k$$

$$\left(W_N^2\right)^{km} = \left(e^{-j\frac{2\pi}{N}}\right)^{km}$$

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] W_N^{k2m} + \sum_{m=0}^{N/2-1} x[2m+1] W_N^{k(2m+1)}$$

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] \left(W_N^2\right)^{km} + \sum_{m=0}^{N/2-1} x[2m+1] \left(W_N^2\right)^{km} W_N^k$$

$$\left(W_N^2\right)^{km} = \left(e^{-j\frac{2\pi}{N}2}\right)^{km} = \left(e^{-j\frac{2\pi}{N/2}}\right)^{km}$$

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] W_N^{k2m} + \sum_{m=0}^{N/2-1} x[2m+1] W_N^{k(2m+1)}$$

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] \left(W_N^2\right)^{km} + \sum_{m=0}^{N/2-1} x[2m+1] \left(W_N^2\right)^{km} W_N^k$$

$$\left(W_N^2\right)^{km} = \left(e^{-j\frac{2\pi}{N}2}\right)^{km} = \left(e^{-j\frac{2\pi}{N/2}}\right)^{km} = W_{N/2}^{km}$$

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] W_N^{k2m} + \sum_{m=0}^{N/2-1} x[2m+1] W_N^{k(2m+1)}$$

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] \left(W_N^2\right)^{km} + \sum_{m=0}^{N/2-1} x[2m+1] \left(W_N^2\right)^{km} W_N^k$$

$$\left(W_N^2\right)^{km} = \left(e^{-j\frac{2\pi}{N}2}\right)^{km} = \left(e^{-j\frac{2\pi}{N/2}}\right)^{km} = W_{N/2}^{km}$$

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] W_{N/2}^{km} + W_N^k \sum_{m=0}^{N/2-1} x[2m+1] W_{N/2}^{km}$$

Decimation-In-Time FFT Algorithm

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] w_{N/2}^{km} + w_N^k \sum_{m=0}^{N/2-1} x[2m+1] w_{N/2}^{km}$$

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] w_{N/2}^{km} + W_N^k \sum_{m=0}^{N/2-1} x[2m+1] w_{N/2}^{km}$$

$$X[k] = G[k] + W_N^k H[k]$$

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] w_{N/2}^{km} + w_N^k \sum_{m=0}^{N/2-1} x[2m+1] w_{N/2}^{km}$$

$$X[k] = G[k] + w_N^k H[k]$$

- $G[k]$ and $H[k]$ are $N/2$ -point DFTs.
- Although $k = 0, 1, \dots, N-1$, since $G[k]$ and $H[k]$ are each periodic in k with period $N/2$ sums must be computed only for $k = 0, 1, \dots, (N/2)-1$.

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] w_{N/2}^{km} + w_N^k \sum_{m=0}^{N/2-1} x[2m+1] w_{N/2}^{km}$$

$$X[k] = G[k] + w_N^k H[k]$$

- $G[k]$ and $H[k]$ are $N/2$ -point DFTs.
- Although $k = 0, 1, \dots, N-1$, since $G[k]$ and $H[k]$ are each periodic in k with period $N/2$ sums must be computed only for $k = 0, 1, \dots, (N/2) - 1$.

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] w_{N/2}^{km} + w_N^k \sum_{m=0}^{N/2-1} x[2m+1] w_{N/2}^{km}$$

$$X[k] = G[k] + w_N^k H[k]$$

- $G[k]$ and $H[k]$ are $N/2$ -point DFTs.
- Although $k = 0, 1, \dots, N-1$, since $G[k]$ and $H[k]$ are each periodic in k with period $N/2$ sums must be computed only for $k = 0, 1, \dots, (N/2) - 1$.

$$X[k] = G[k] + w_N^k H[k] \qquad k = (N/2), \dots, N-1$$

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] w_{N/2}^{km} + w_N^k \sum_{m=0}^{N/2-1} x[2m+1] w_{N/2}^{km}$$

$$X[k] = G[k] + w_N^k H[k]$$

- $G[k]$ and $H[k]$ are $N/2$ -point DFTs.
- Although $k = 0, 1, \dots, N-1$, since $G[k]$ and $H[k]$ are each periodic in k with period $N/2$ sums must be computed only for $k = 0, 1, \dots, (N/2) - 1$.

$$X[k] = G[k] + w_N^k H[k] \quad k = (N/2), \dots, N-1$$

$$w_N^{(k+N/2)}$$

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] w_{N/2}^{km} + w_N^k \sum_{m=0}^{N/2-1} x[2m+1] w_{N/2}^{km}$$

$$X[k] = G[k] + w_N^k H[k]$$

- $G[k]$ and $H[k]$ are $N/2$ -point DFTs.
- Although $k = 0, 1, \dots, N-1$, since $G[k]$ and $H[k]$ are each periodic in k with period $N/2$ sums must be computed only for $k = 0, 1, \dots, (N/2) - 1$.

$$X[k] = G[k] + w_N^k H[k] \quad k = (N/2), \dots, N-1$$

$$w_N^{(k+N/2)} = w_N^k w_N^{N/2}$$

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] w_{N/2}^{km} + w_N^k \sum_{m=0}^{N/2-1} x[2m+1] w_{N/2}^{km}$$

$$X[k] = G[k] + w_N^k H[k]$$

- $G[k]$ and $H[k]$ are $N/2$ -point DFTs.
- Although $k = 0, 1, \dots, N-1$, since $G[k]$ and $H[k]$ are each periodic in k with period $N/2$ sums must be computed only for $k = 0, 1, \dots, (N/2) - 1$.

$$X[k] = G[k] + w_N^k H[k] \quad k = (N/2), \dots, N-1$$

$$w_N^{(k+N/2)} = w_N^k w_N^{N/2} = w_N^k e^{-j\frac{2\pi N}{N}/2}$$

Decimation-In-Time FFT Algorithm

$$X[k] = \sum_{m=0}^{N/2-1} x[2m] w_{N/2}^{km} + w_N^k \sum_{m=0}^{N/2-1} x[2m+1] w_{N/2}^{km}$$

$$X[k] = G[k] + w_N^k H[k]$$

- $G[k]$ and $H[k]$ are $N/2$ -point DFTs.
- Although $k = 0, 1, \dots, N-1$, since $G[k]$ and $H[k]$ are each periodic in k with period $N/2$ sums must be computed only for $k = 0, 1, \dots, (N/2) - 1$.

$$X[k] = G[k] + w_N^k H[k] \quad k = (N/2), \dots, N-1$$

$$w_N^{(k+N/2)} = w_N^k w_N^{N/2} = w_N^k e^{-j\frac{2\pi N}{N}/2} = -w_N^k$$

Decimation-In-Time FFT Algorithm

Decimation-In-Time FFT Algorithm

$$G[k] = \sum_{l=0}^{N/2-1} g[2l] w_{N/2}^{k2l} + \sum_{l=0}^{N/2-1} g[2l+1] w_{N/2}^{k(2l+1)}$$

Decimation-In-Time FFT Algorithm

$$G[k] = \sum_{l=0}^{N/4-1} g[2l] w_{N/2}^{k2l} + \sum_{l=0}^{N/4-1} g[2l+1] w_{N/2}^{k(2l+1)}$$

$$G[k] = \sum_{l=0}^{N/4-1} g[2l] \left(w_{N/2}^2 \right)^{kl} + \sum_{l=0}^{N/4-1} g[2l+1] \left(w_{N/2}^2 \right)^{kl} w_{N/2}^k$$

Decimation-In-Time FFT Algorithm

$$G[k] = \sum_{l=0}^{N/4-1} g[2l] w_{N/2}^{k2l} + \sum_{l=0}^{N/4-1} g[2l+1] w_{N/2}^{k(2l+1)}$$

$$G[k] = \sum_{l=0}^{N/4-1} g[2l] \left(w_{N/2}^2 \right)^{kl} + \sum_{l=0}^{N/4-1} g[2l+1] \left(w_{N/2}^2 \right)^{kl} w_{N/2}^k$$

$$G[k] = \sum_{l=0}^{N/4-1} g[2l] w_{N/4}^{kl} + w_{N/2}^k \sum_{l=0}^{N/4-1} g[2l+1] w_{N/4}^{kl}$$

Decimation-In-Time FFT Algorithm

$$G[k] = \sum_{l=0}^{N/4-1} g[2l] w_{N/2}^{k2l} + \sum_{l=0}^{N/4-1} g[2l+1] w_{N/2}^{k(2l+1)}$$

$$G[k] = \sum_{l=0}^{N/4-1} g[2l] \left(w_{N/2}^2 \right)^{kl} + \sum_{l=0}^{N/4-1} g[2l+1] \left(w_{N/2}^2 \right)^{kl} w_{N/2}^k$$

$$G[k] = \sum_{l=0}^{N/4-1} g[2l] w_{N/4}^{kl} + w_{N/2}^k \sum_{l=0}^{N/4-1} g[2l+1] w_{N/4}^{kl}$$

Similarly

$$H[k] = \sum_{l=0}^{N/4-1} h[2l] w_{N/4}^{kl} + w_{N/2}^k \sum_{l=0}^{N/4-1} h[2l+1] w_{N/4}^{kl}$$

4-point DITFFT

4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$

4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$

X_0

X_2

4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$

X_0

X_2

X_1

X_3

4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$

x_0

X_0

x_2

X_1

x_1

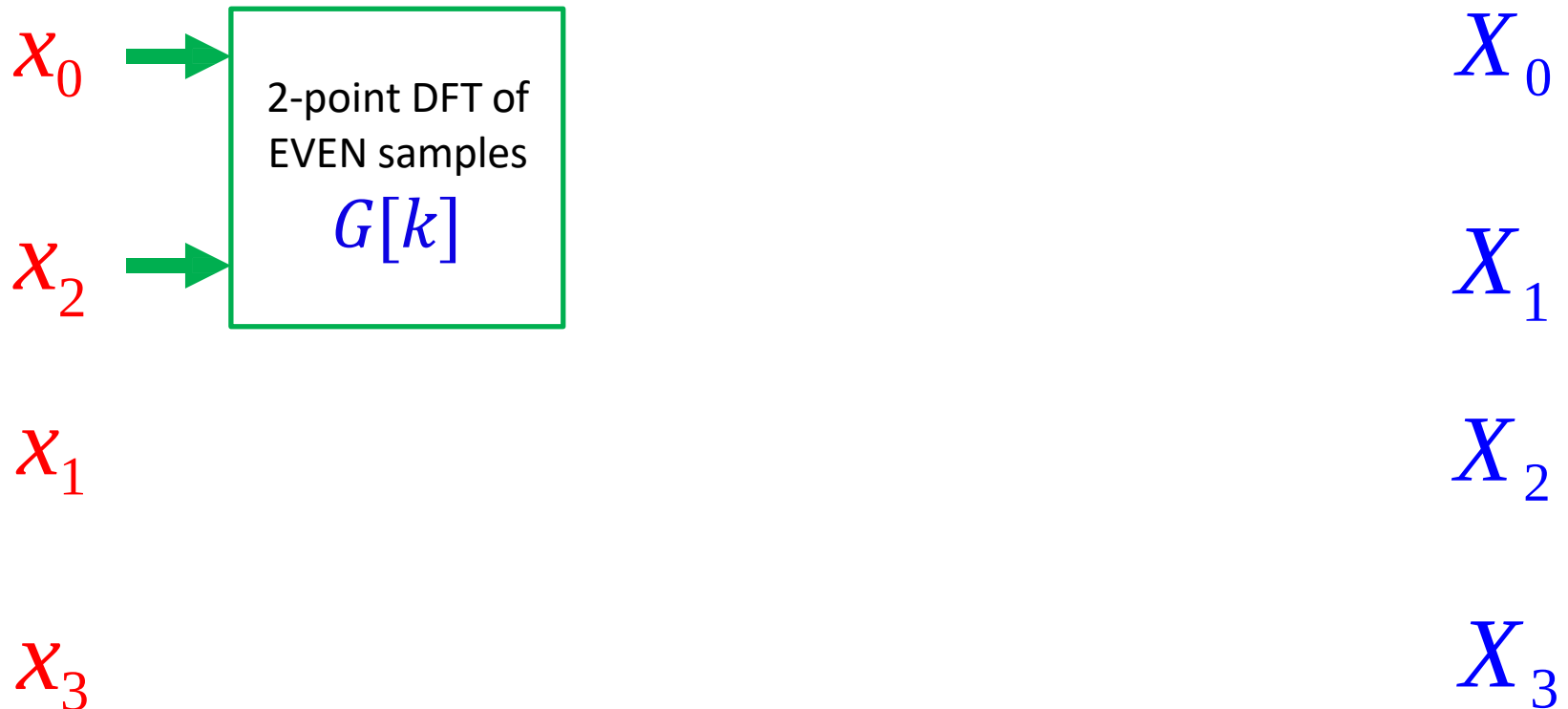
X_2

x_3

X_3

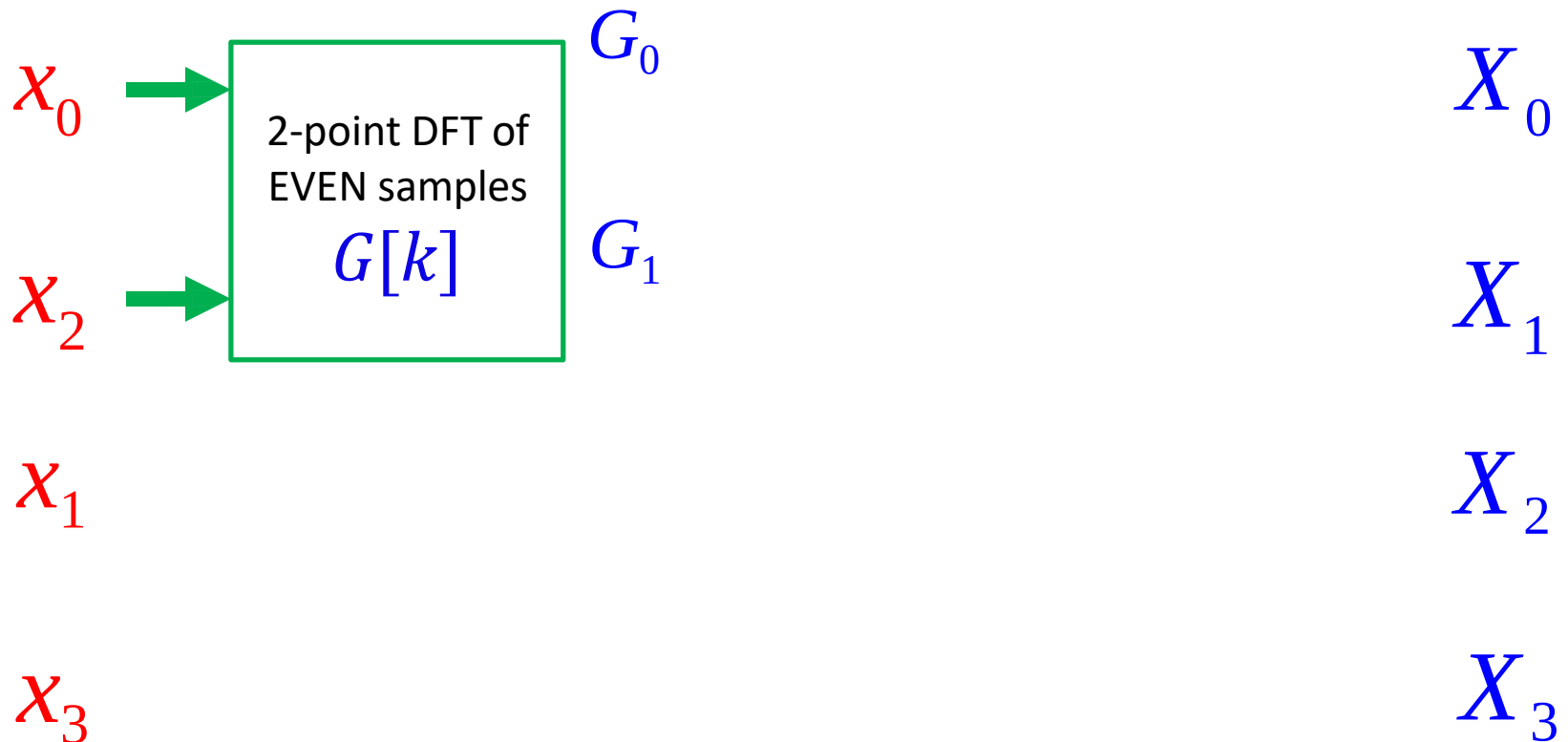
4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



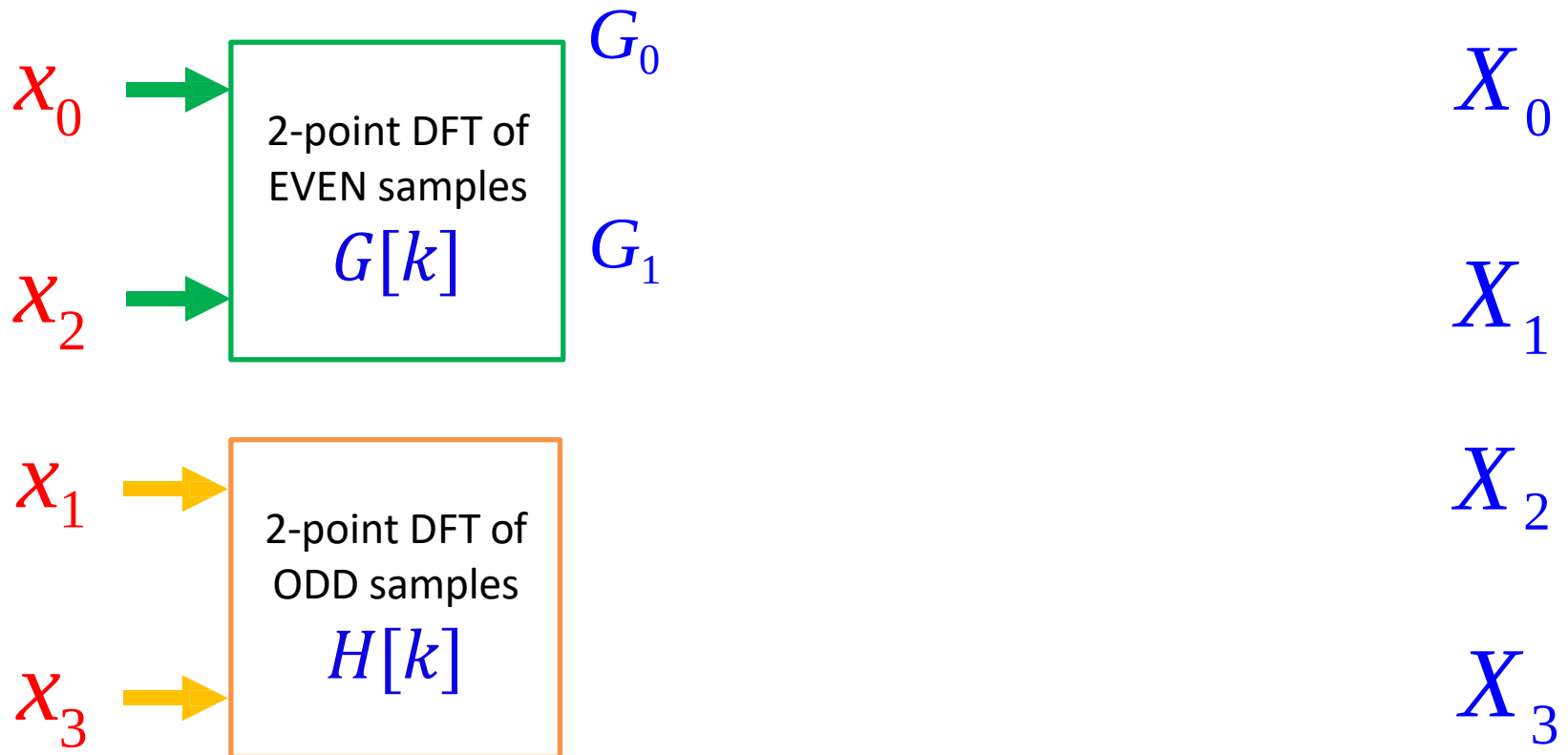
4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



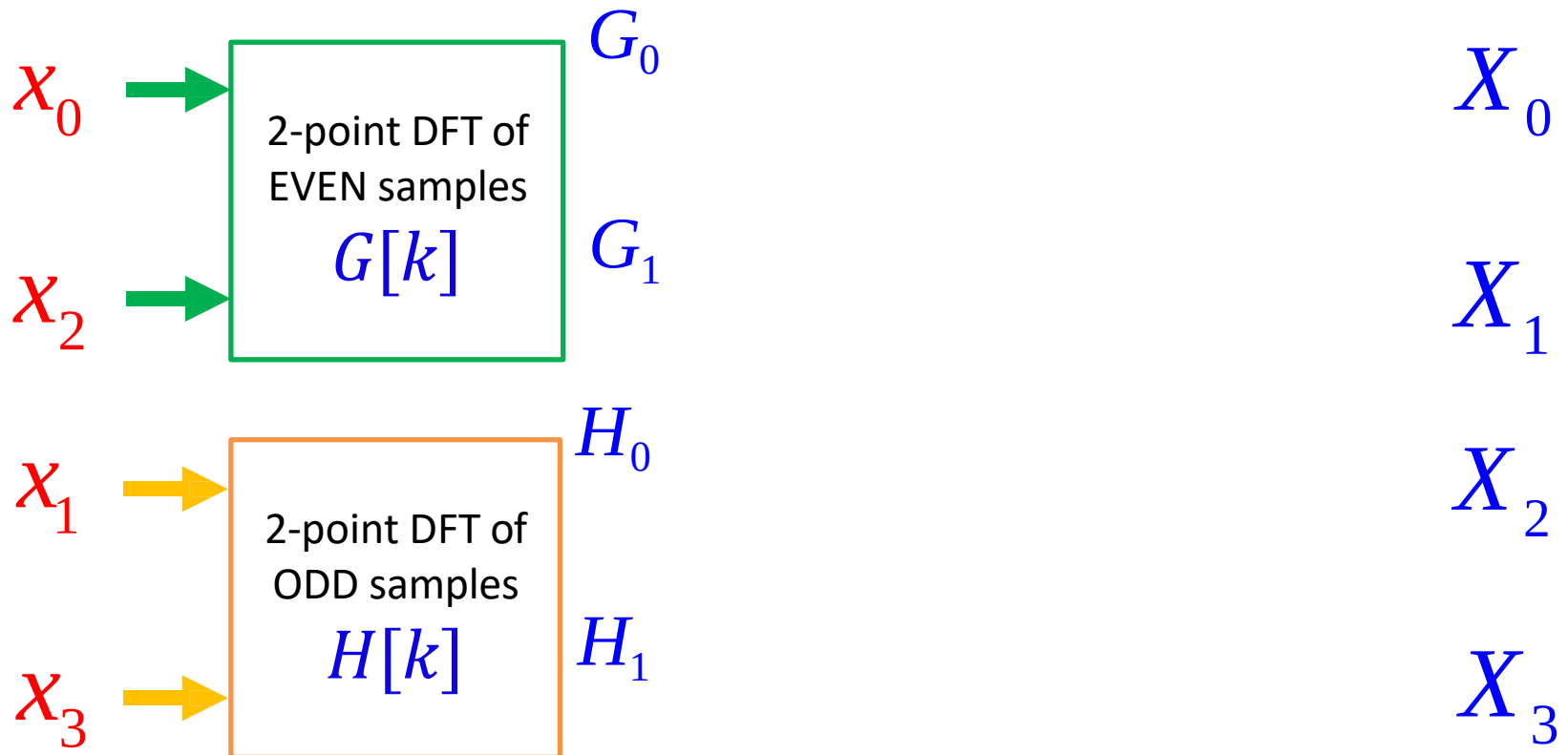
4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



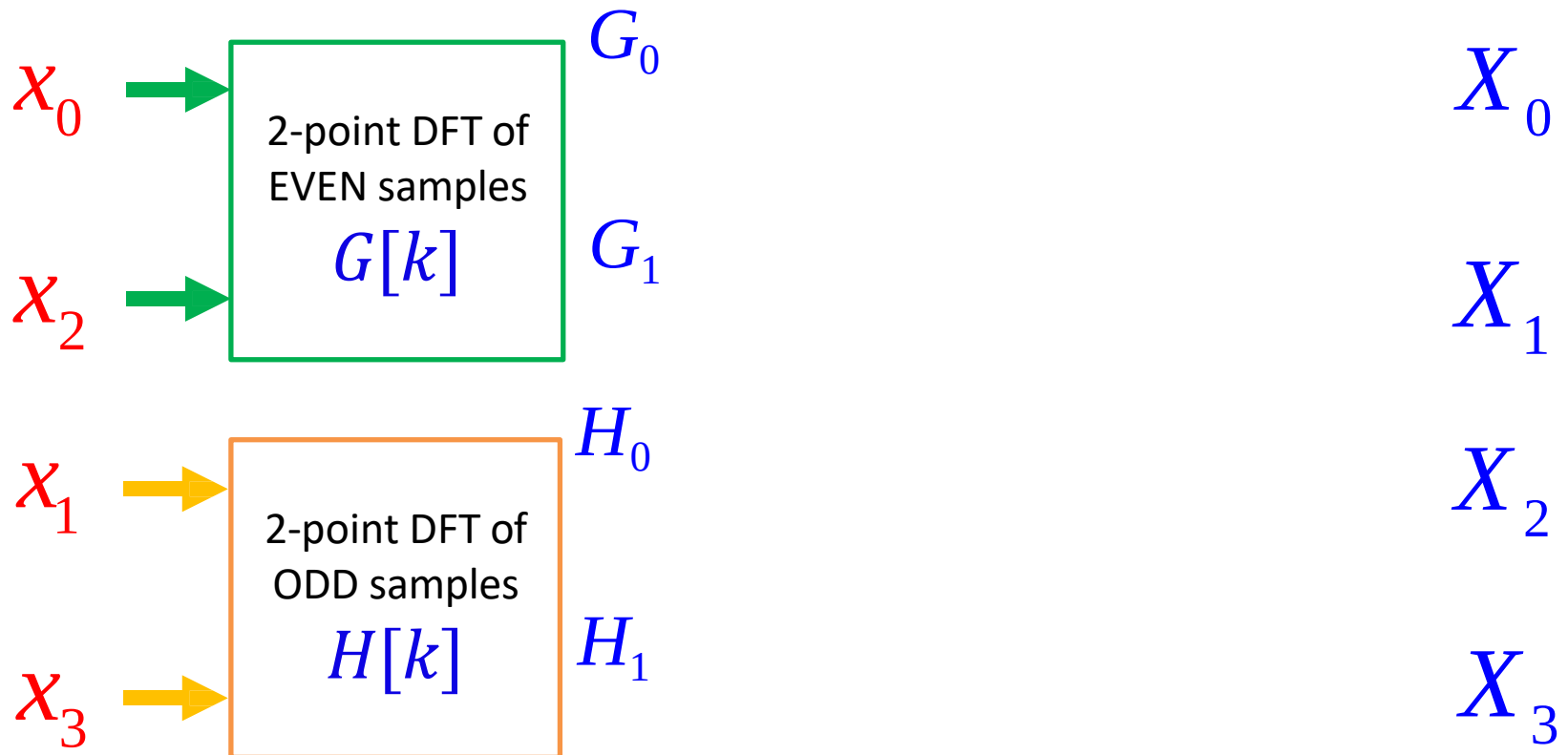
4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



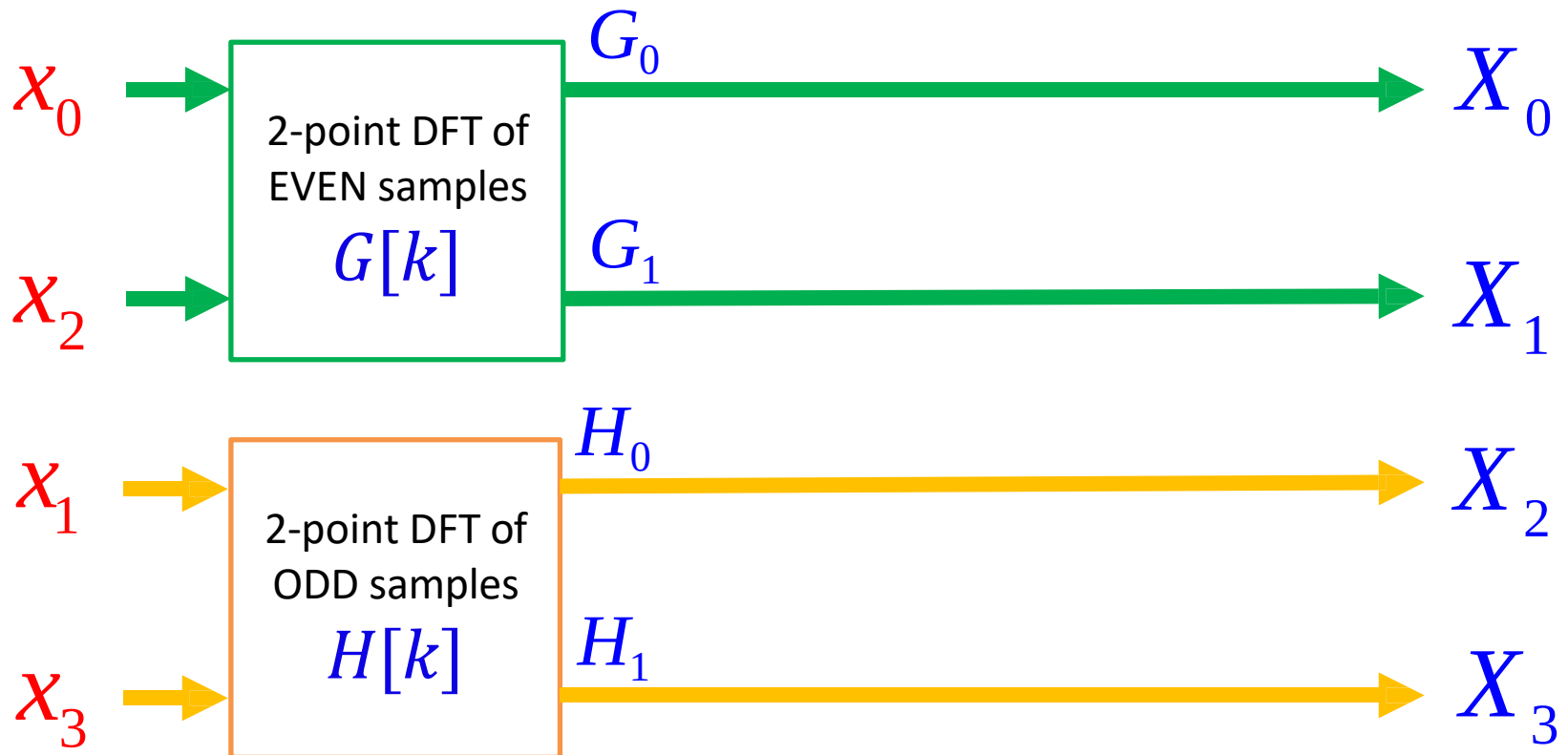
4-point DITFFT $X[k] = G[k] + W_4^k H[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



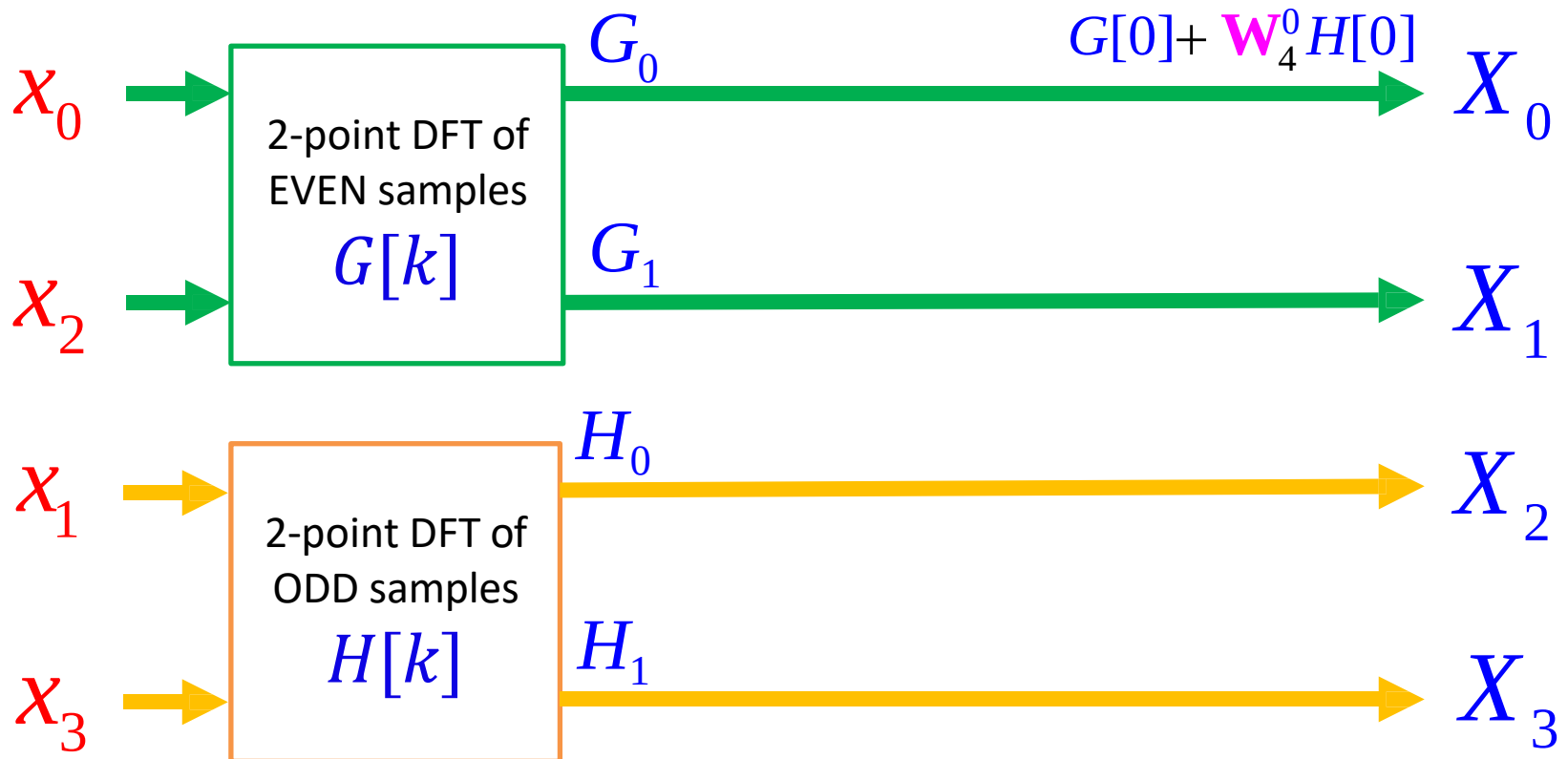
4-point DITFFT $X[k] = G[k] + W_4^k H[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



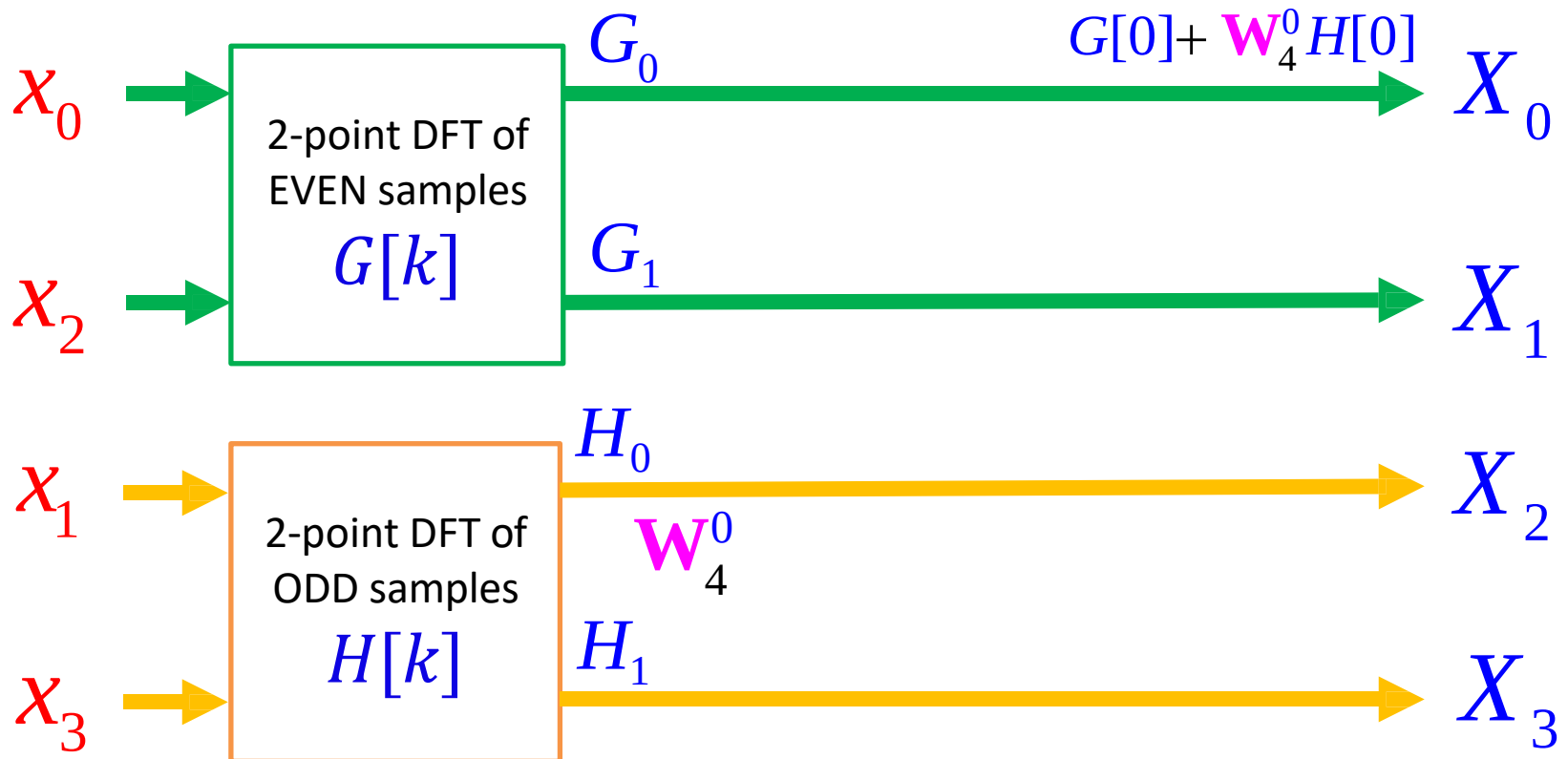
4-point DITFFT $X[k] = G[k] + \mathbf{W}_4^k H[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



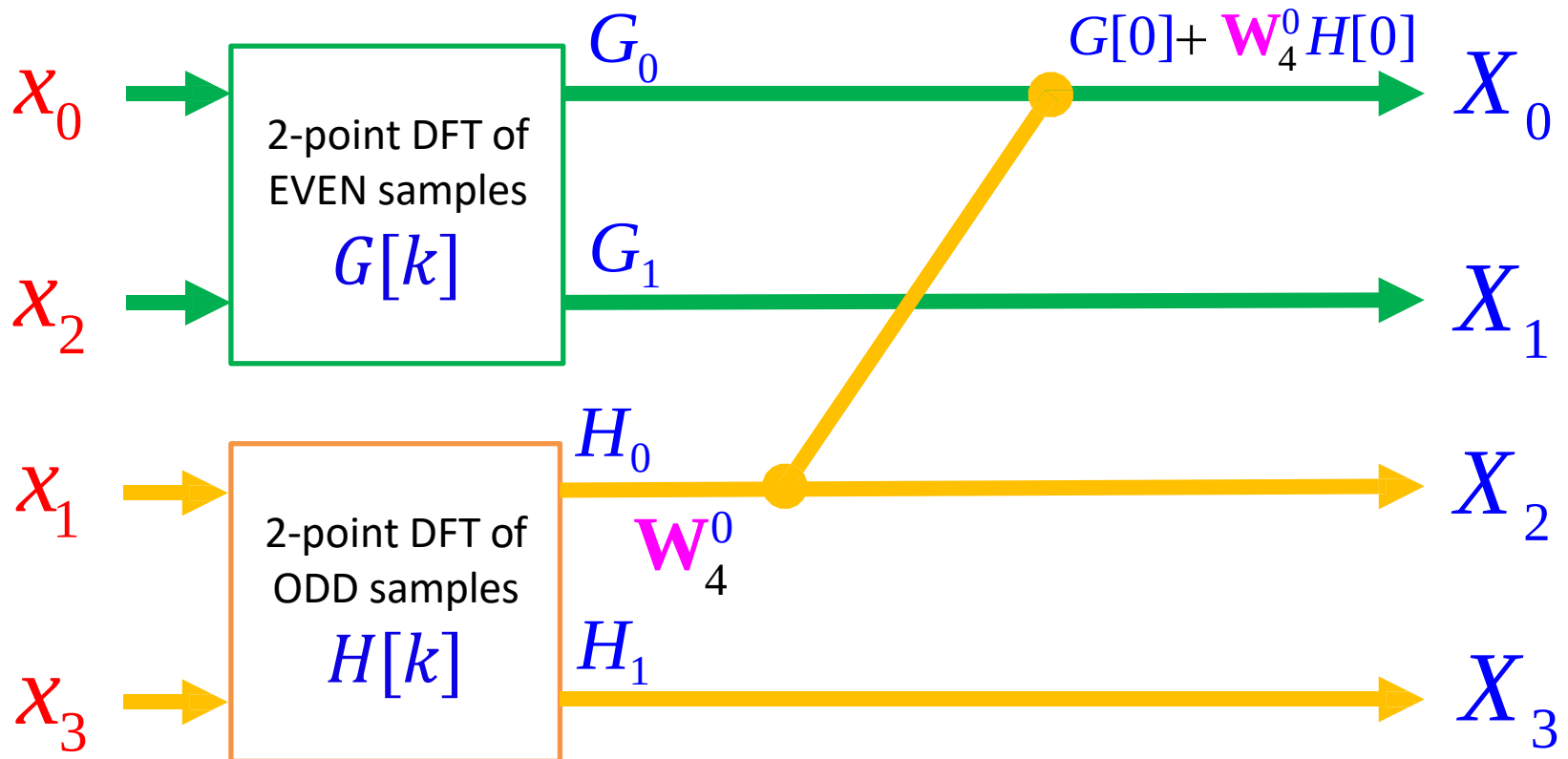
4-point DITFFT $X[k] = G[k] + \mathbf{W}_4^k H[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



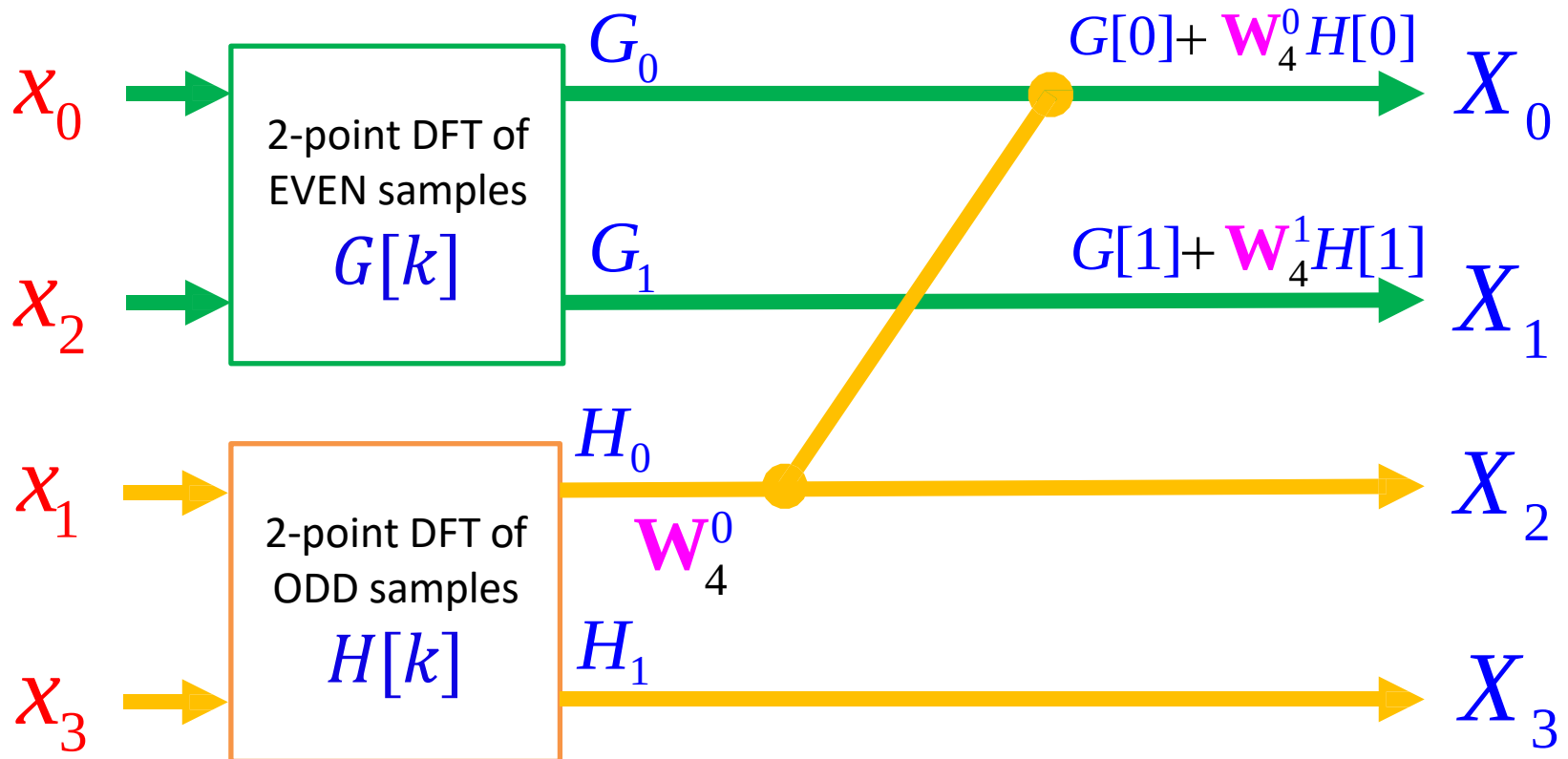
4-point DITFFT $X[k] = G[k] + \mathbf{W}_4^k H[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



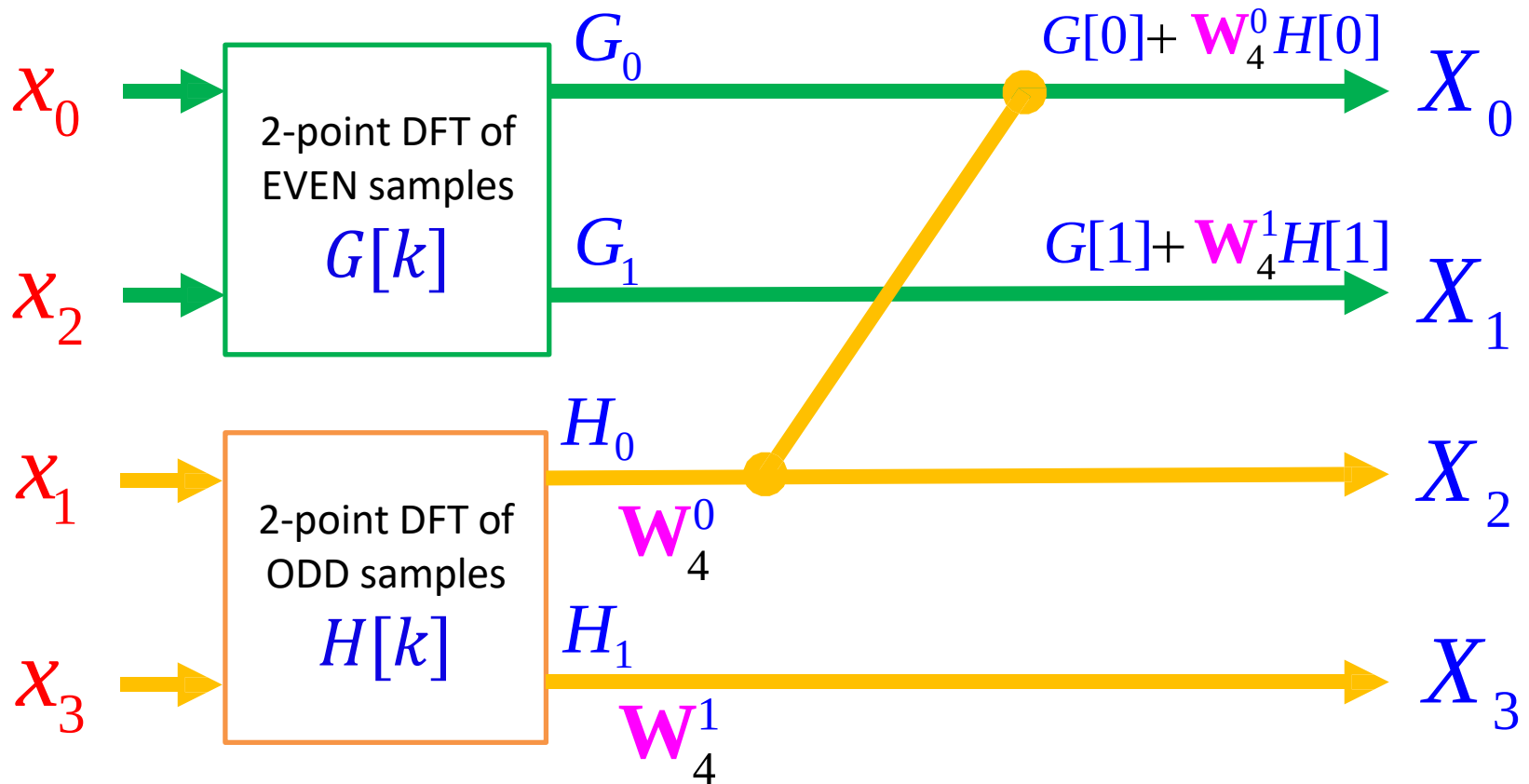
4-point DITFFT $X[k] = G[k] + \mathbf{W}_4^k H[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



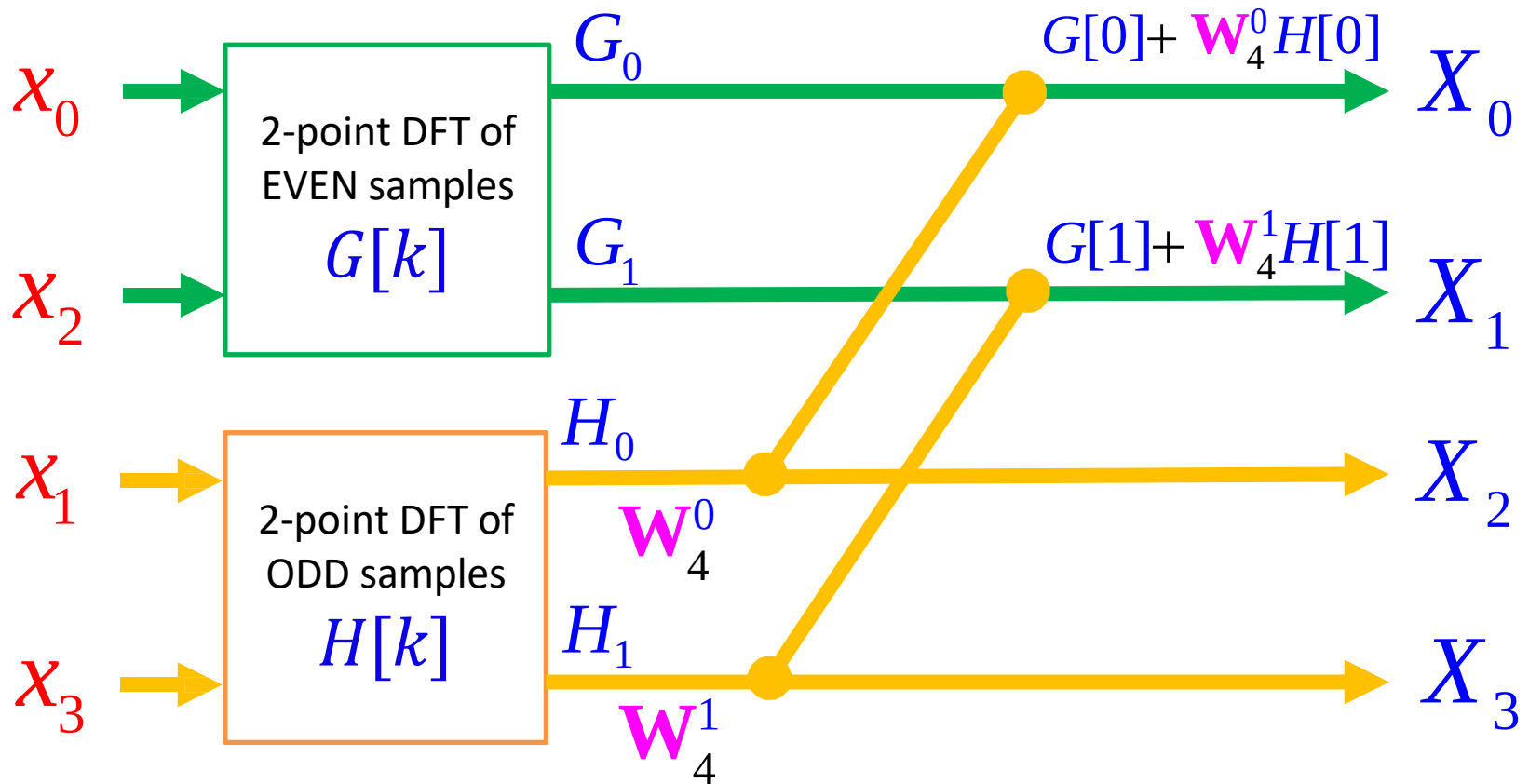
4-point DITFFT $X[k] = G[k] + \mathbf{W}_4^k H[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



4-point DITFFT $X[k] = G[k] + \mathbf{W}_4^k H[k]$

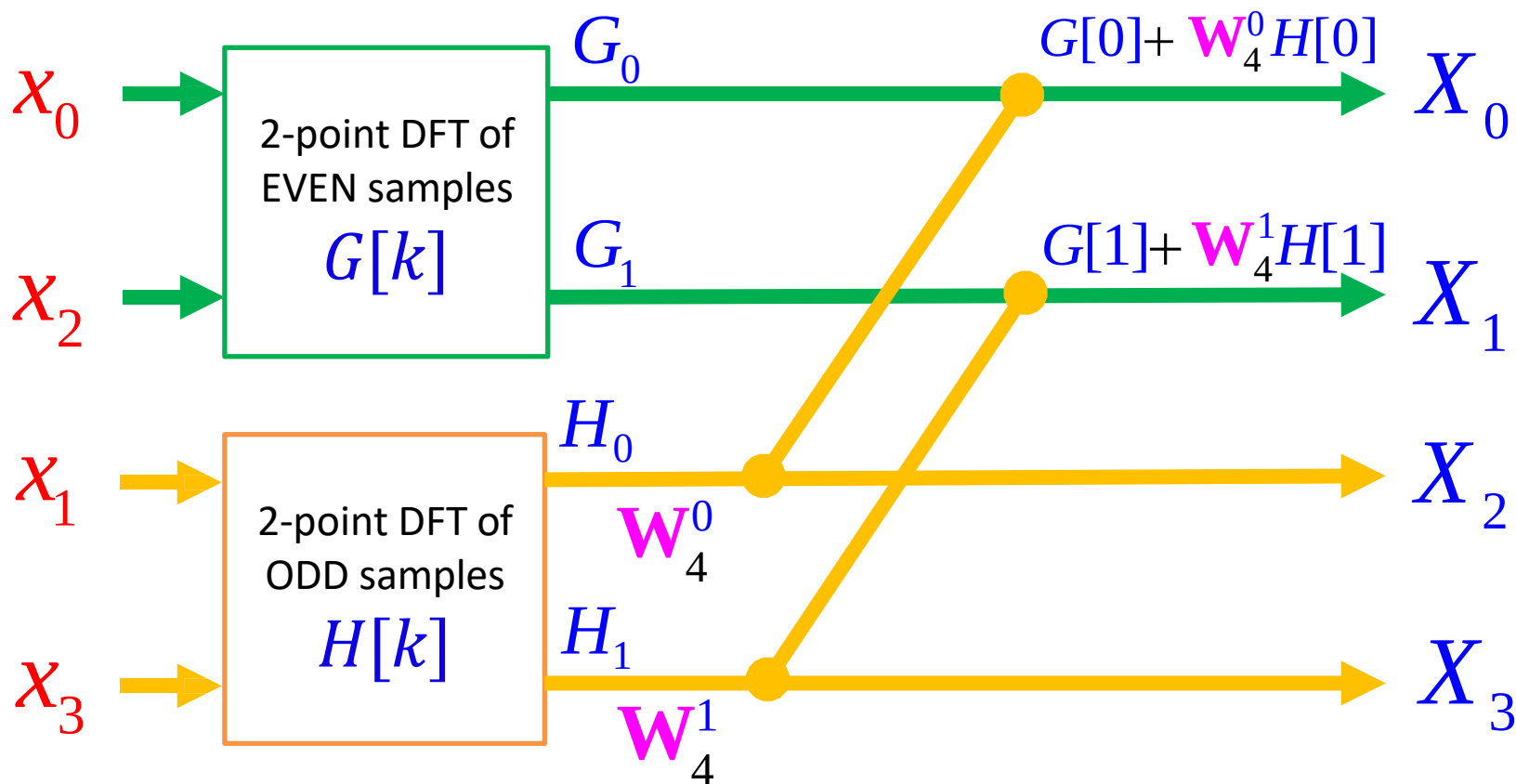
- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



4-point DITFFT

$$X[k] = G[k] - \mathbf{W}_4^k H[k]$$

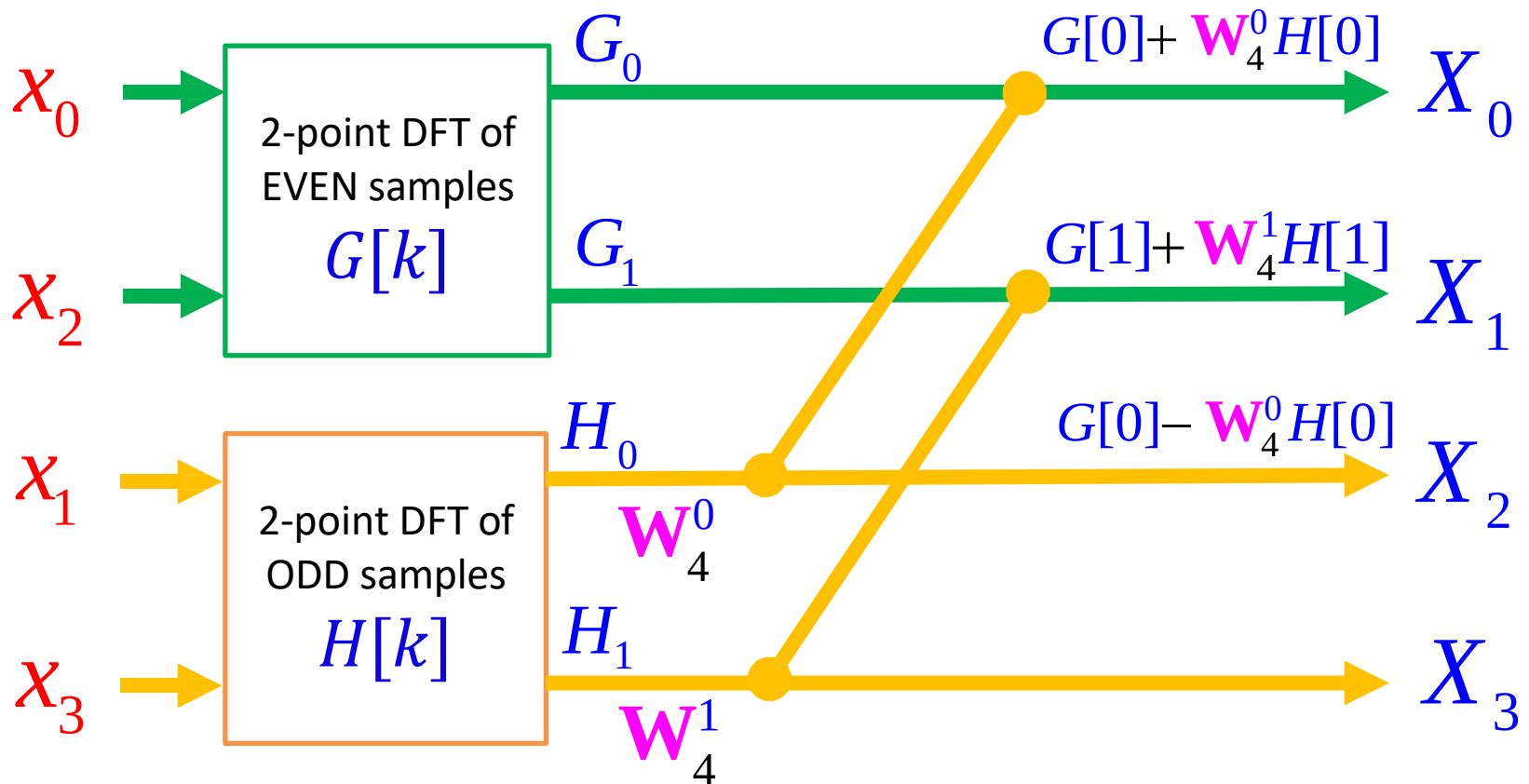
- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



4-point DITFFT

$$X[k] = G[k] - \mathbf{W}_4^k H[k]$$

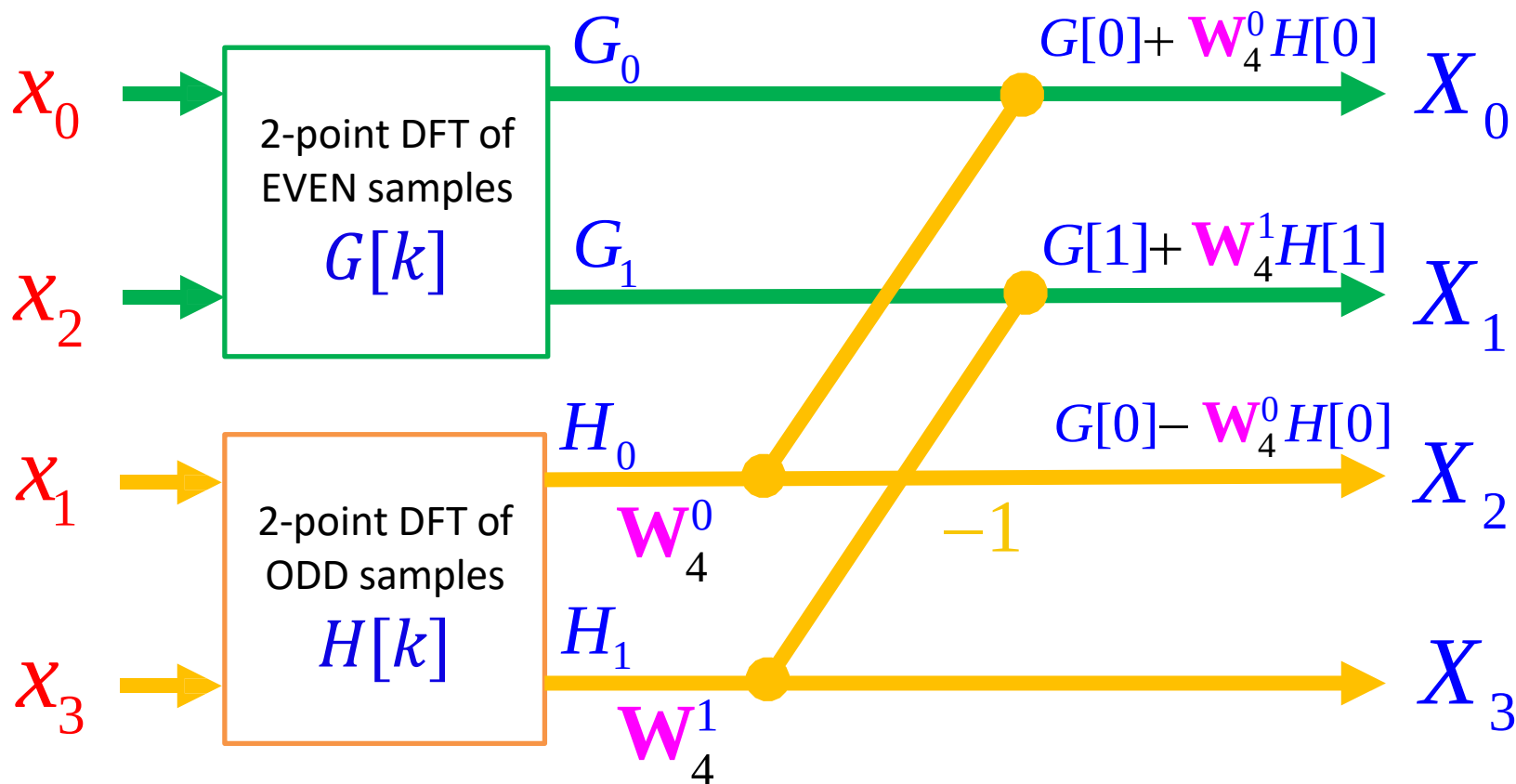
- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



4-point DITFFT

$$X[k] = G[k] - \mathbf{W}_4^k H[k]$$

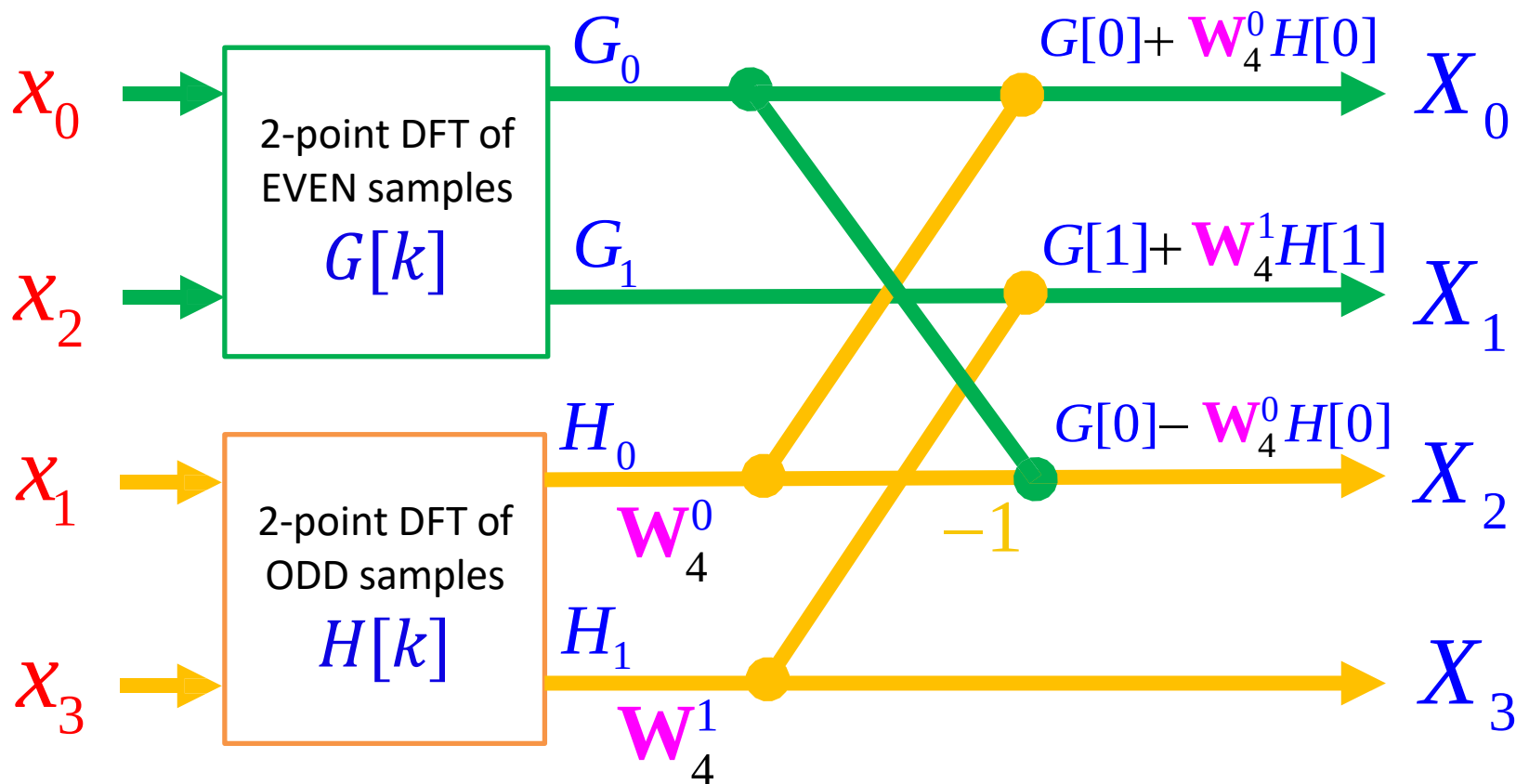
- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



4-point DITFFT

$$X[k] = G[k] - \mathbf{W}_4^k H[k]$$

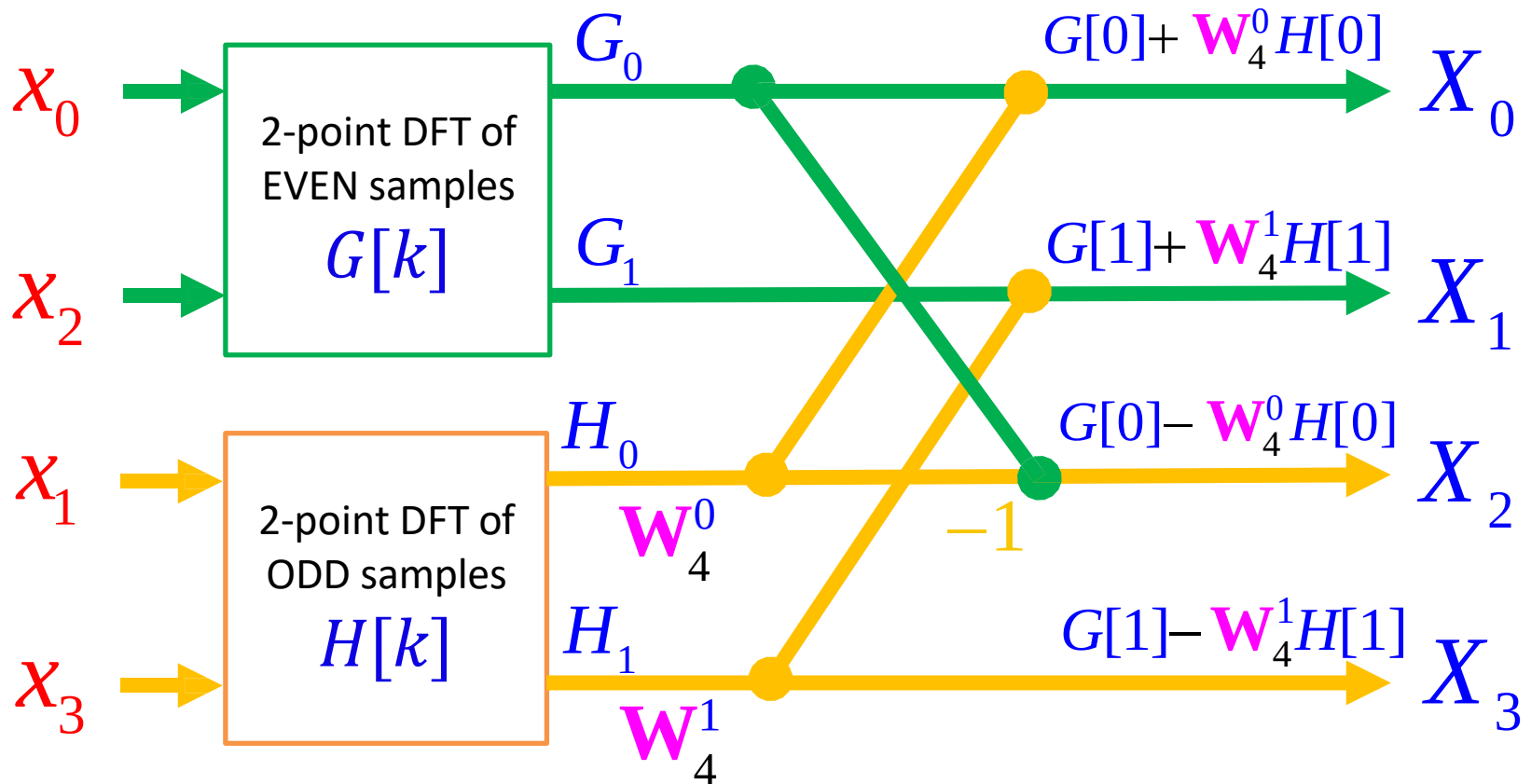
- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



4-point DITFFT

$$X[k] = G[k] - \mathbf{W}_4^k H[k]$$

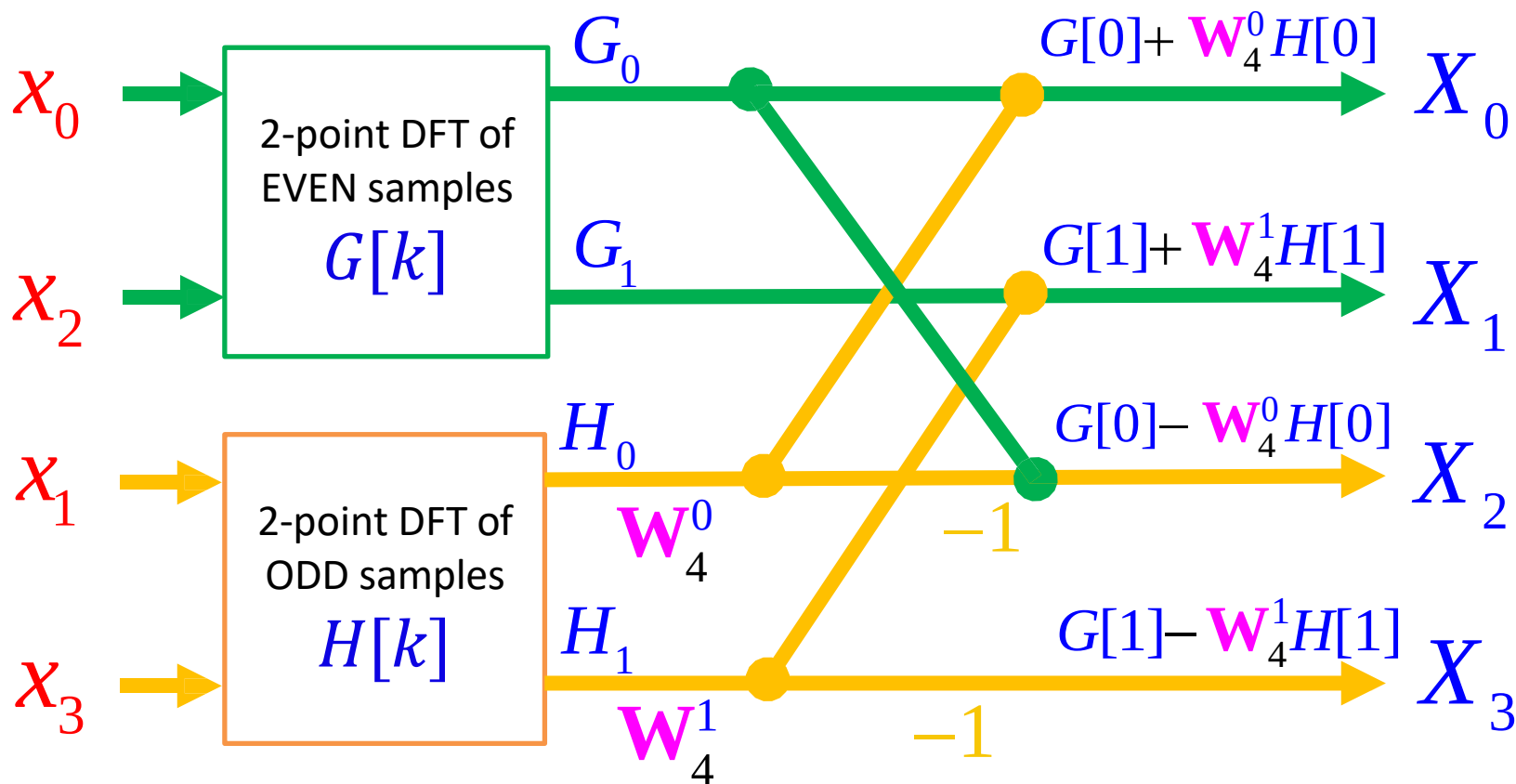
- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



4-point DITFFT

$$X[k] = G[k] - \mathbf{W}_4^k H[k]$$

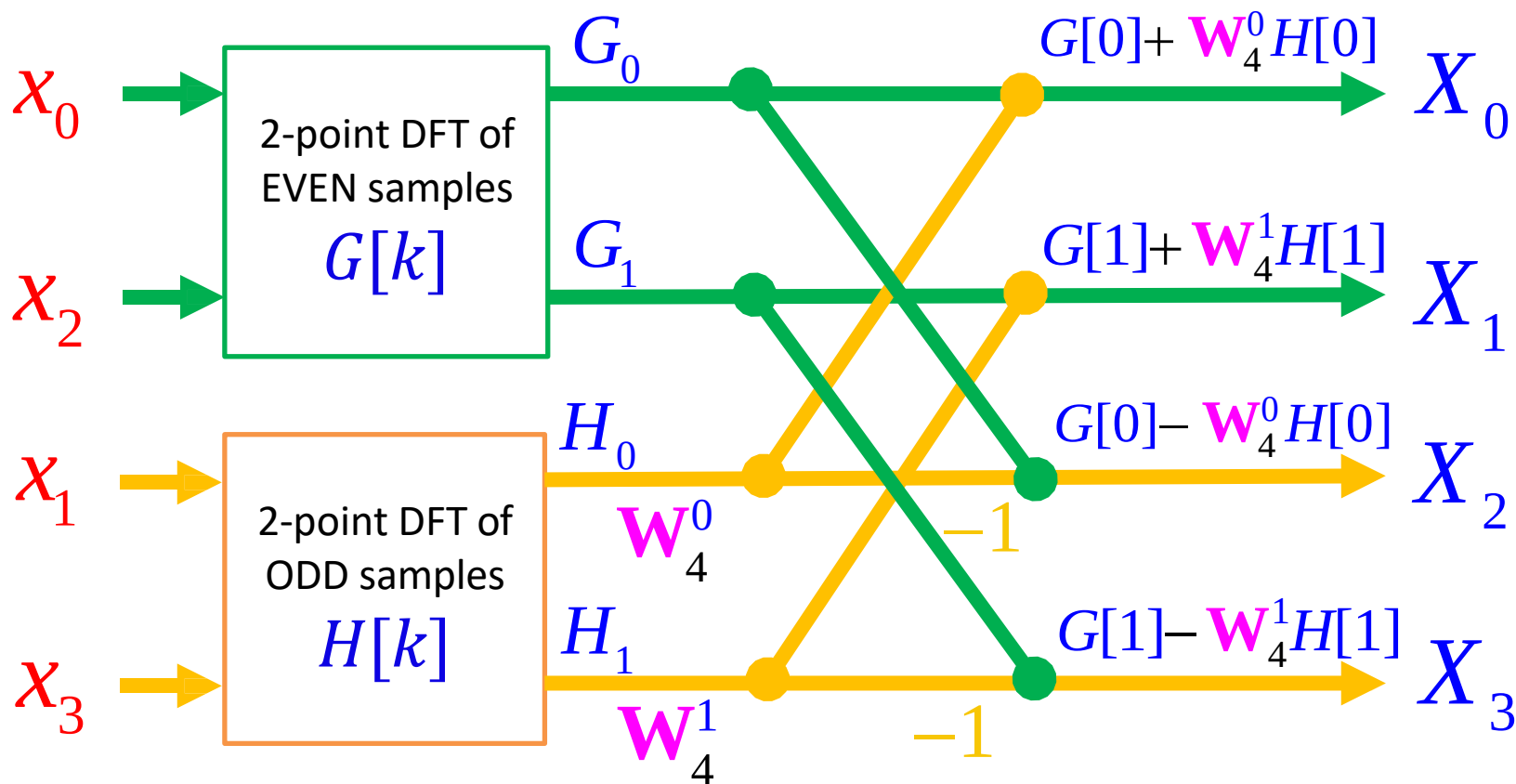
- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



4-point DITFFT

$$X[k] = G[k] - \mathbf{W}_4^k H[k]$$

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



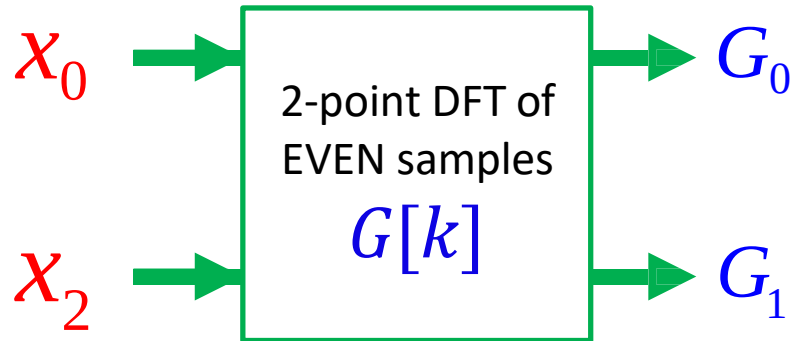
4-point DITFFT

4-point DITFFT

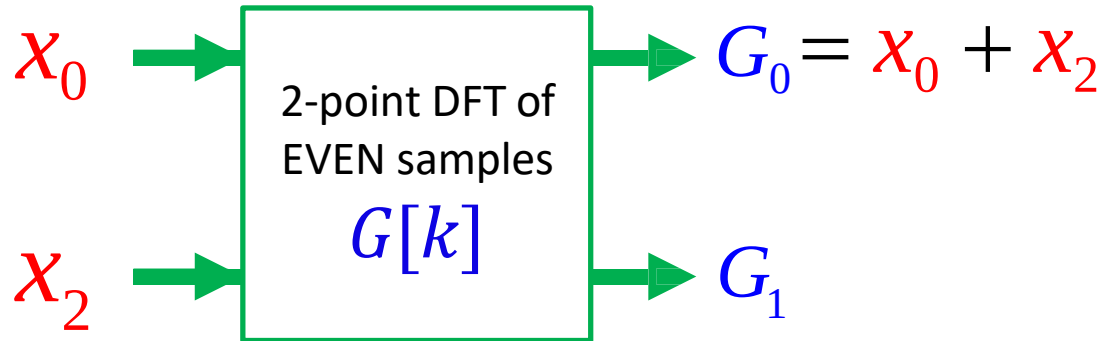
x_0

x_2

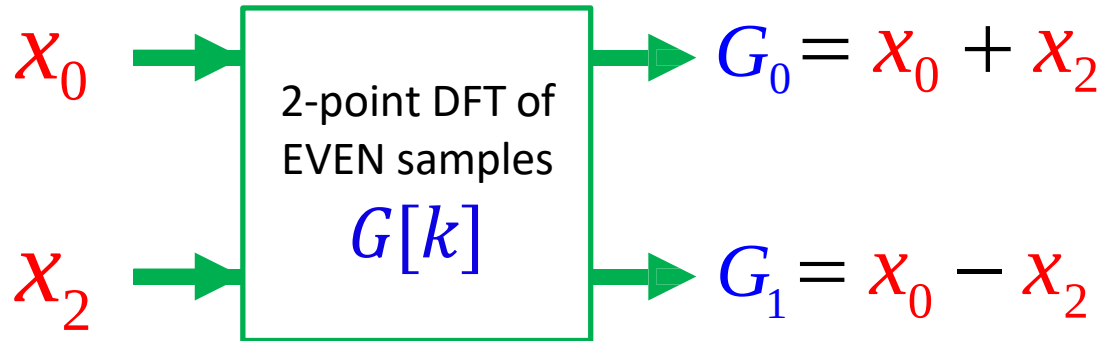
4-point DITFFT



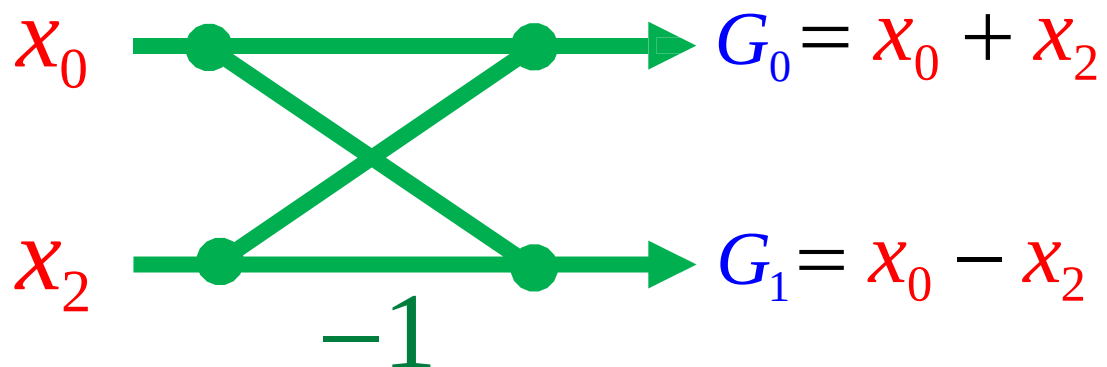
4-point DITFFT



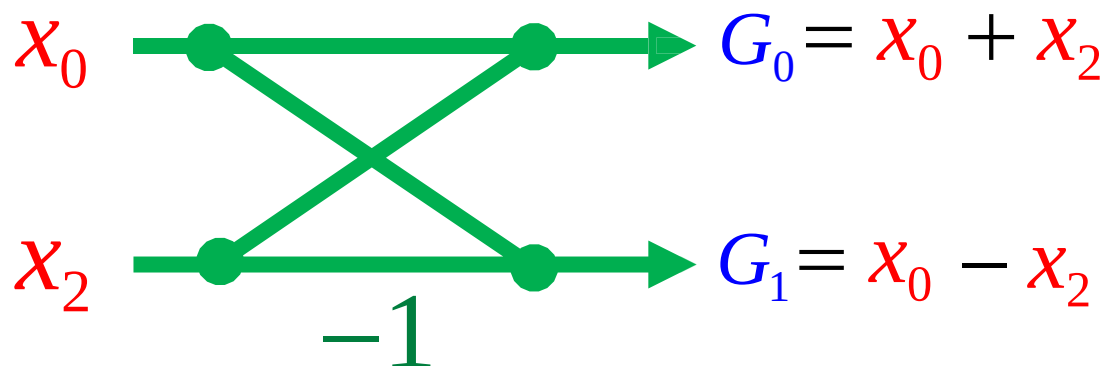
4-point DITFFT



4-point DITFFT



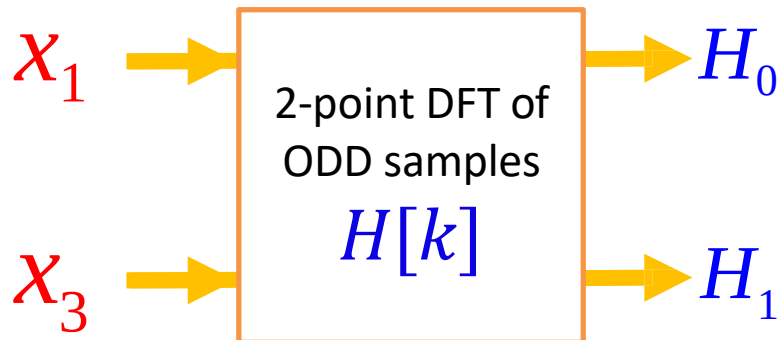
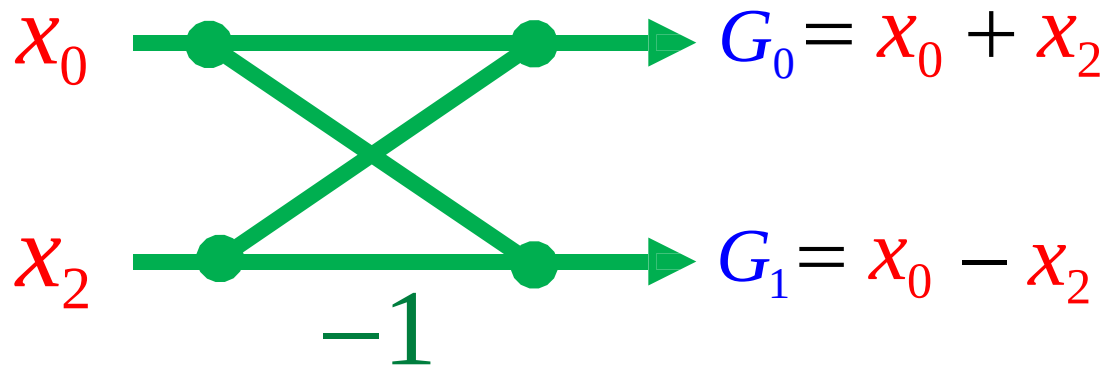
4-point DITFFT



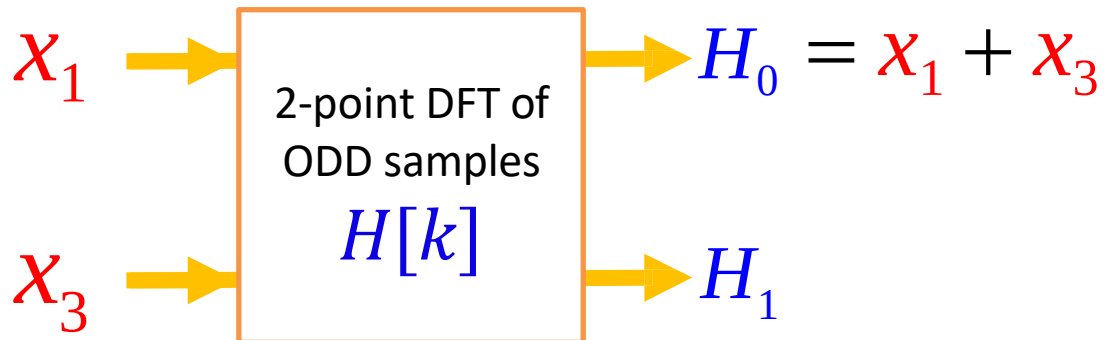
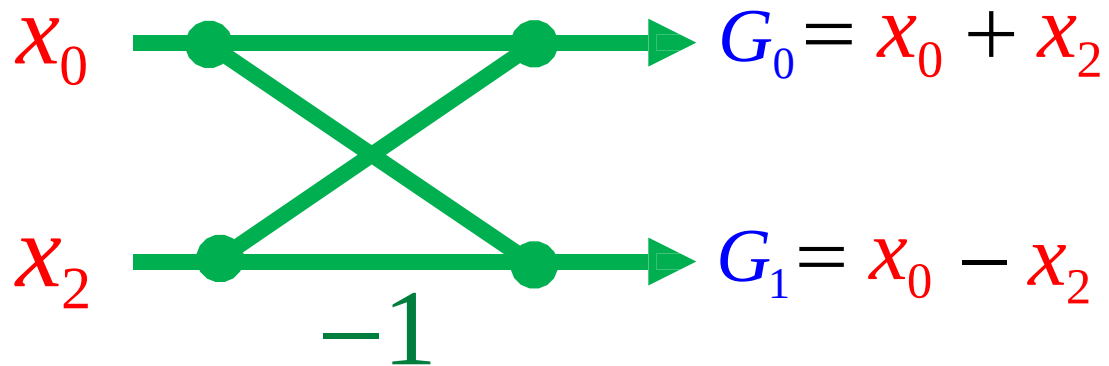
x_1

x_3

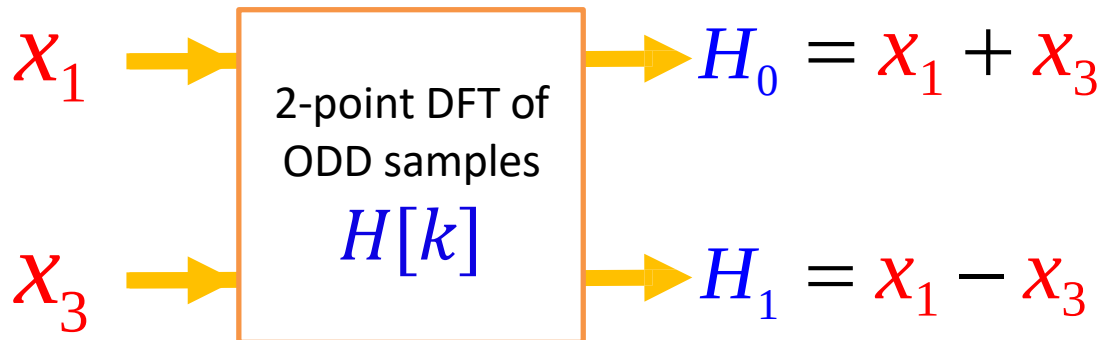
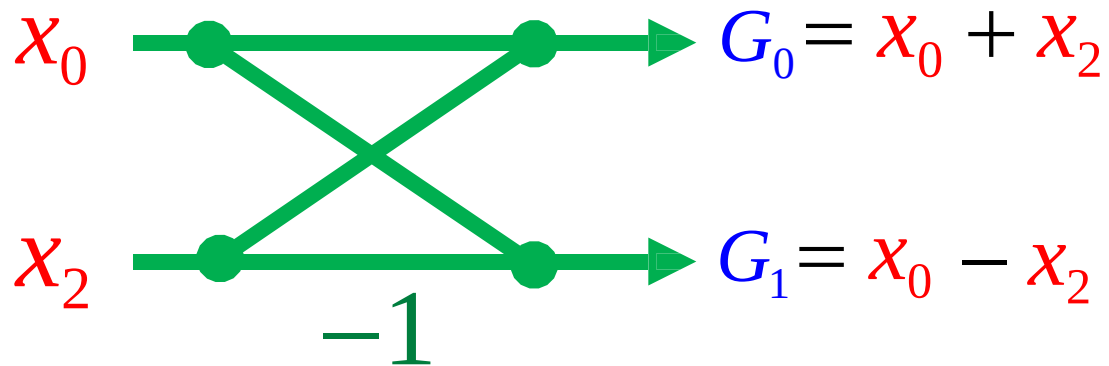
4-point DITFFT



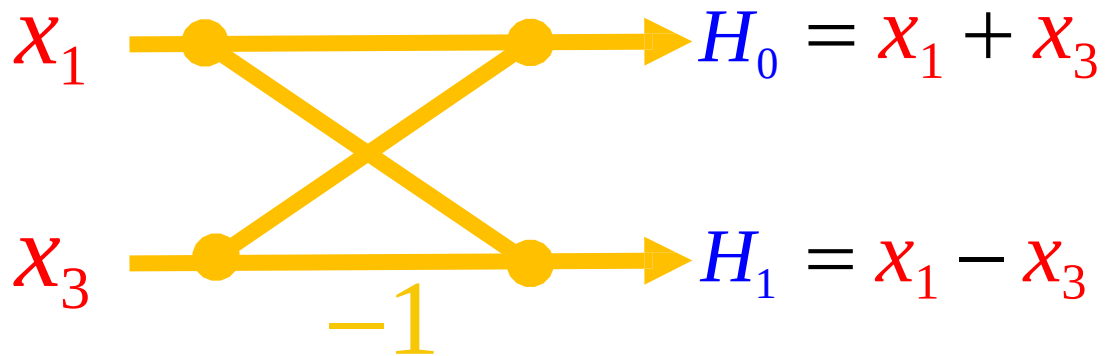
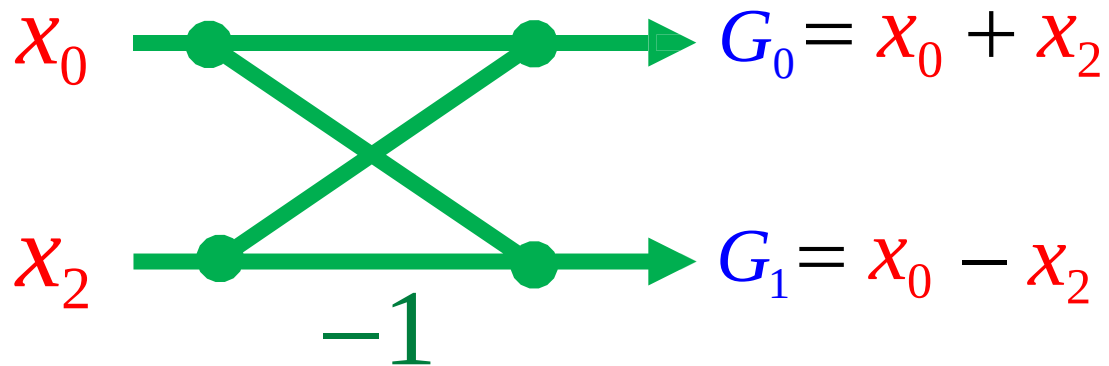
4-point DITFFT



4-point DITFFT

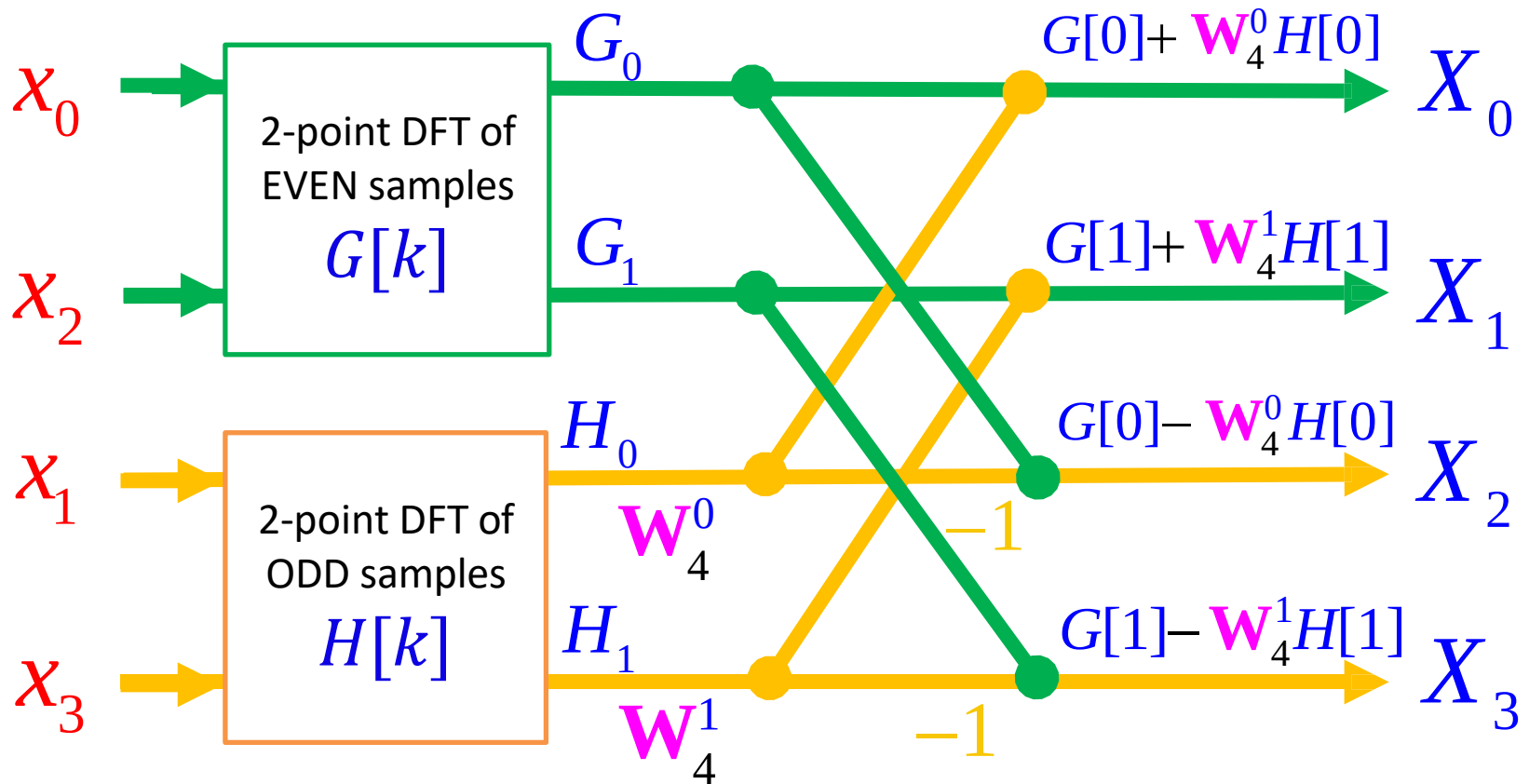


4-point DITFFT



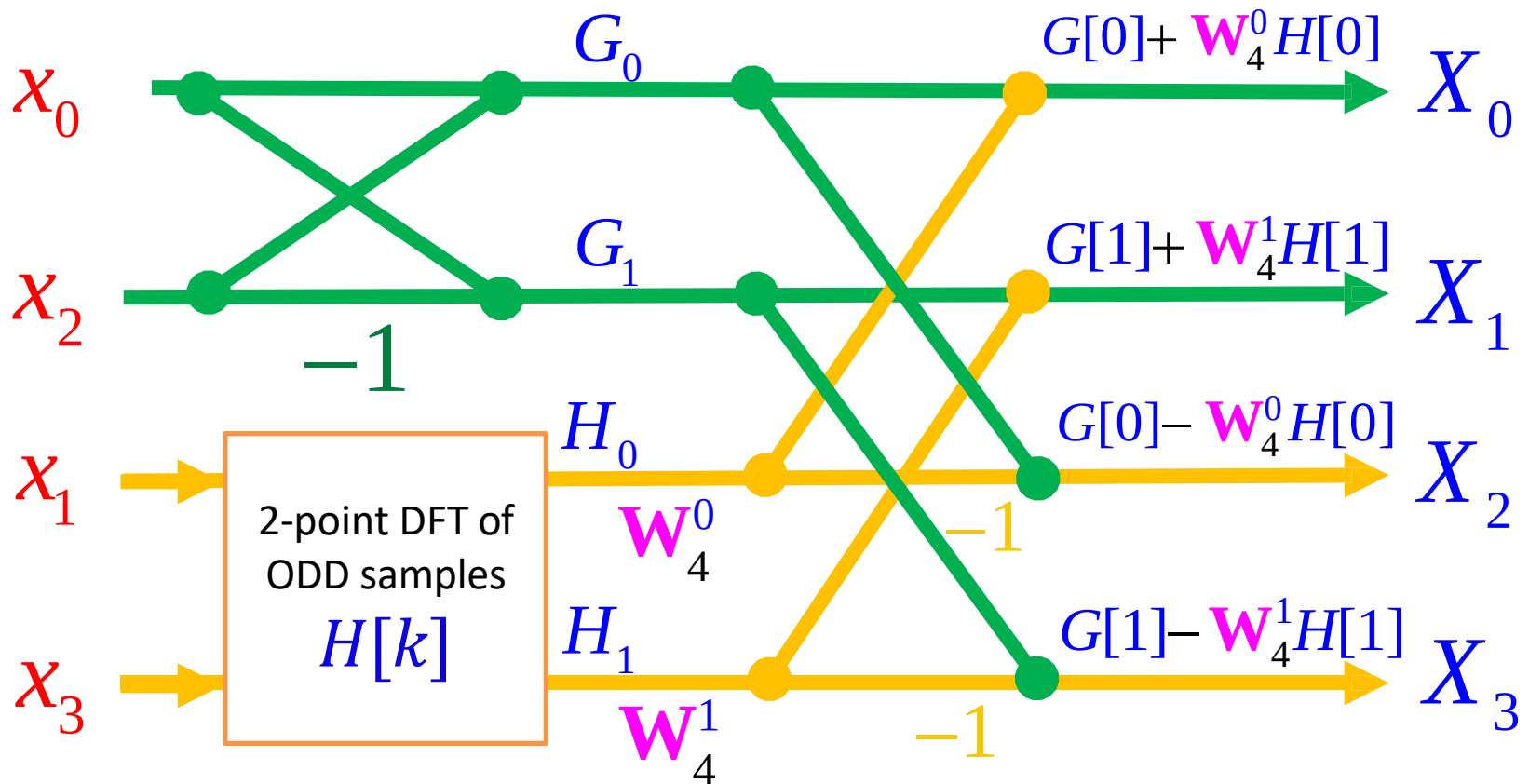
4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



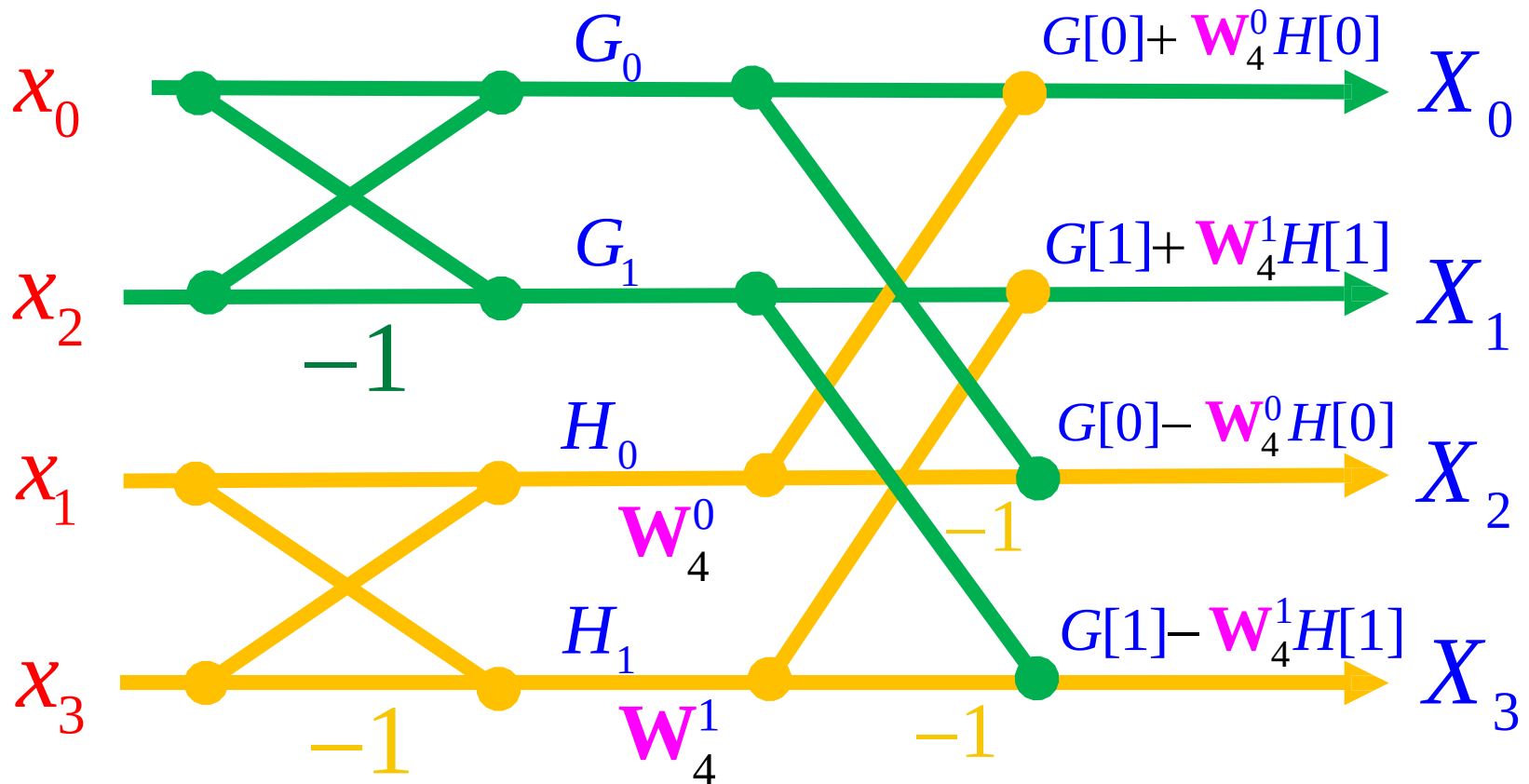
4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



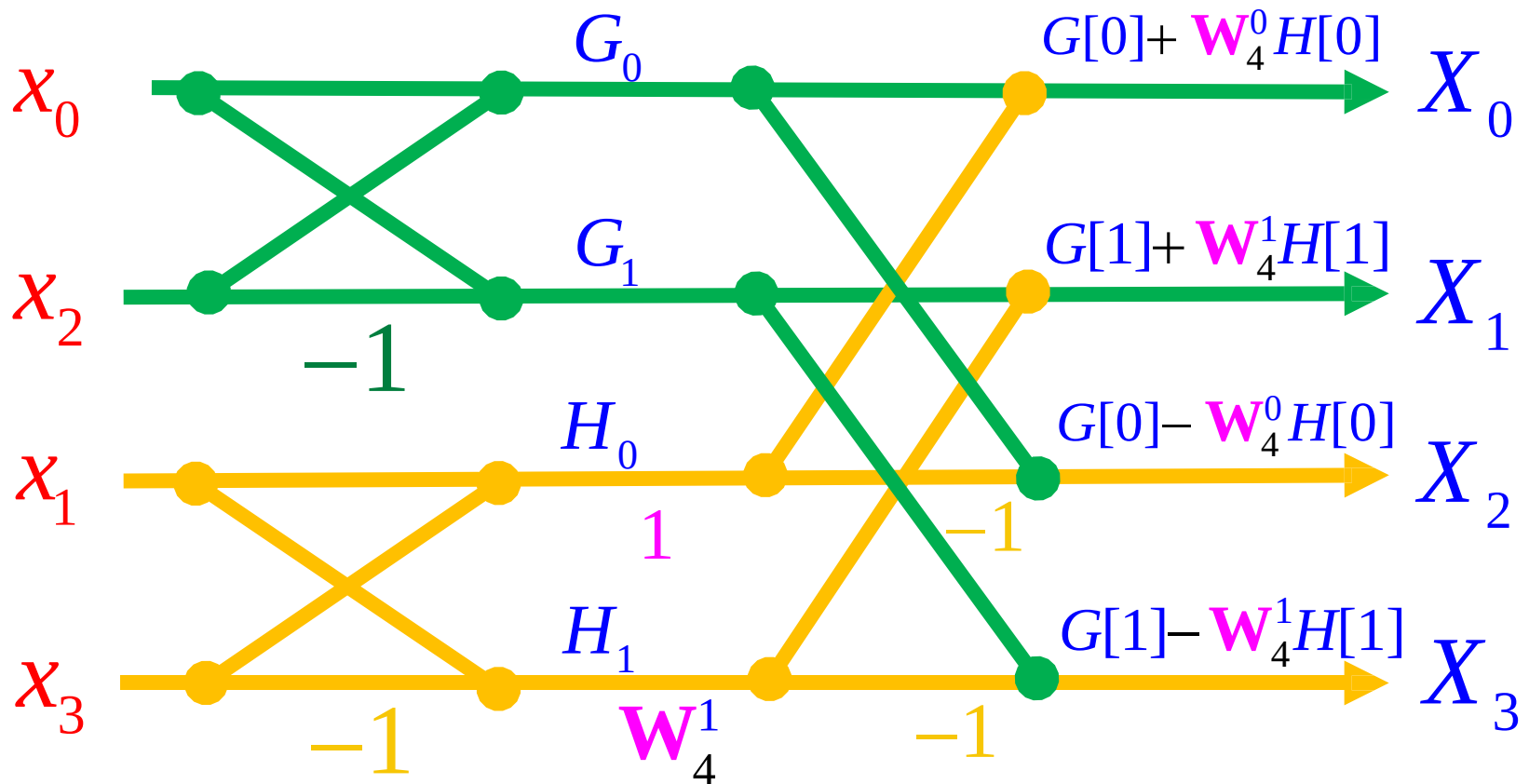
4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



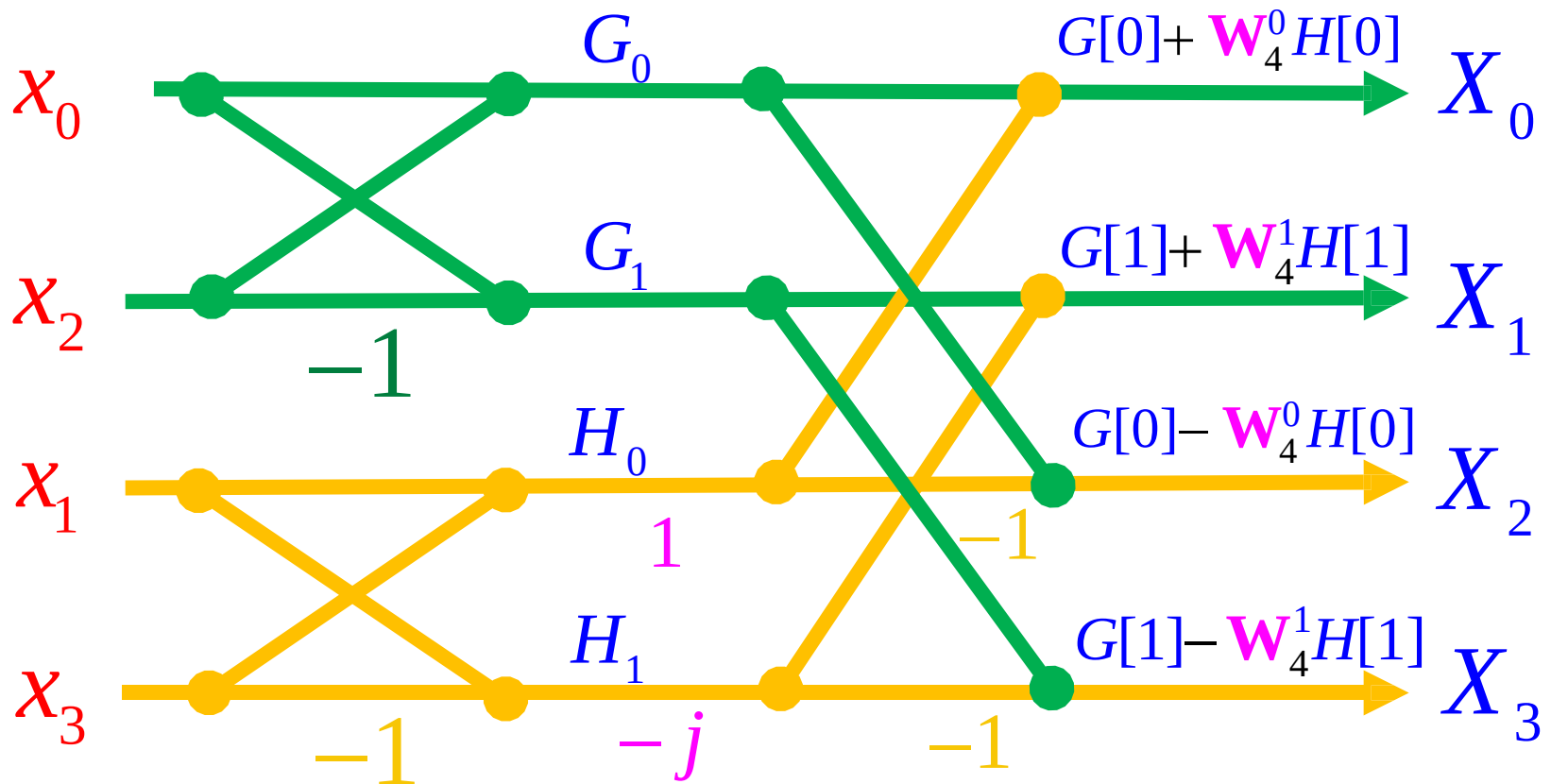
4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



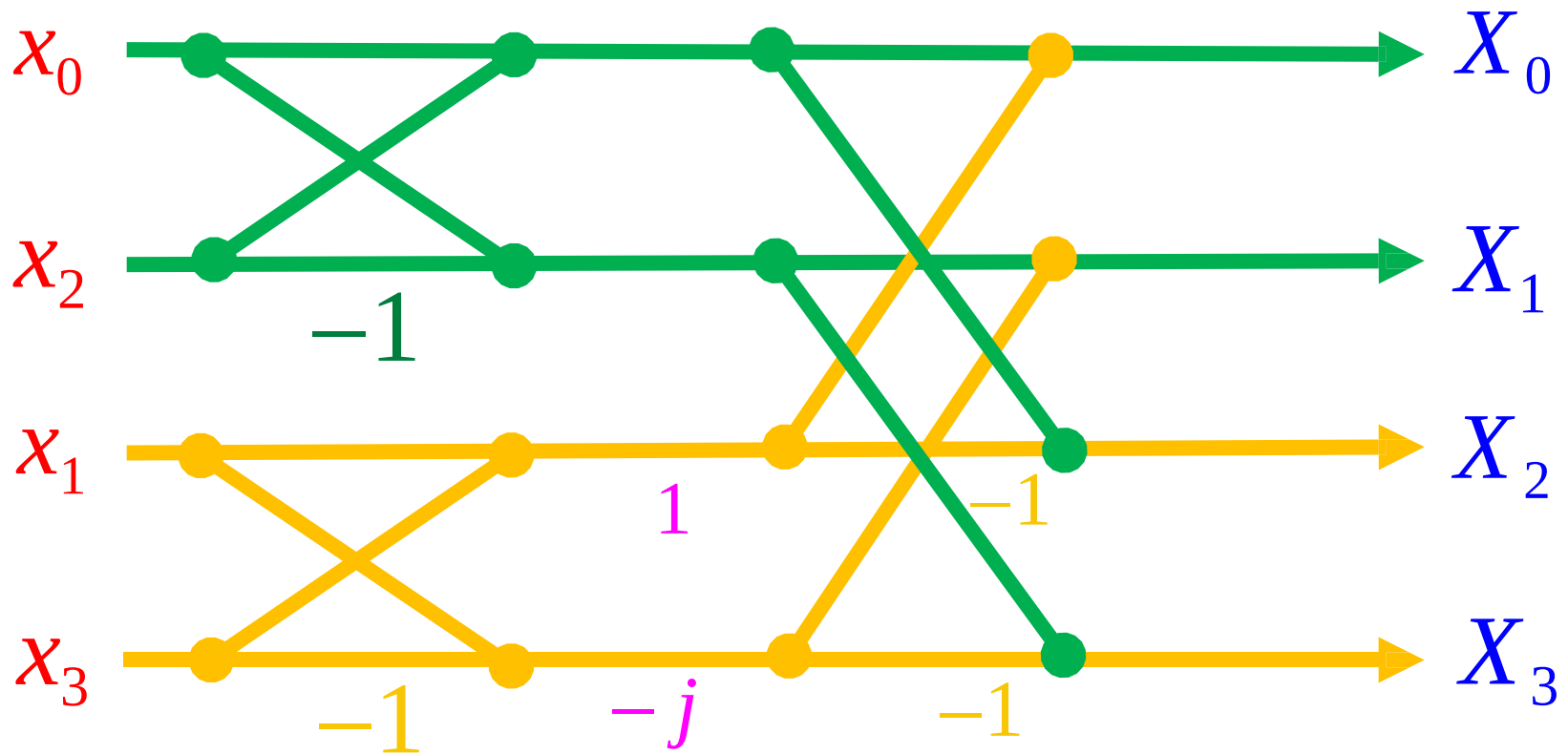
4-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\}$ and corresponding DFT $X(k) = \{X_0, X_1, X_2, X_3\}$



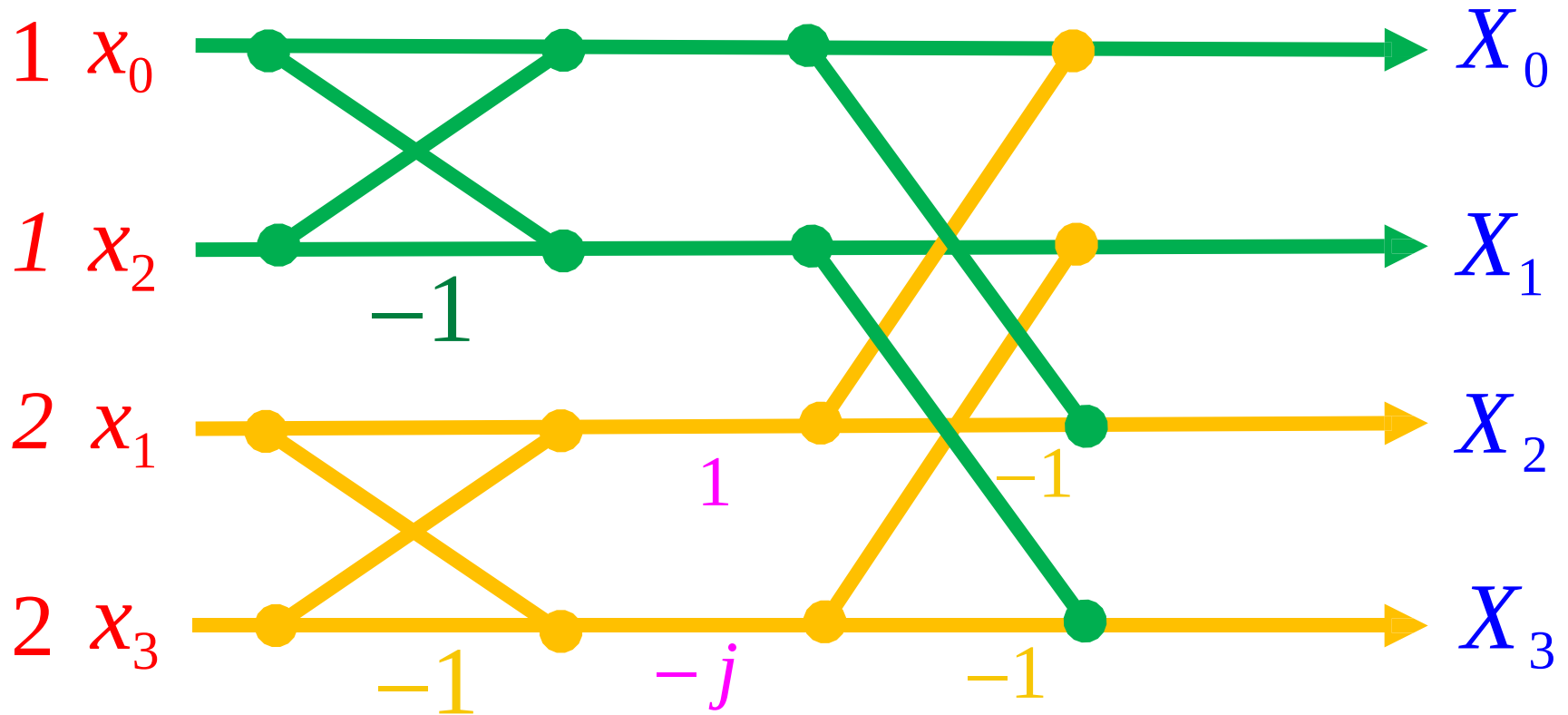
Example-9

- Find DFT of $x[n] = \{1, 2, 1, 2\}$ using DITFFT (DEC18, 4marks)



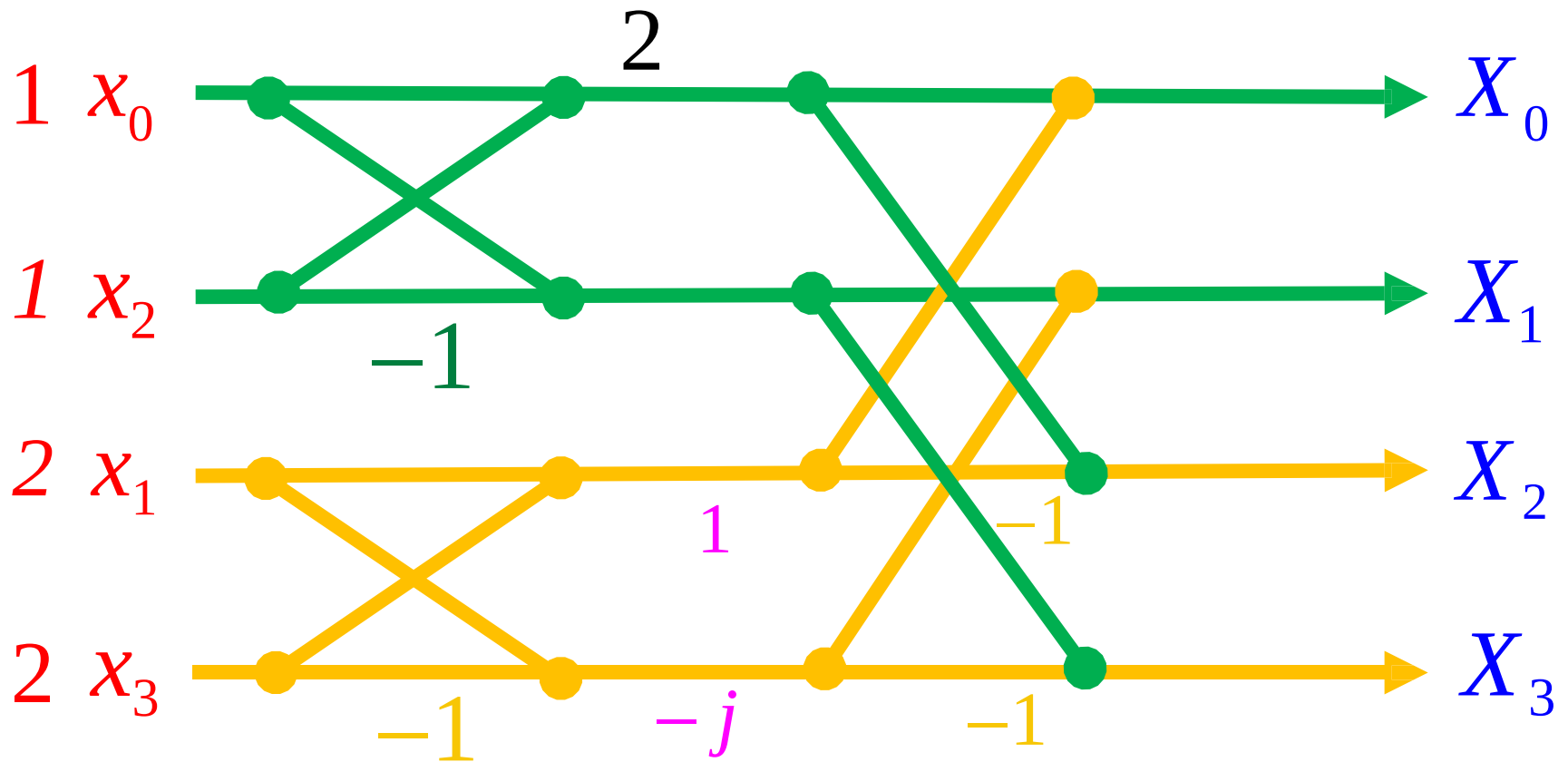
Example-9

- Find DFT of $x[n] = \{1, 2, 1, 2\}$ using DITFFT (DEC18, 4marks)



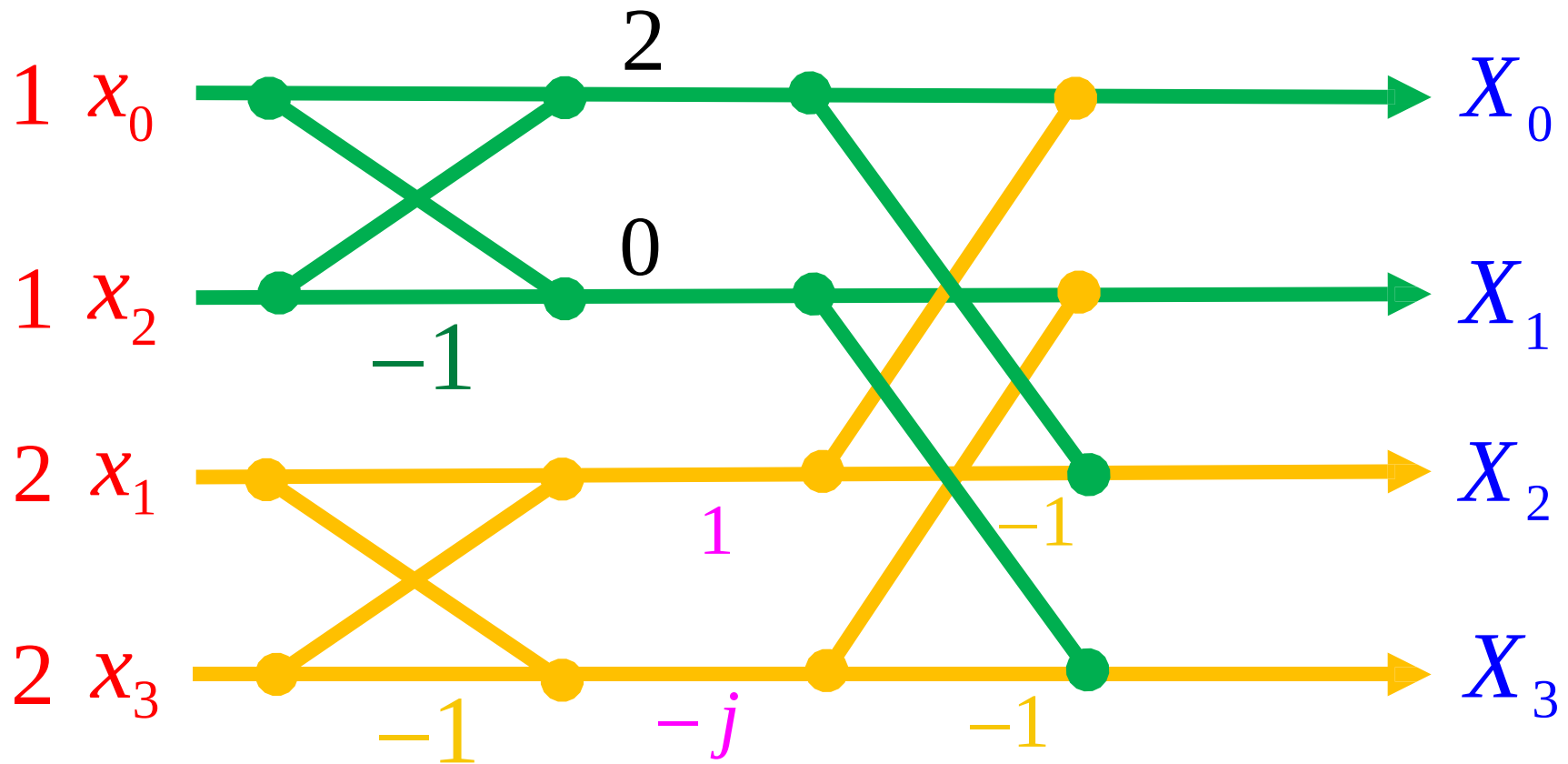
Example-9

- Find DFT of $x[n] = \{1, 2, 1, 2\}$ using DITFFT (DEC18, 4marks)



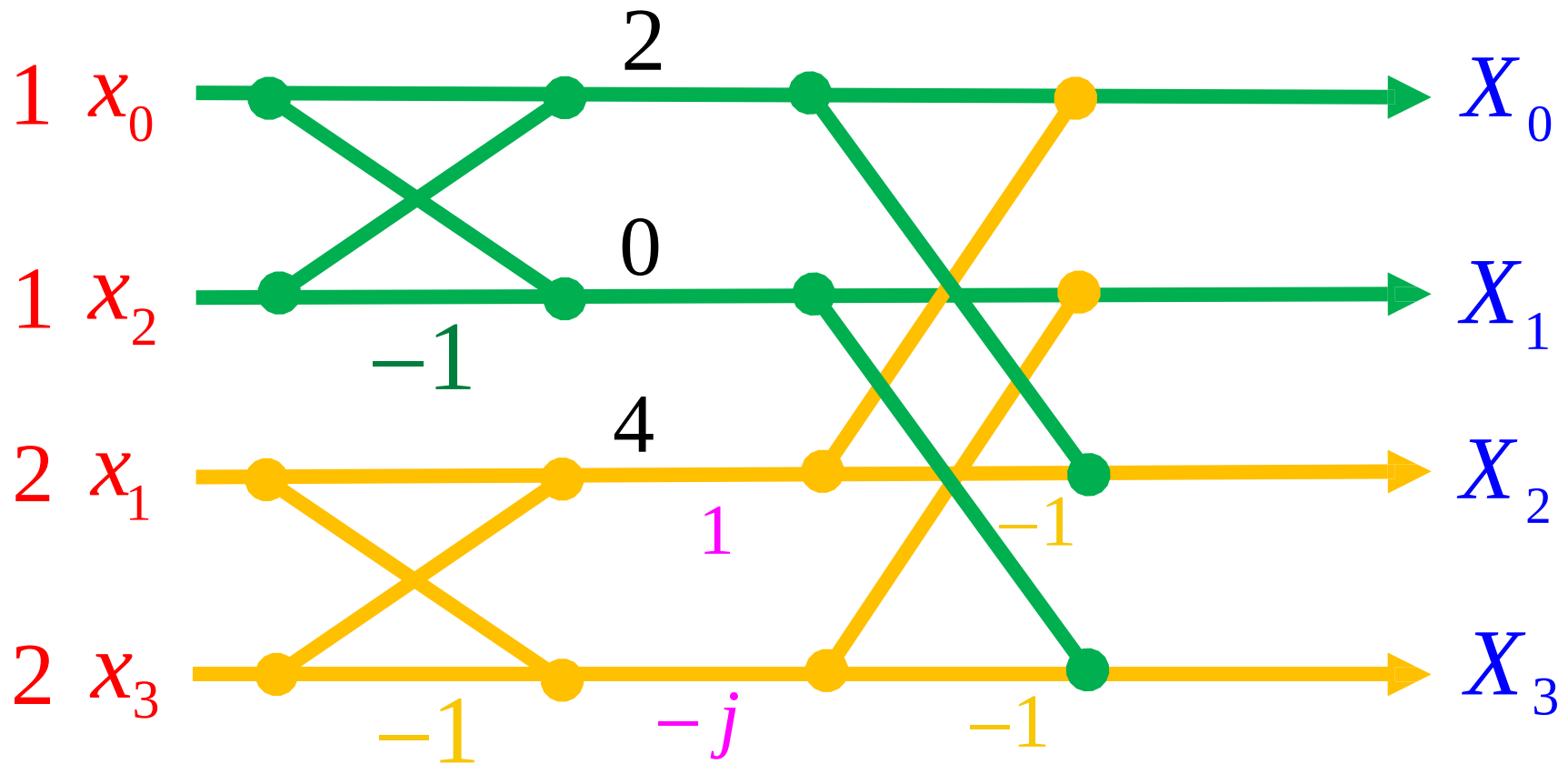
Example-9

- Find DFT of $x[n] = \{1, 2, 1, 2\}$ using DITFFT (DEC18, 4marks)



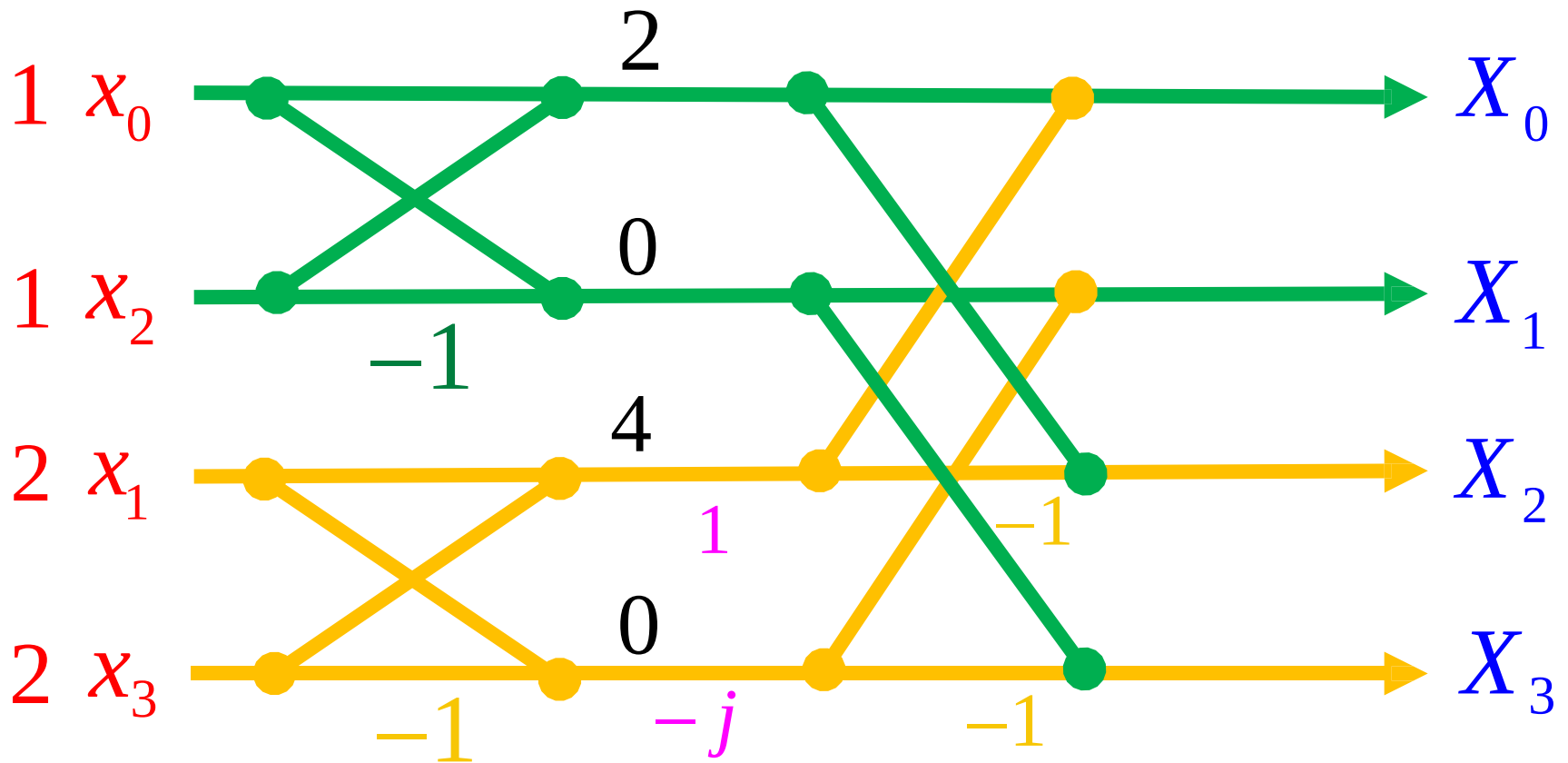
Example-9

- Find DFT of $x[n] = \{1, 2, 1, 2\}$ using DITFFT (DEC18, 4marks)



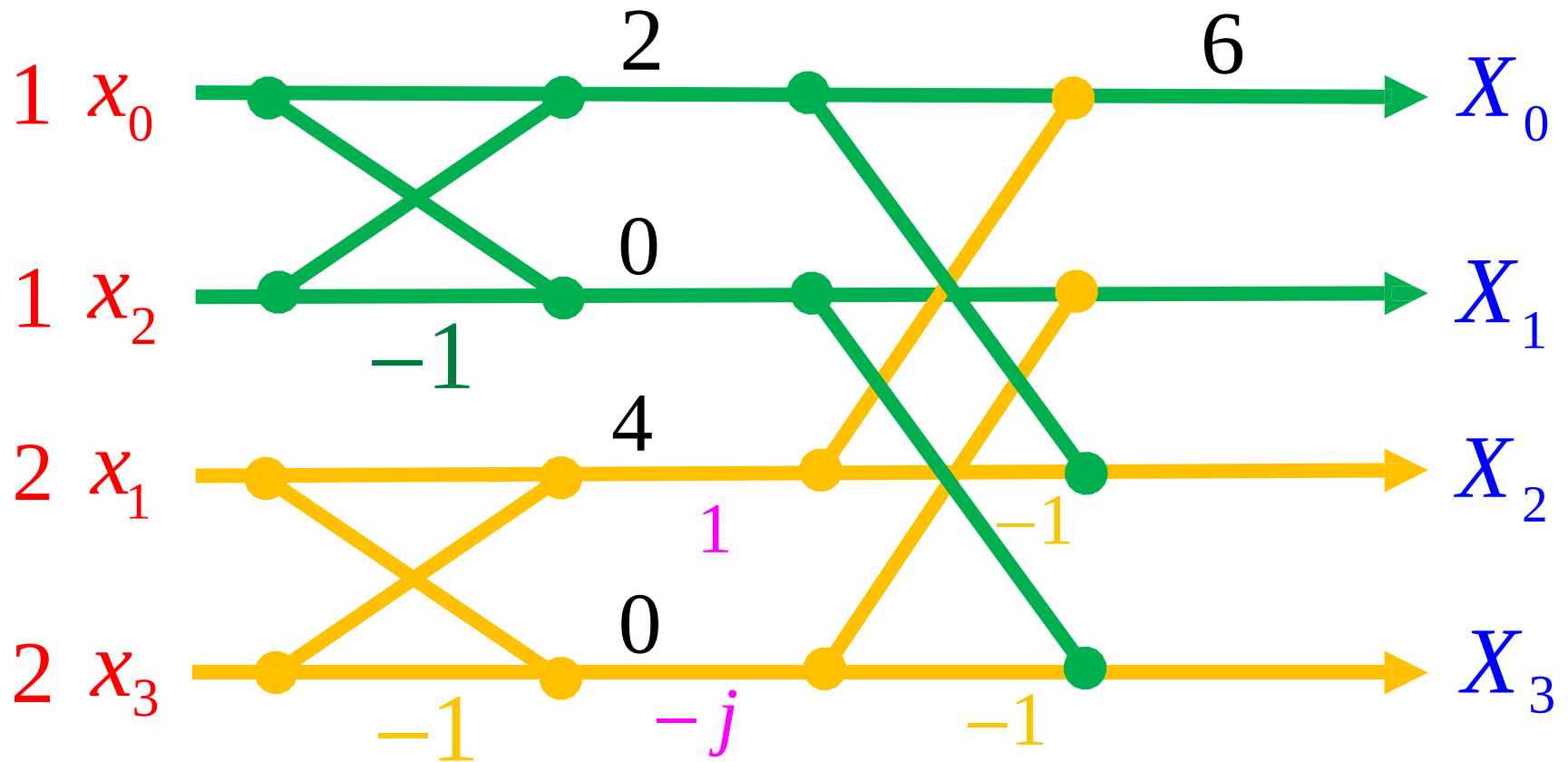
Example-9

- Find DFT of $x[n] = \{1, 2, 1, 2\}$ using DITFFT (DEC18, 4marks)



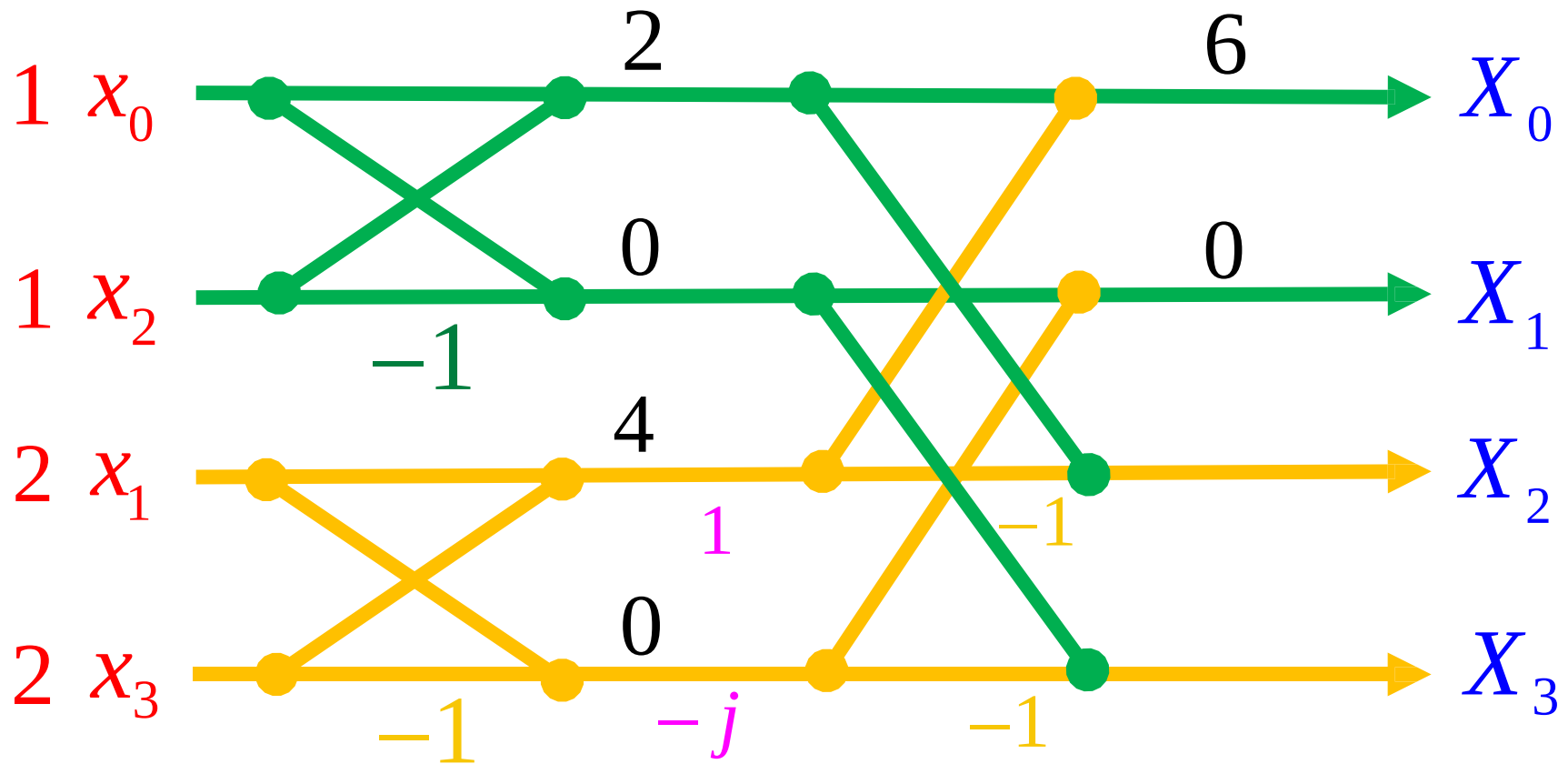
Example-9

- Find DFT of $x[n] = \{1, 2, 1, 2\}$ using DITFFT (DEC18, 4marks)



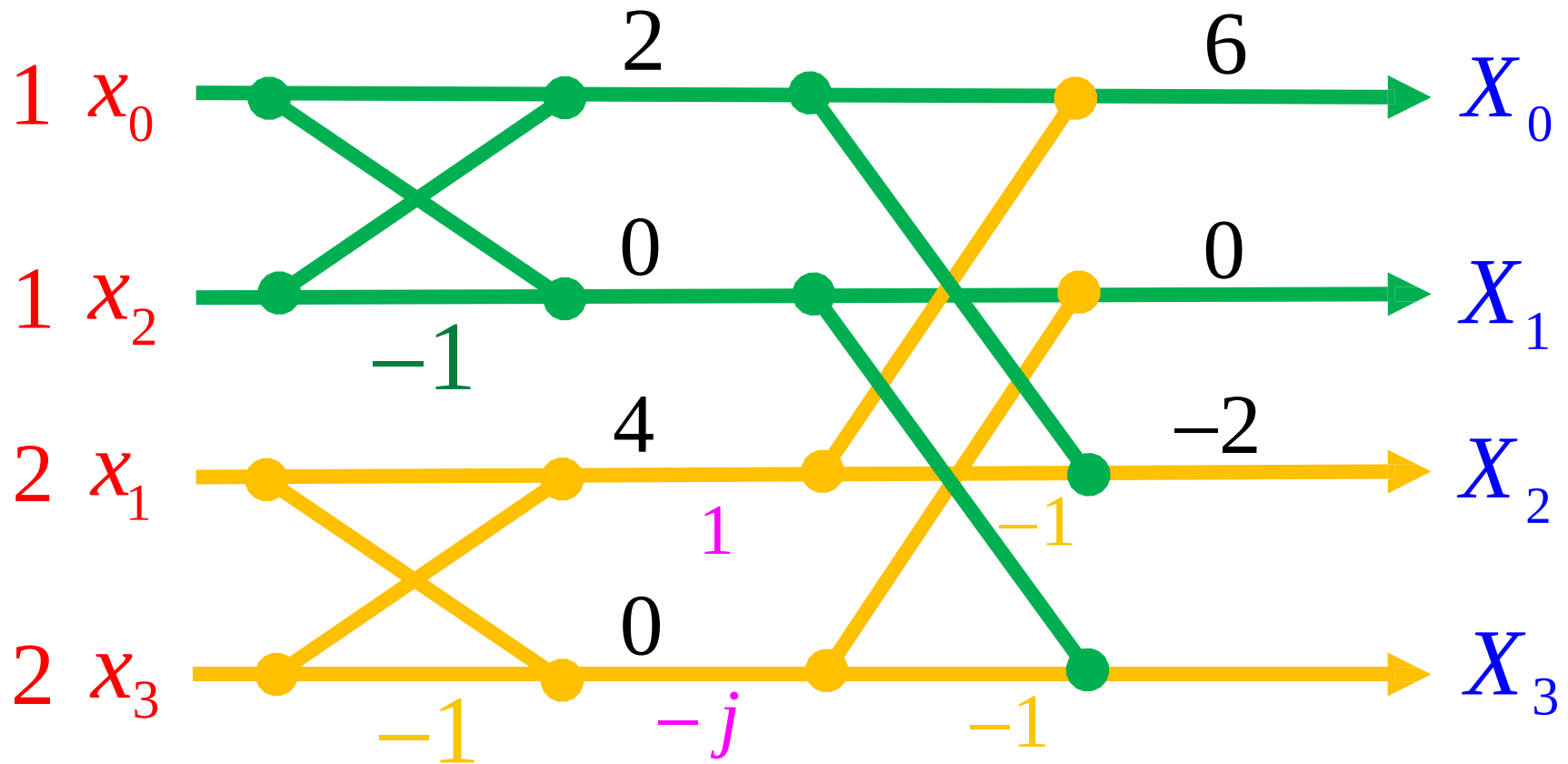
Example-9

- Find DFT of $x[n] = \{1, 2, 1, 2\}$ using DITFFT (DEC18, 4marks)



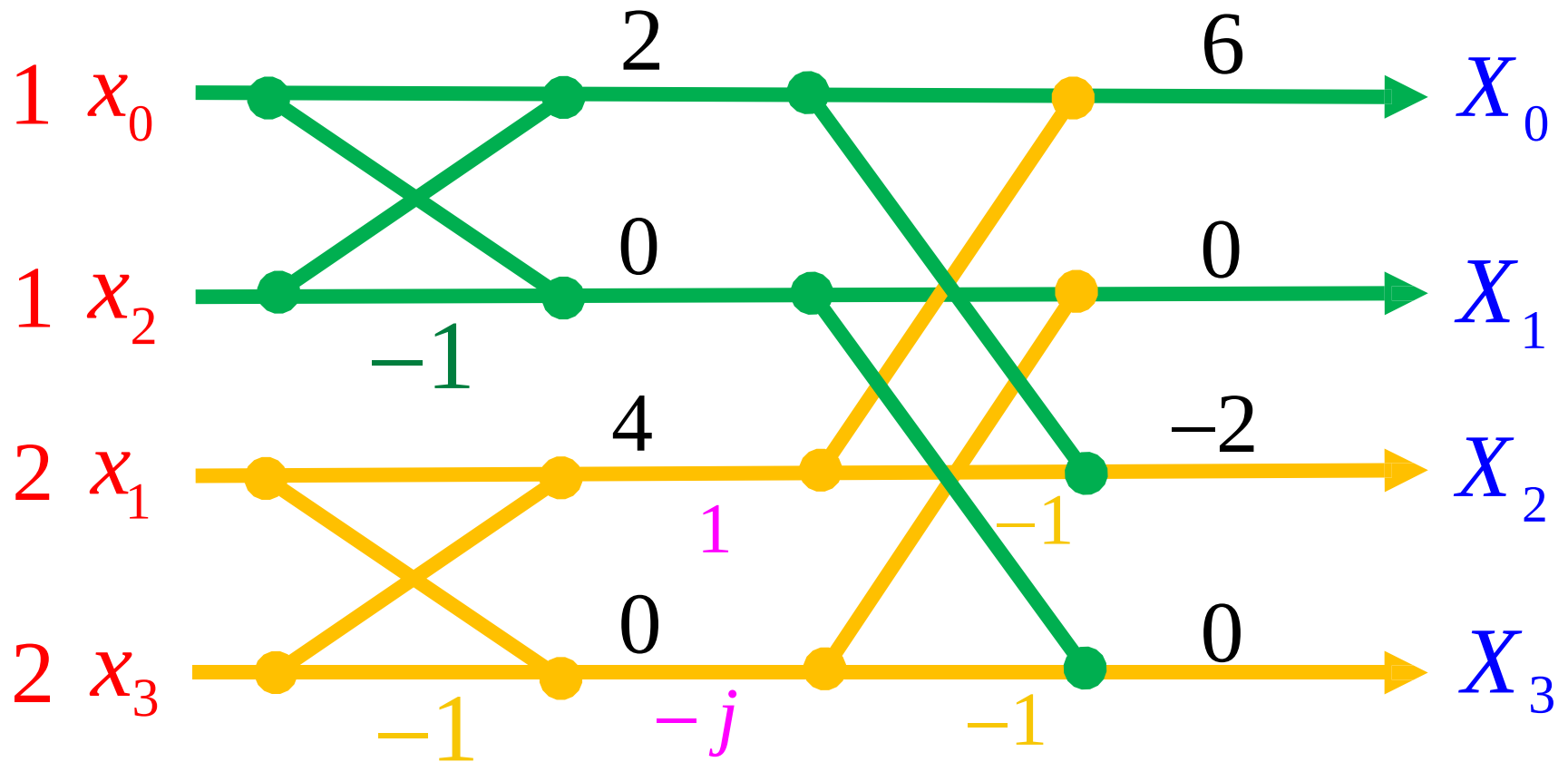
Example-9

- Find DFT of $x[n] = \{1, 2, 1, 2\}$ using DITFFT (DEC18, 4marks)



Example-9

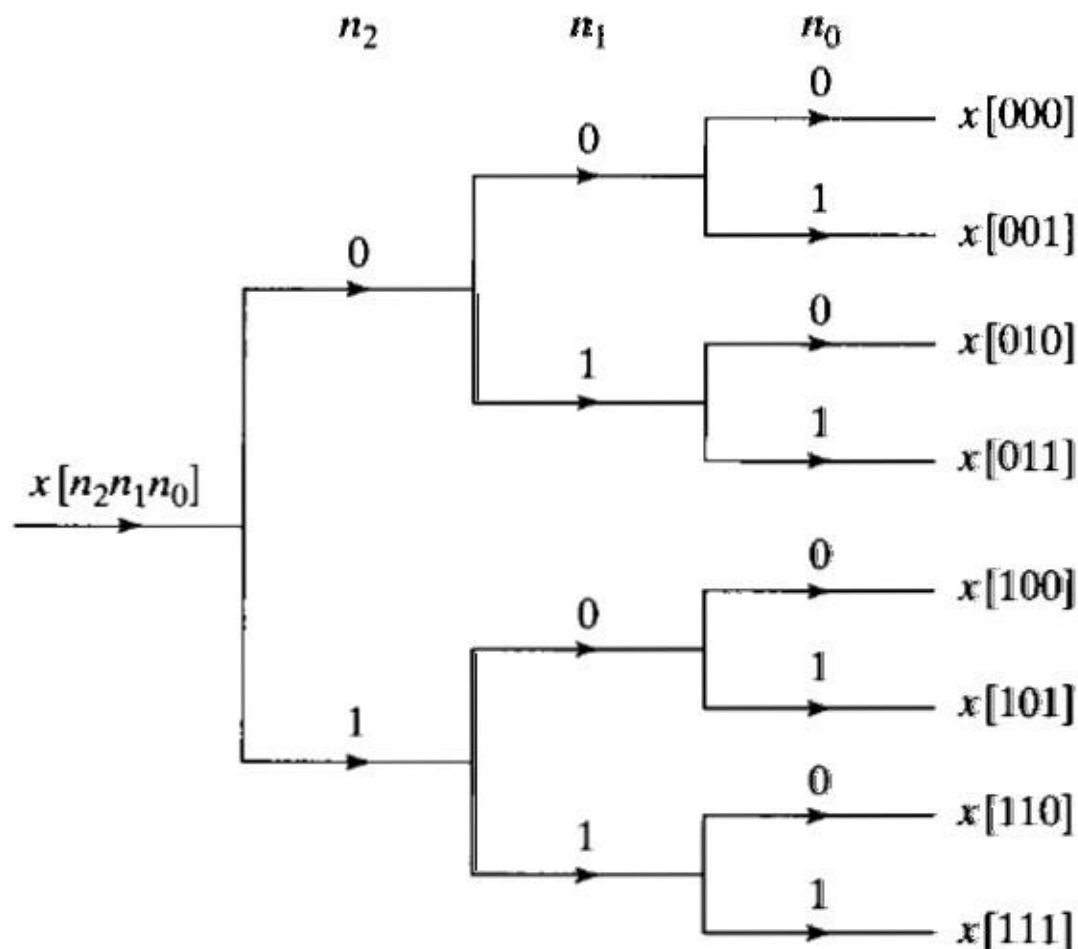
- Find DFT of $x[n] = \{1, 2, 1, 2\}$ using DITFFT (DEC18, 4marks)



Decimation In Time FFT (DIT-FFT)



Bit-reversed order



$$X_0[000] = x[000]$$

$$X_0[001] = x[100]$$

$$X_0[010] = x[010]$$

$$X_0[011] = x[110]$$

$$X_0[100] = x[001]$$

$$X_0[101] = x[101]$$

$$X_0[110] = x[011]$$

$$X_0[111] = x[111]$$

Tree diagram depicting normal order sorting [Source: Chapter 9, Digital Signal Processing by Prokakis]

8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$

8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$

x_0

x_4

x_2

x_6

x_1

x_5

x_3

x_7

8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$

x_0	X_0
x_4	X_1
x_2	X_2
x_6	X_3
x_1	X_4
x_5	X_5
x_3	X_6
x_7	X_7

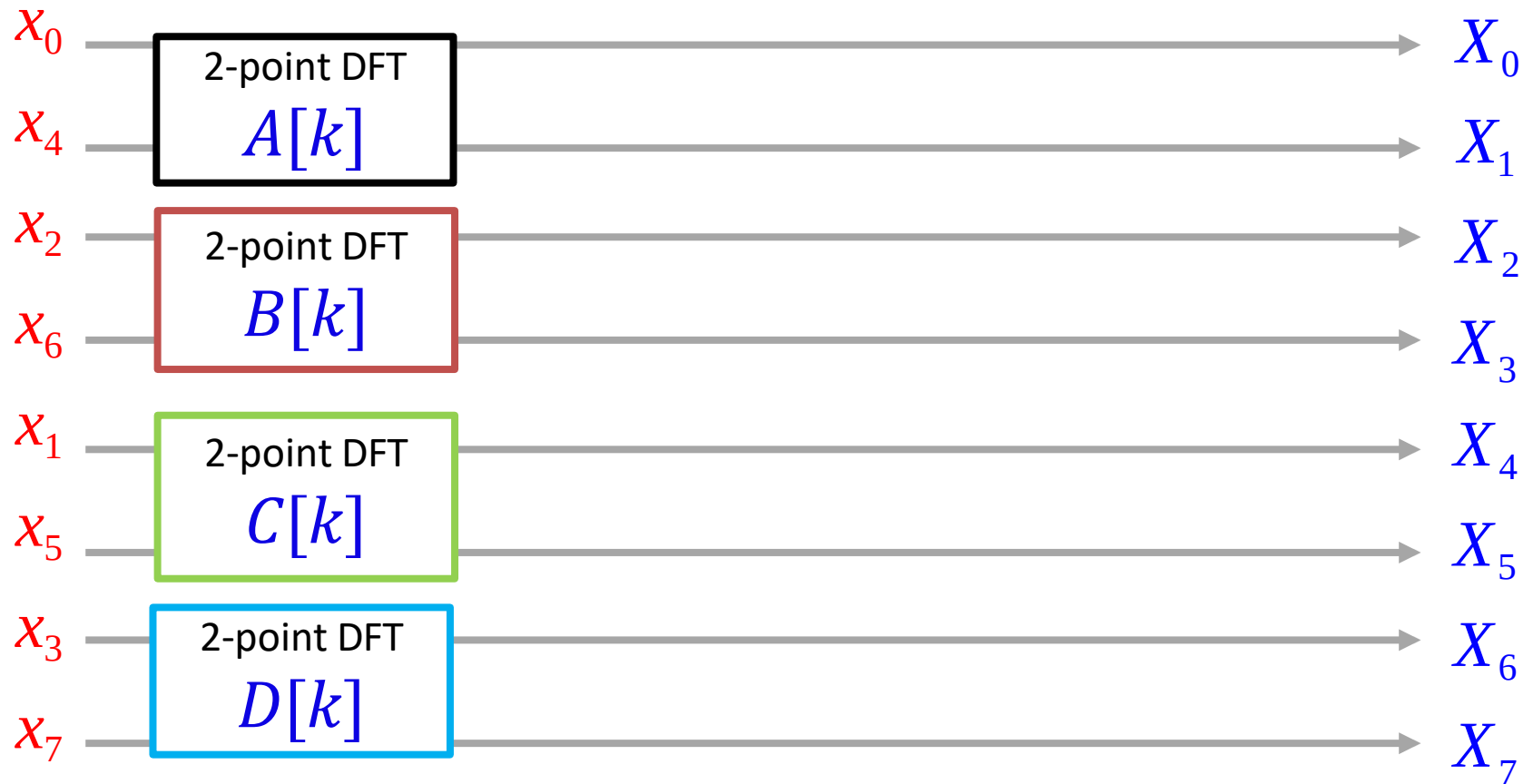
8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



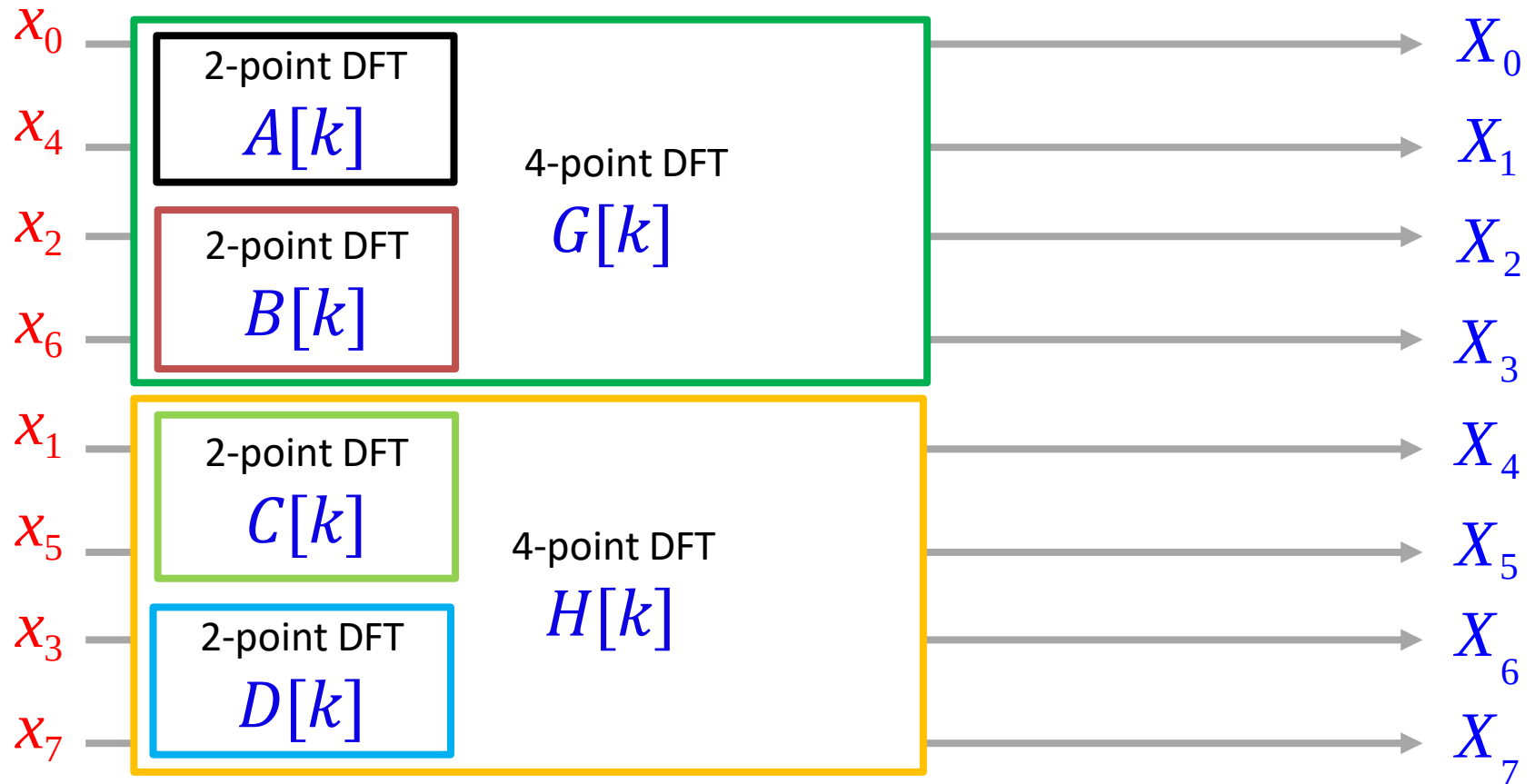
8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



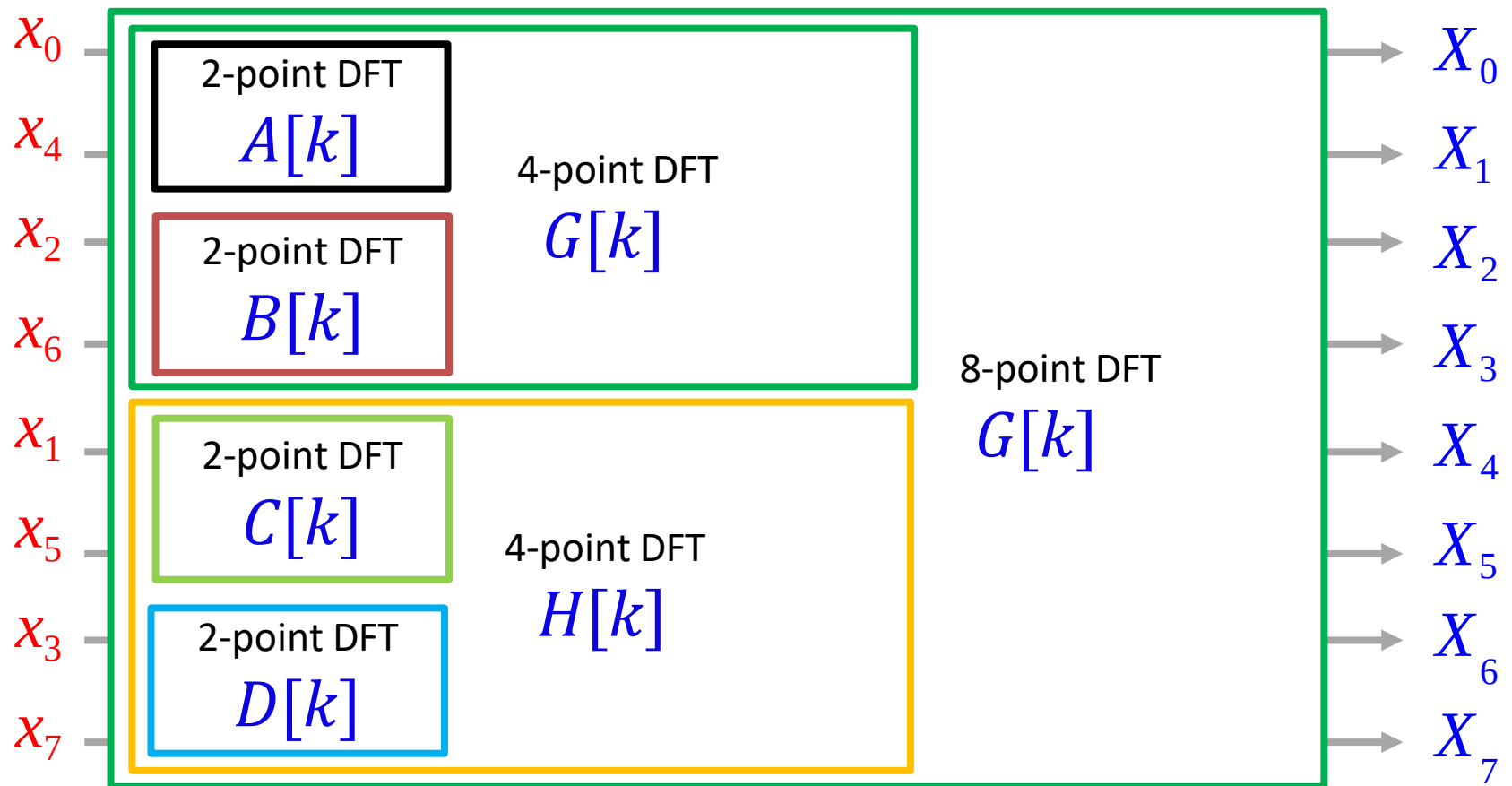
8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$

8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$

X_0

X_4

X_2

X_6

X_1

X_5

X_3

X_7

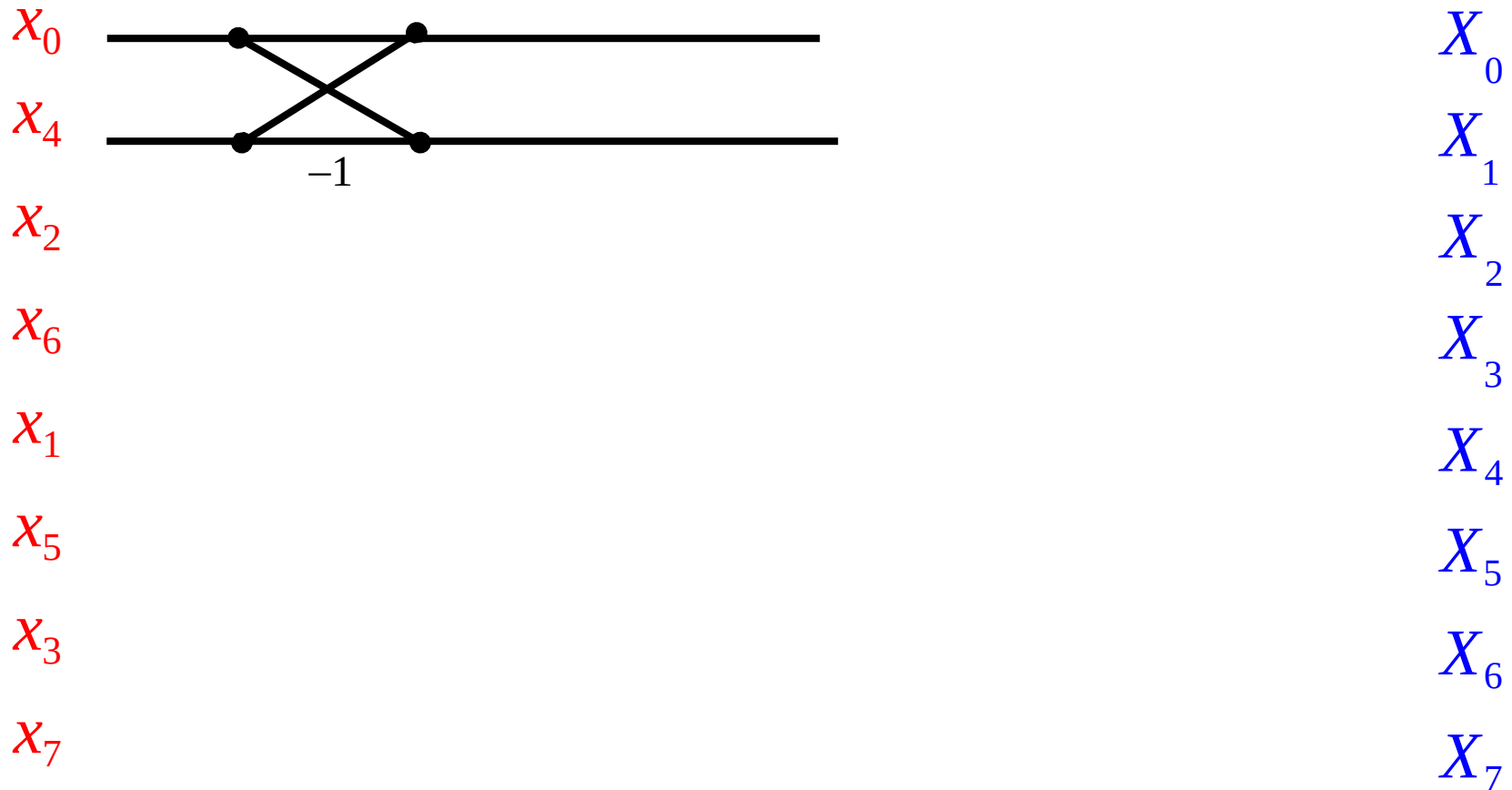
8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$

x_0	X_0
x_4	X_1
x_2	X_2
x_6	X_3
x_1	X_4
x_5	X_5
x_3	X_6
x_7	X_7

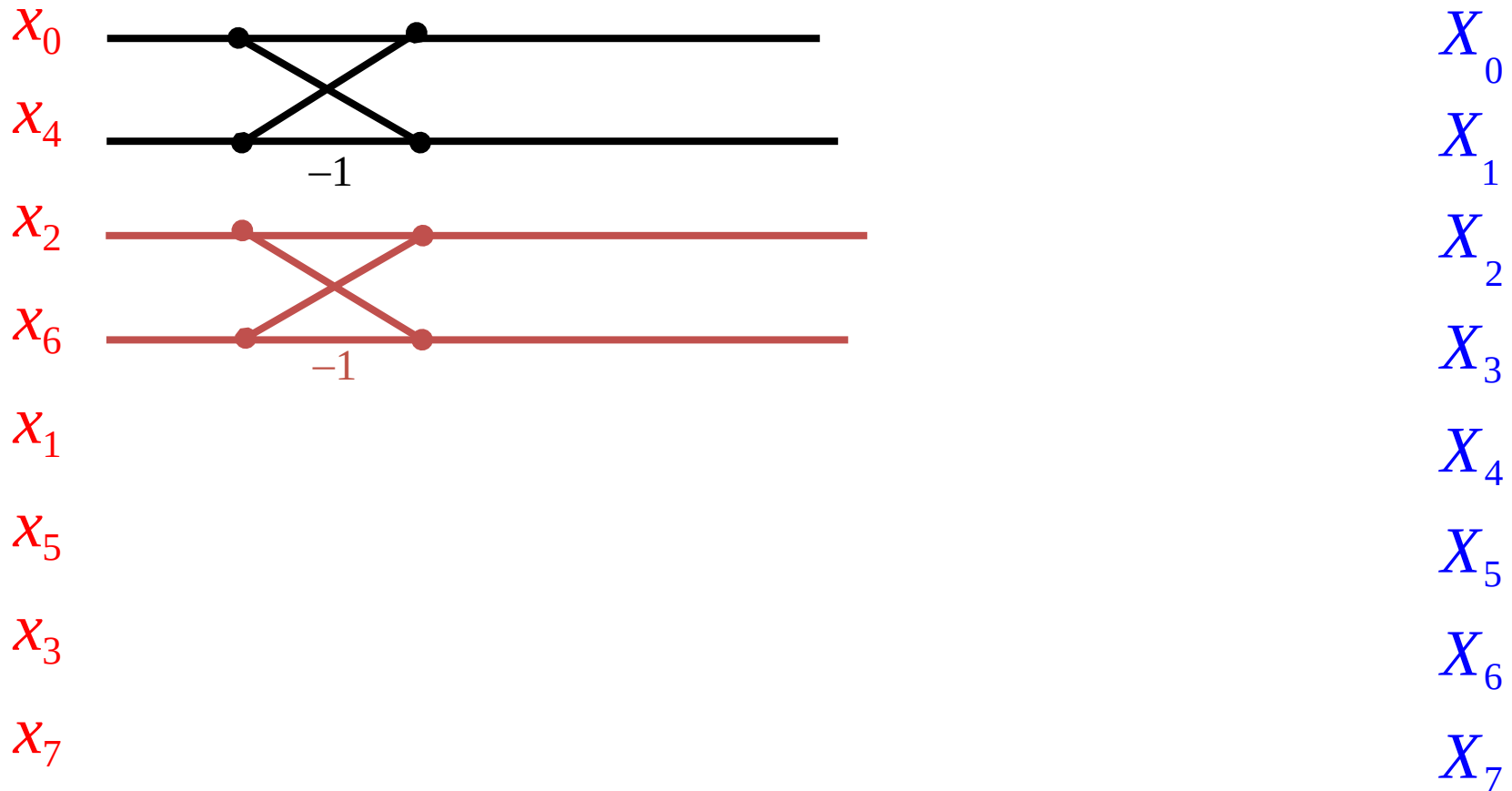
8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



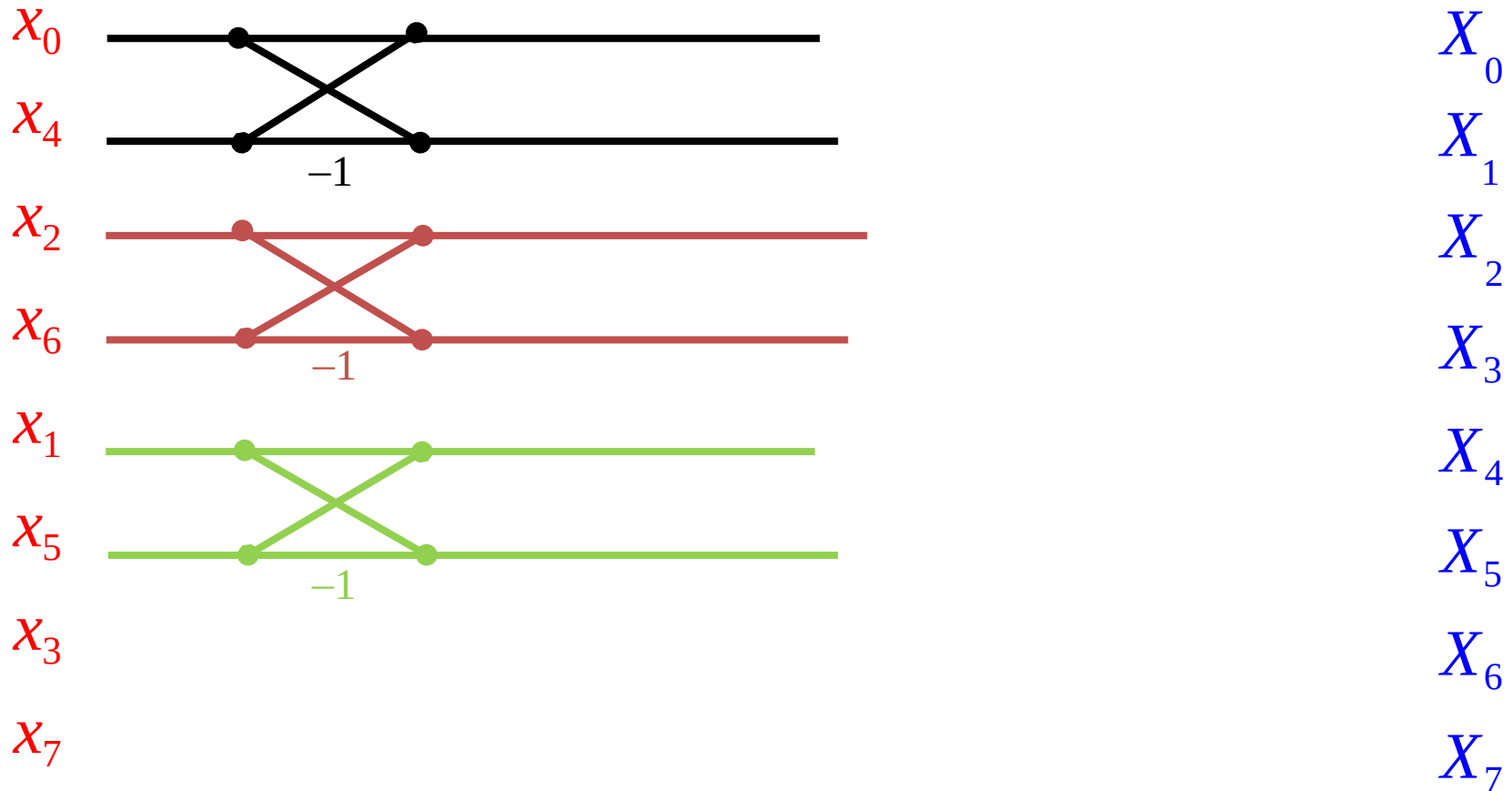
8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



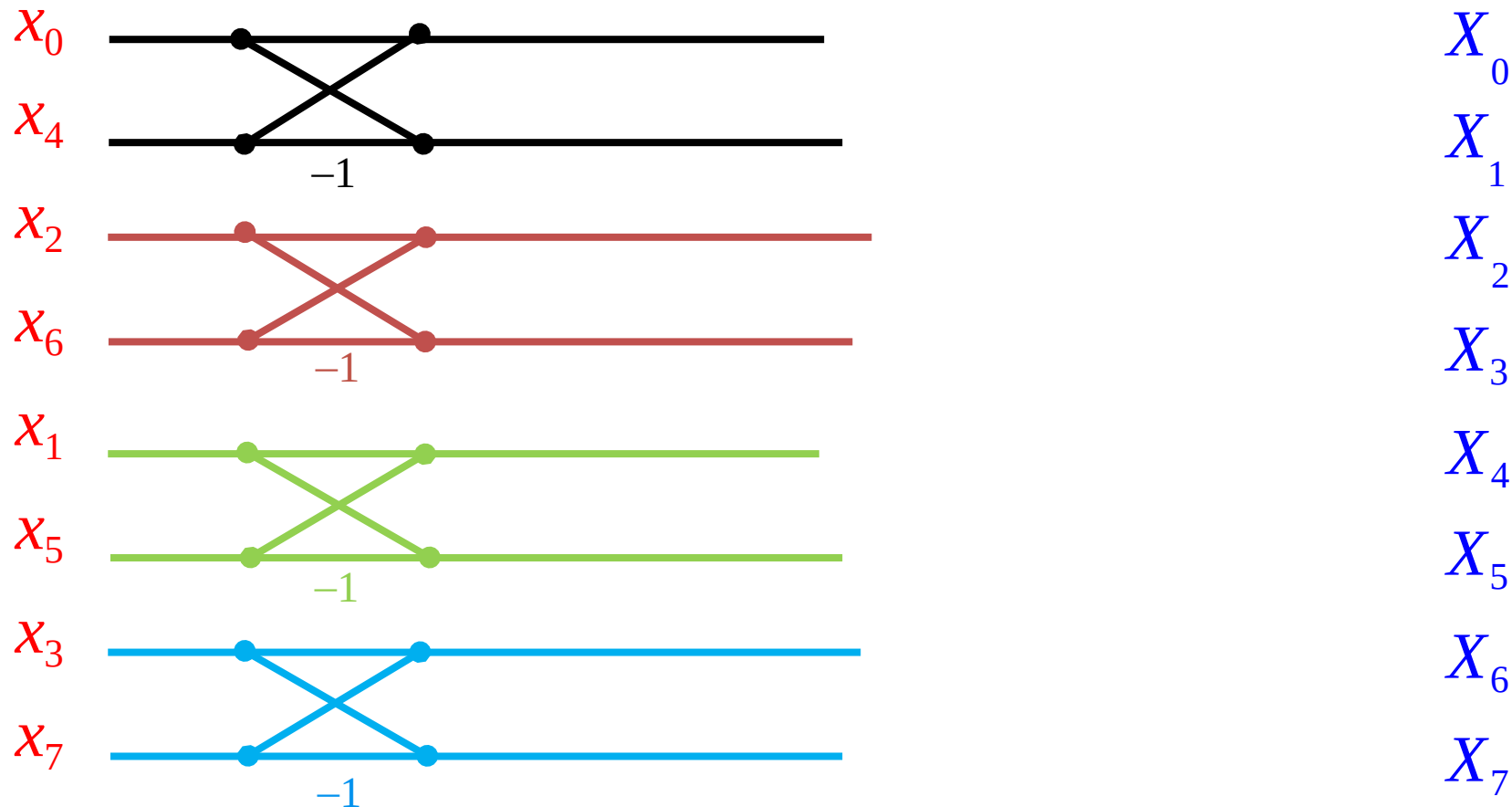
8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



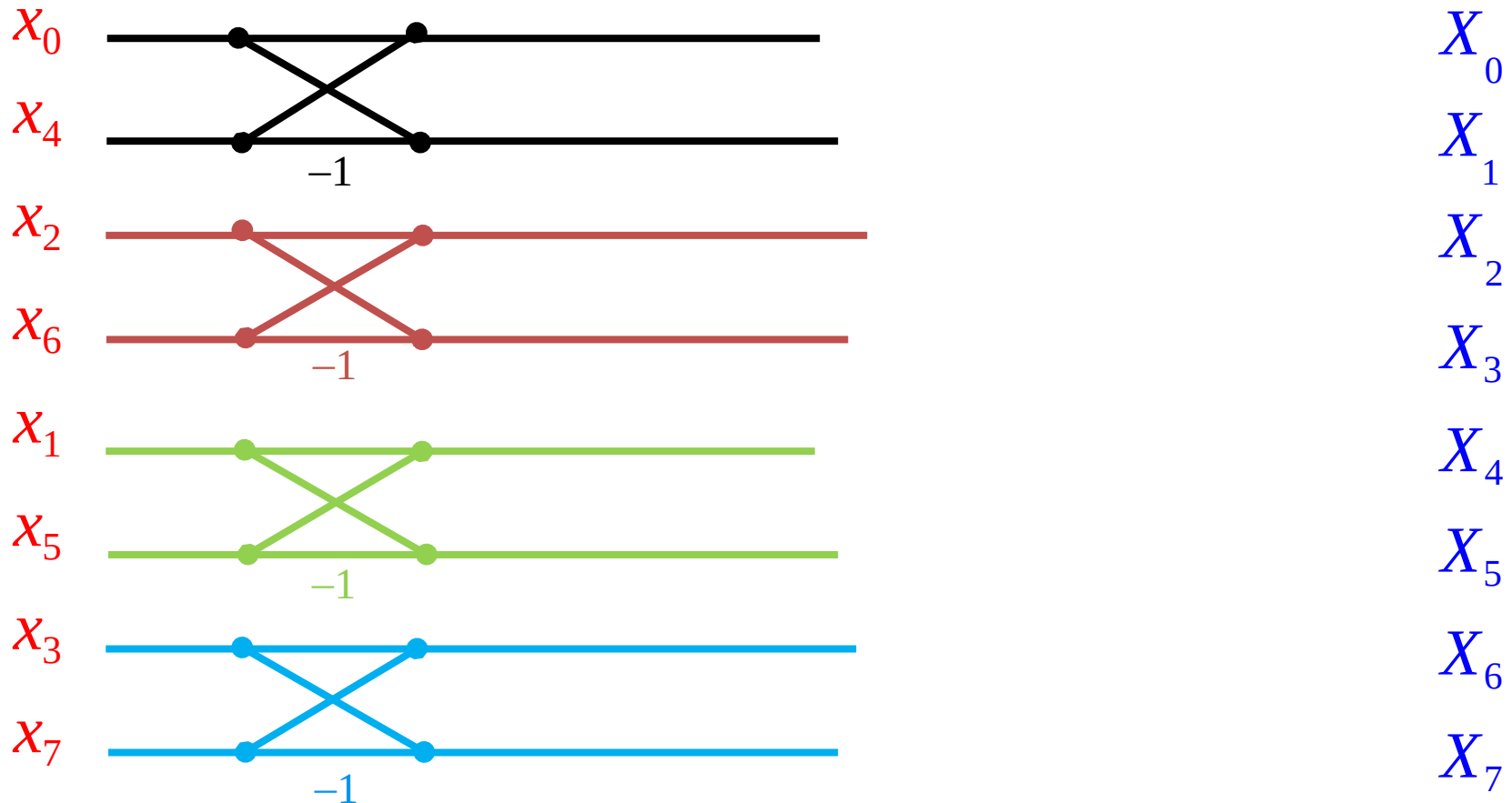
8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



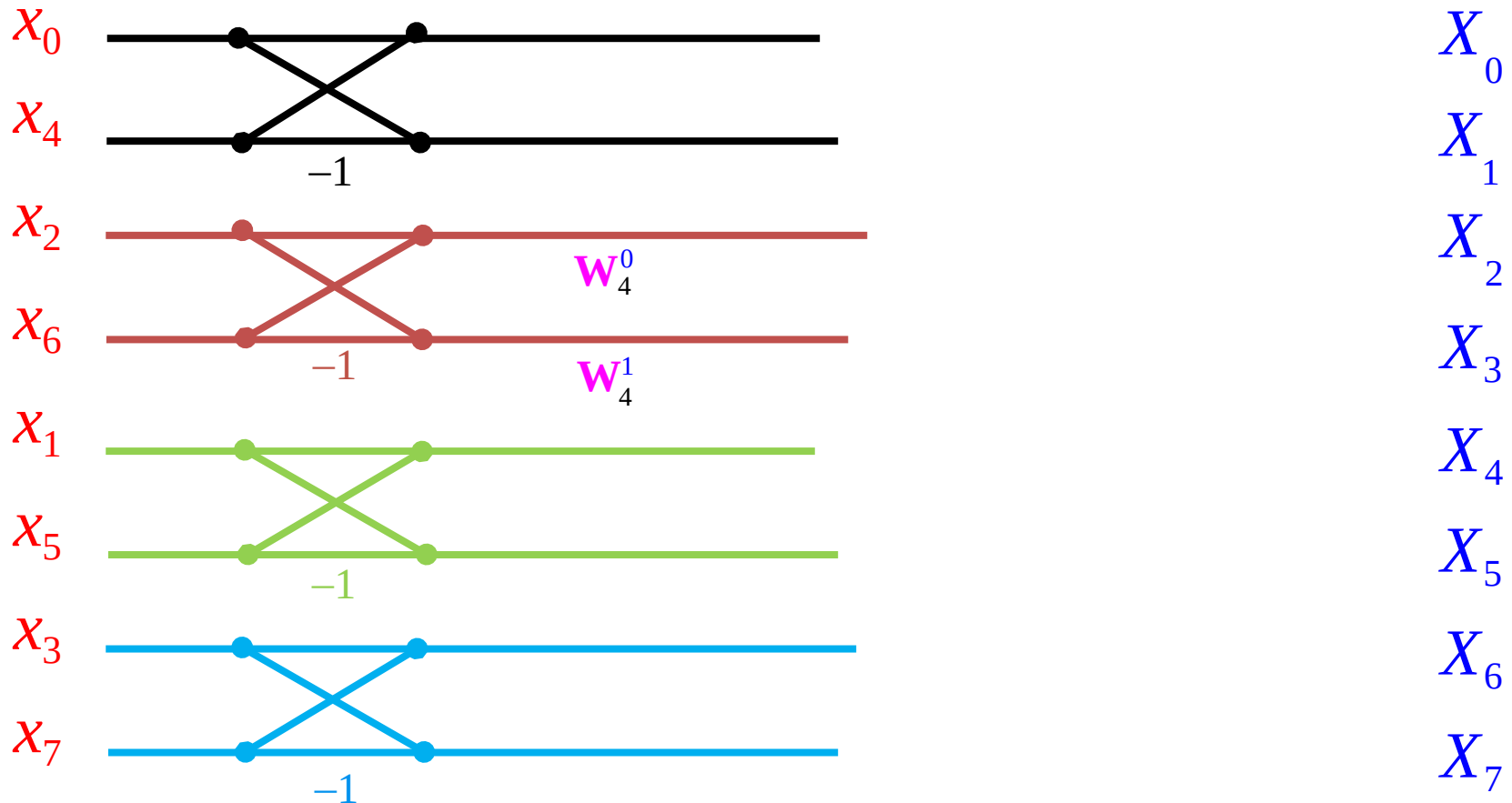
8-point DITFFT $G[k] = A[k] + W_4^k B[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



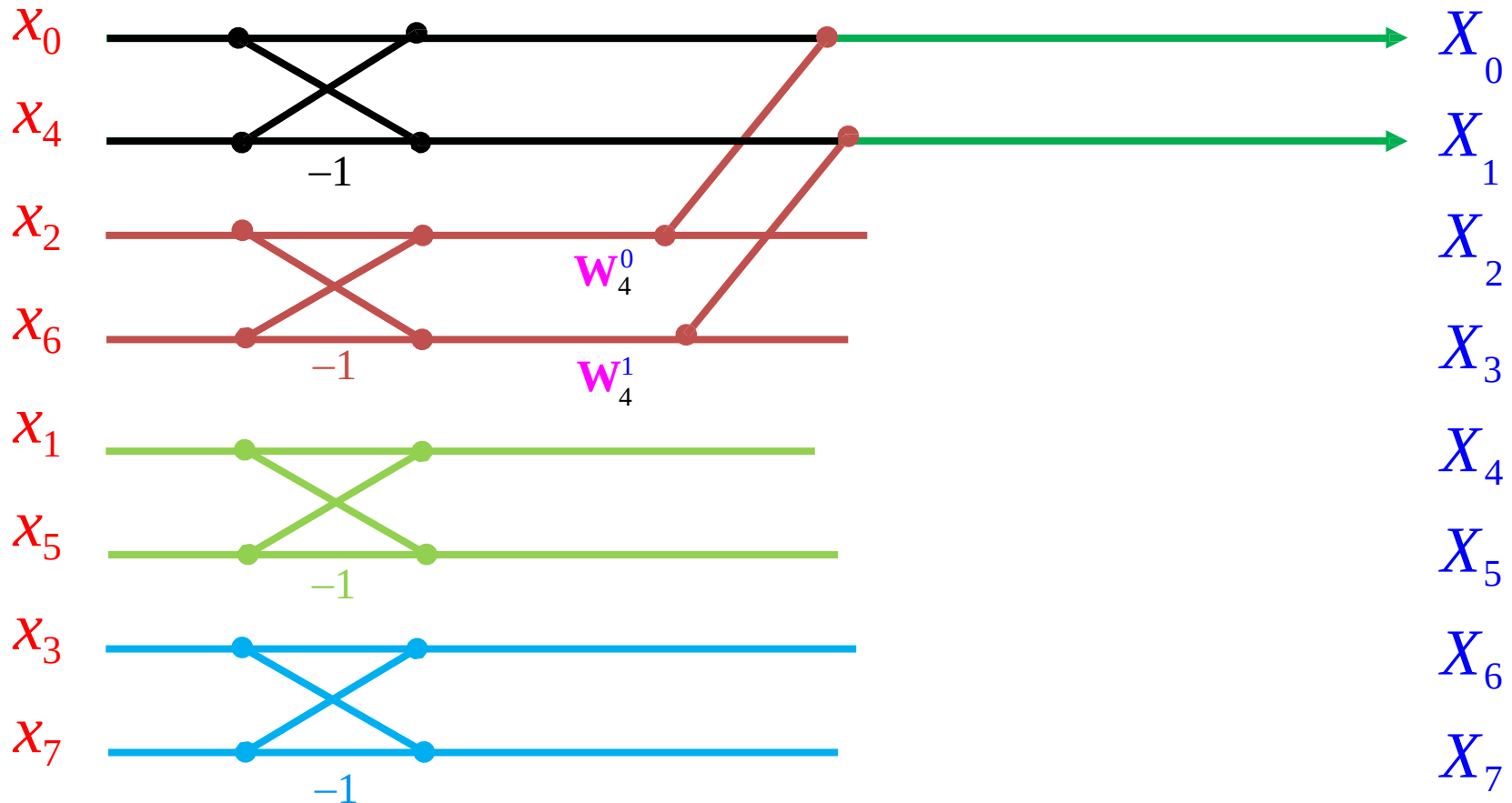
8-point DITFFT $G[k] = A[k] + W_4^k B[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



8-point DITFFT $G[k] = A[k] + W_4^k B[k]$

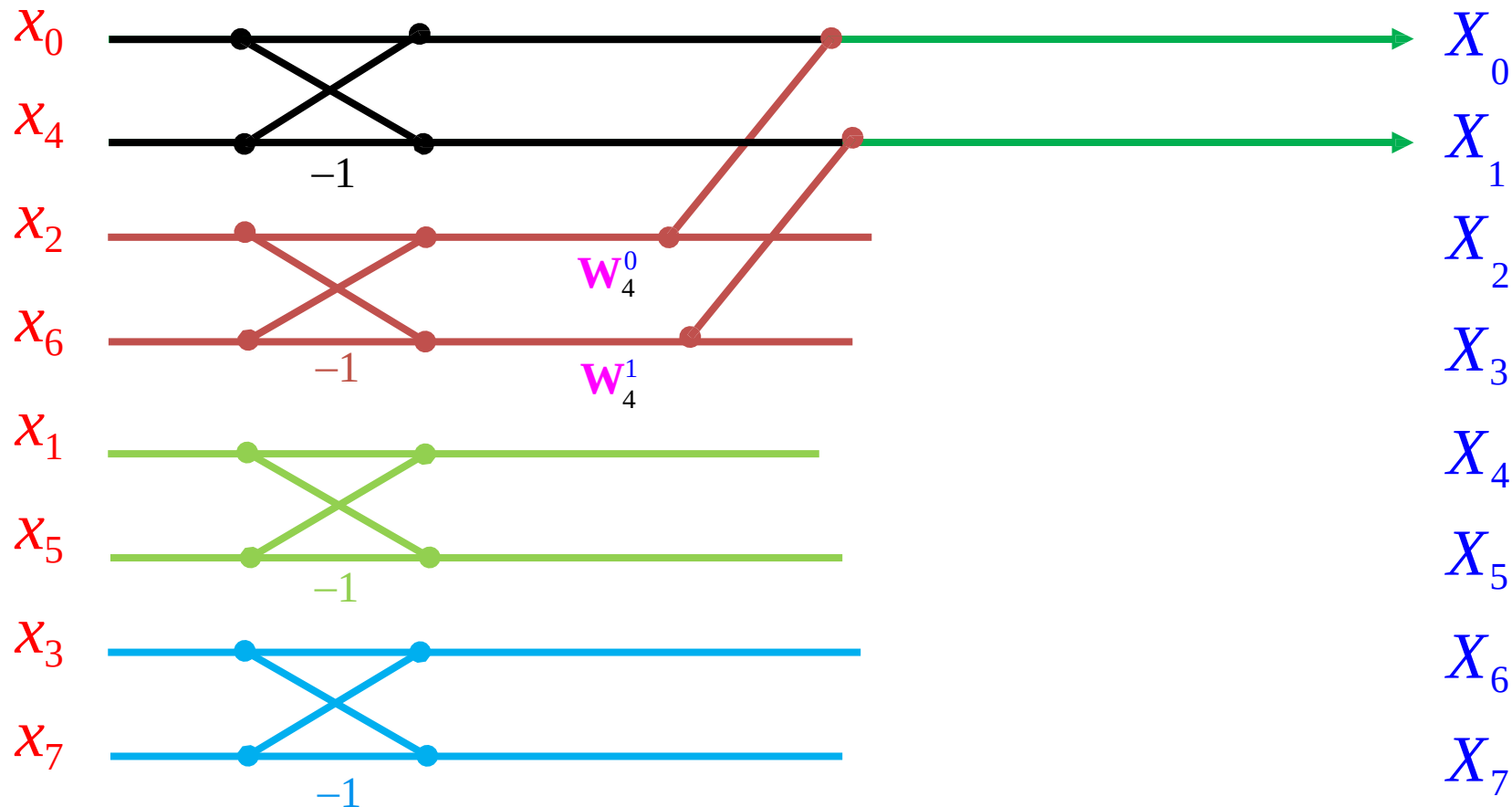
- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



8-point DITFFT

$$G[k] = A[k] - W_4^k B[k]$$

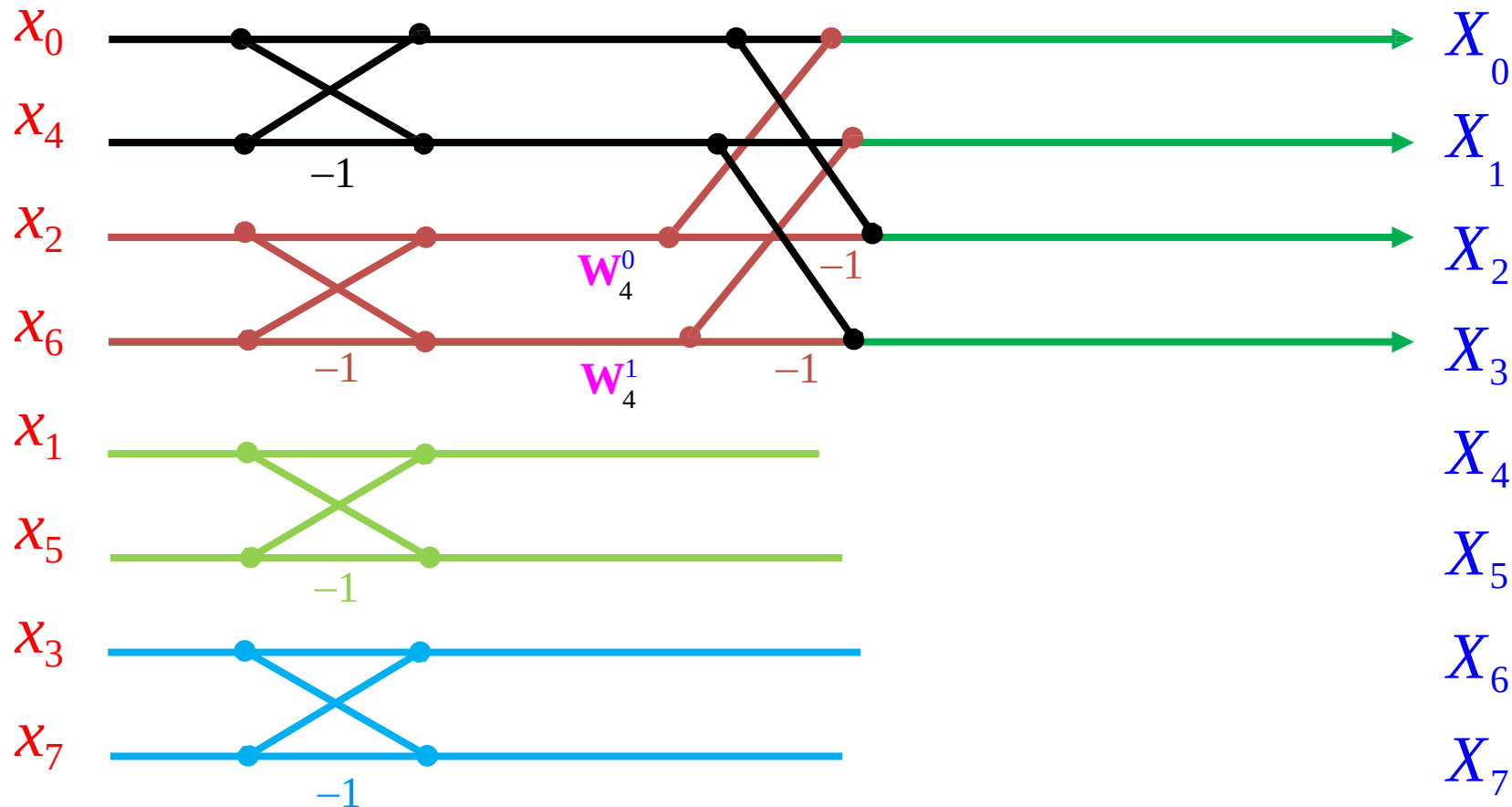
- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



8-point DITFFT

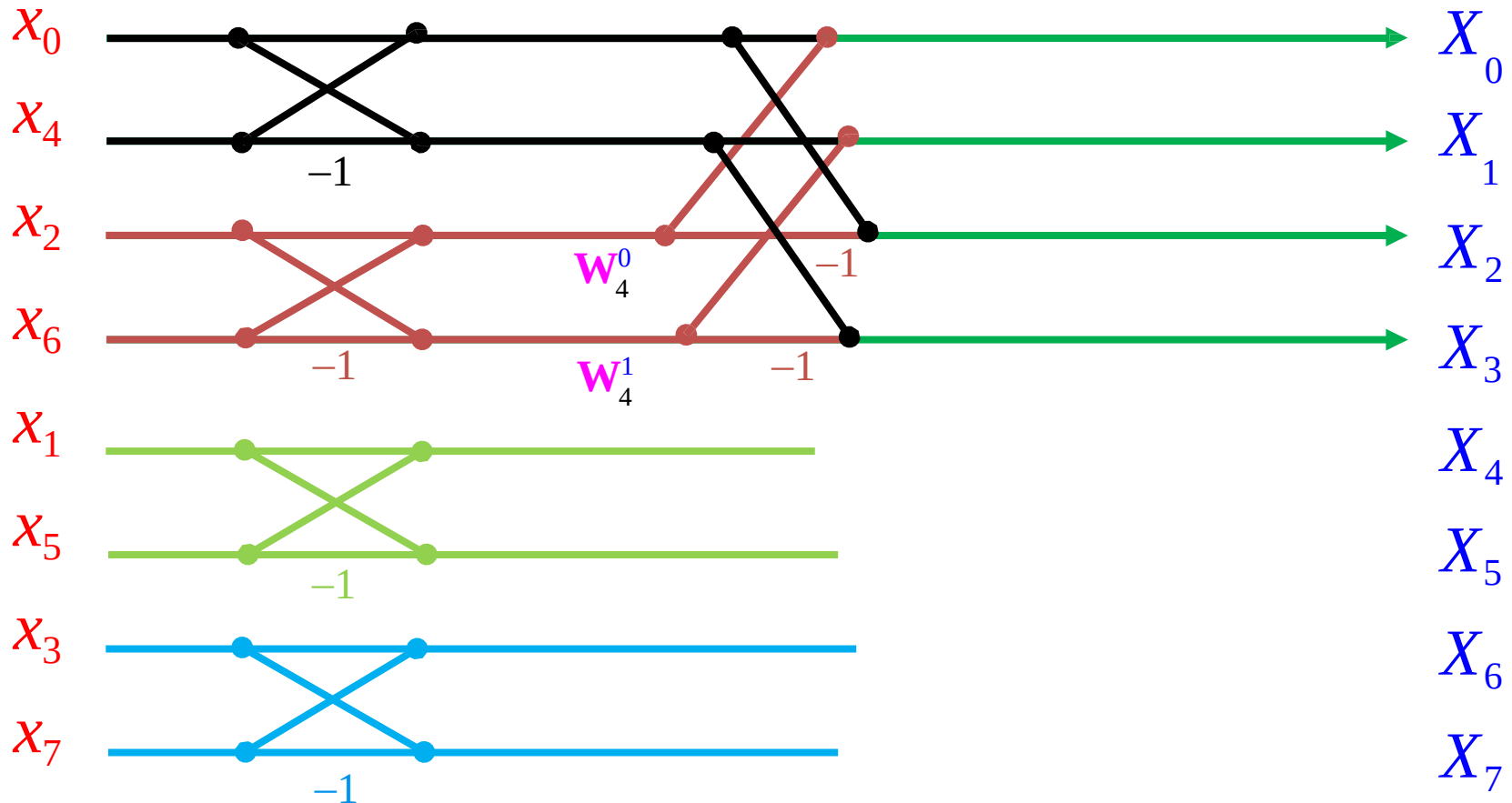
$$G[k] = A[k] - W_4^k B[k]$$

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



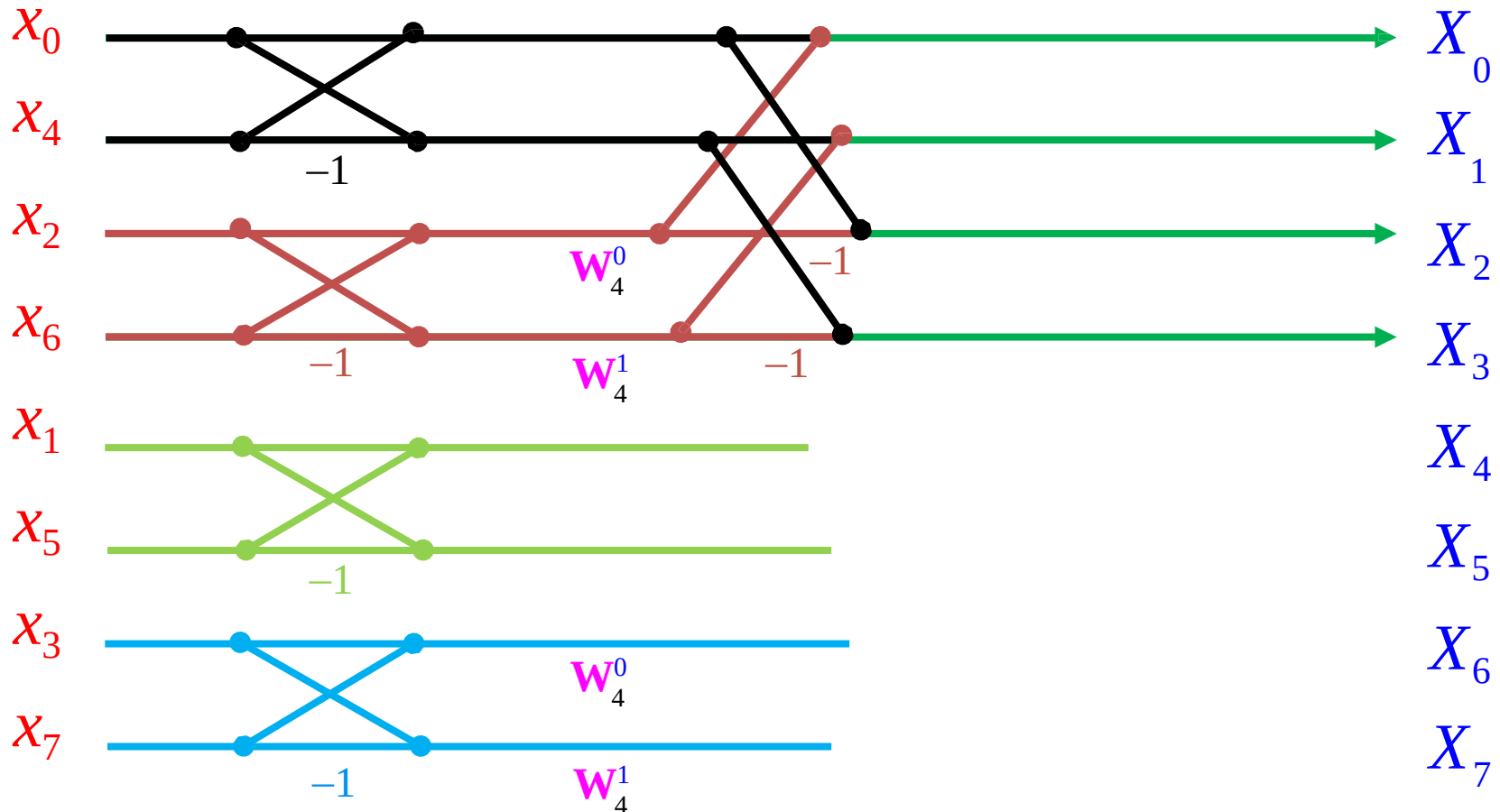
8-point DITFFT $H[k] = C[k] + \mathbf{W}_4^k D[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



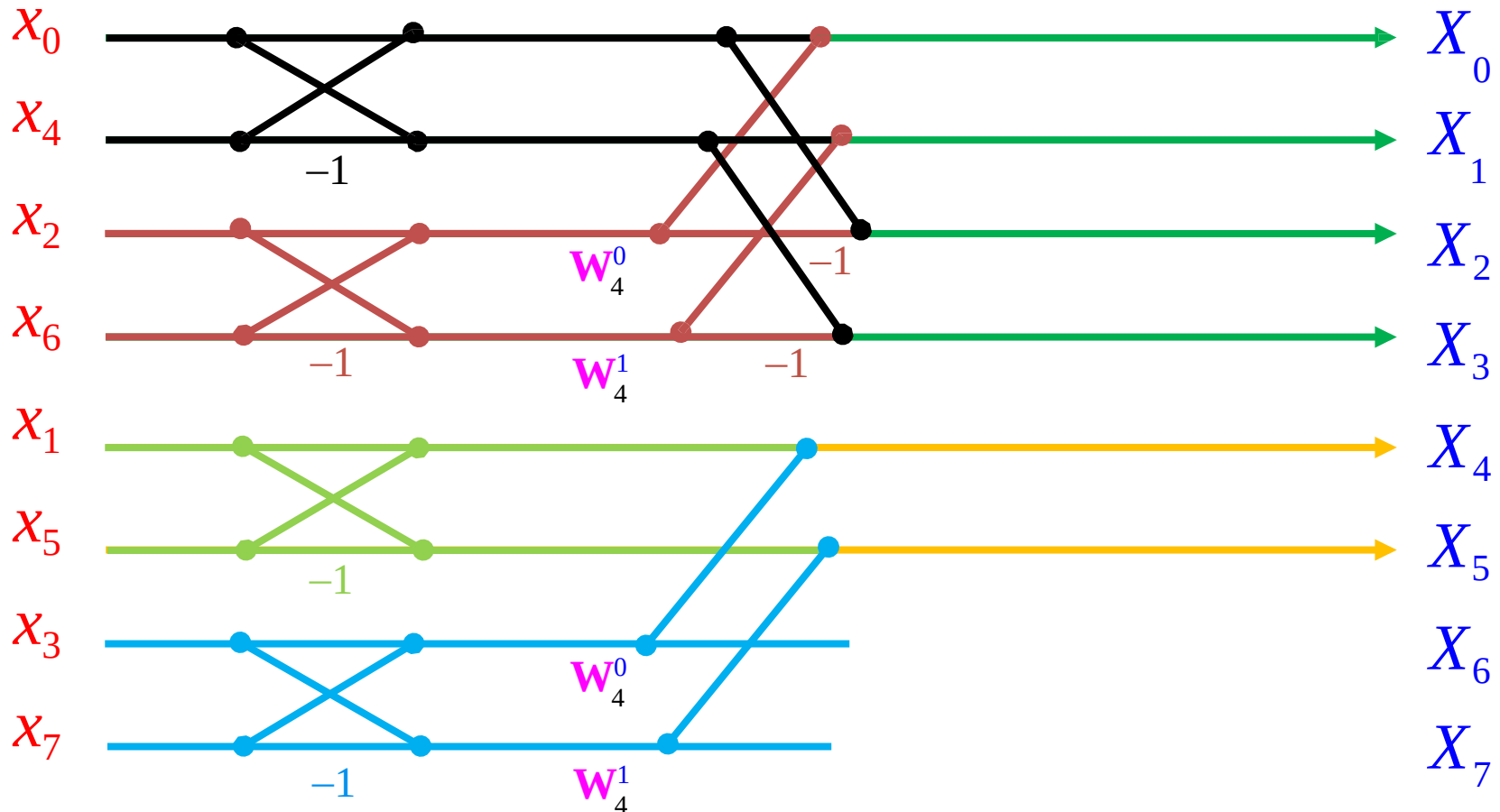
8-point DITFFT $H[k] = C[k] + \mathbf{W}_4^k D[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



8-point DITFFT $H[k] = C[k] + \mathbf{W}_4^k D[k]$

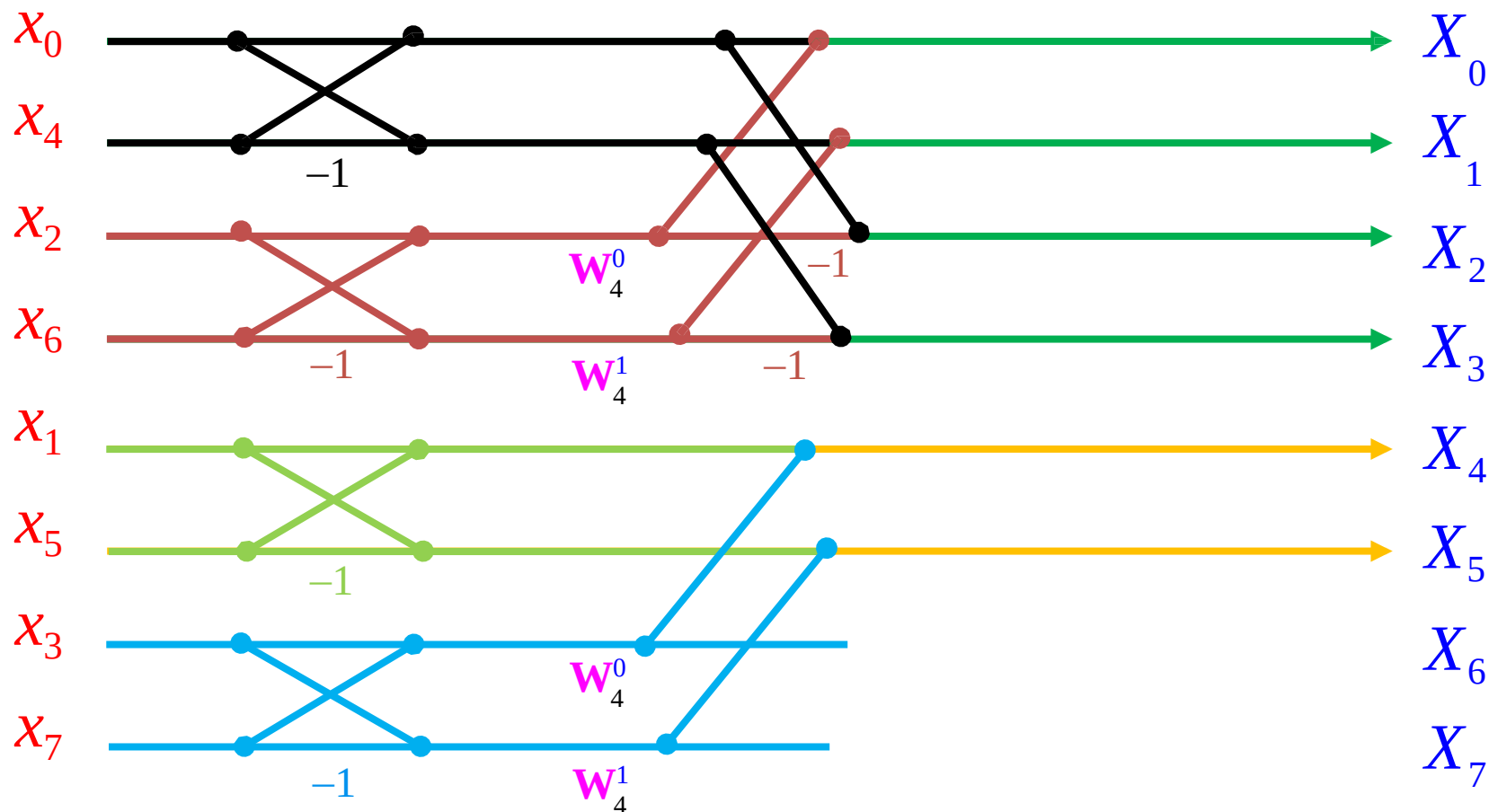
- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



8-point DITFFT

$$H[k] = C[k] - \mathbf{W}_4^k D[k]$$

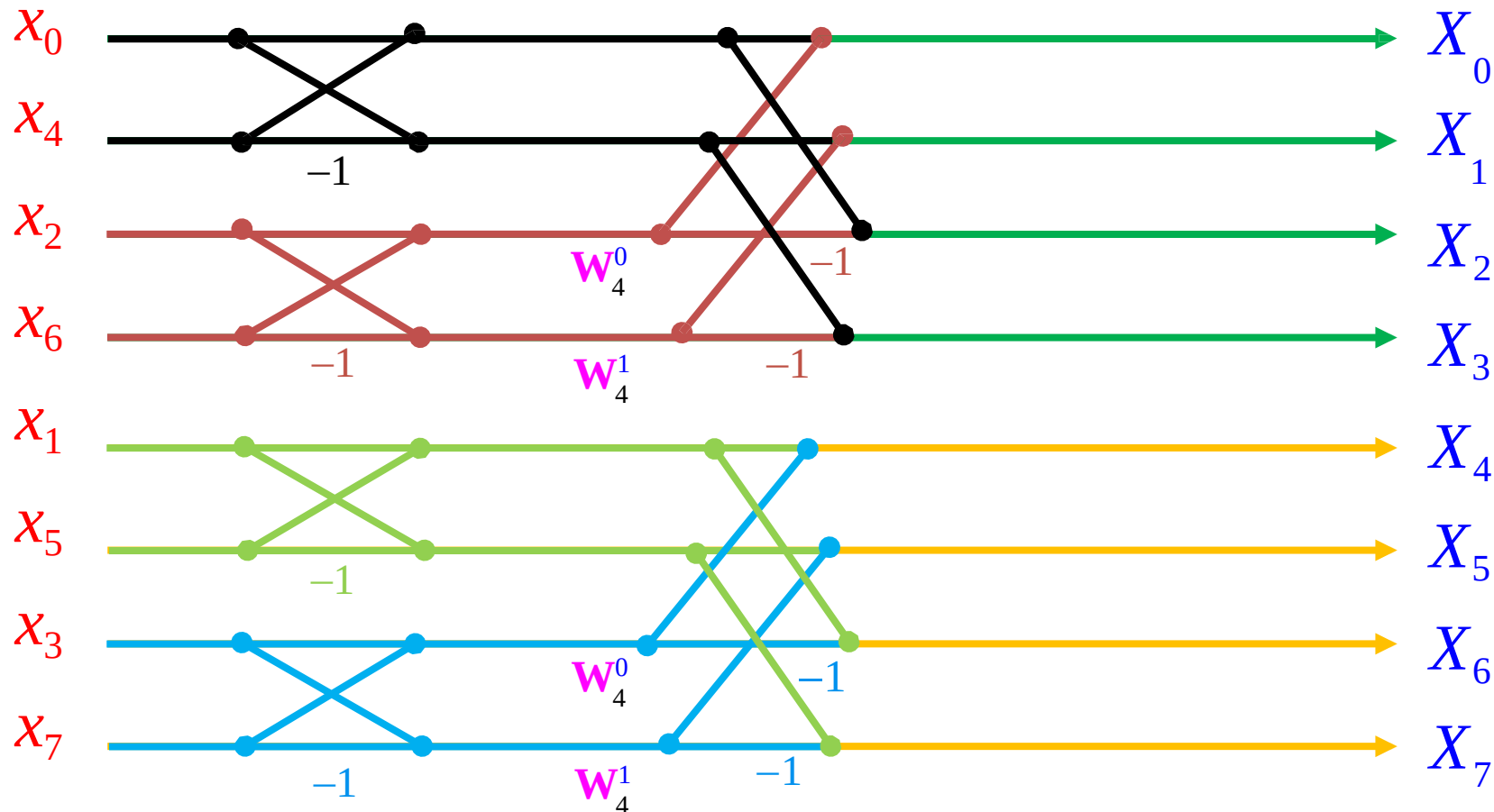
- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



8-point DITFFT

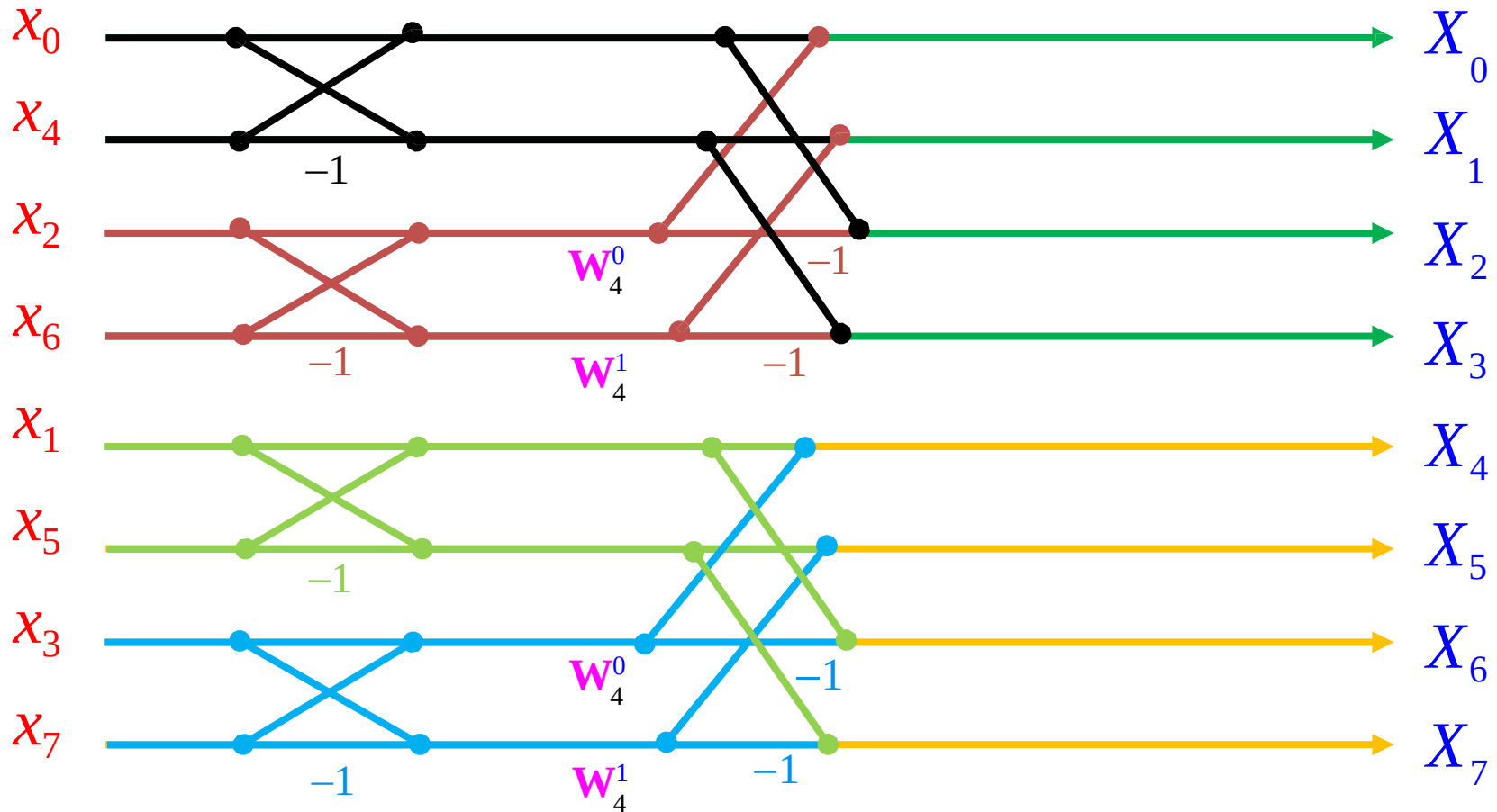
$$H[k] = C[k] - \mathbf{W}_4^k D[k]$$

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



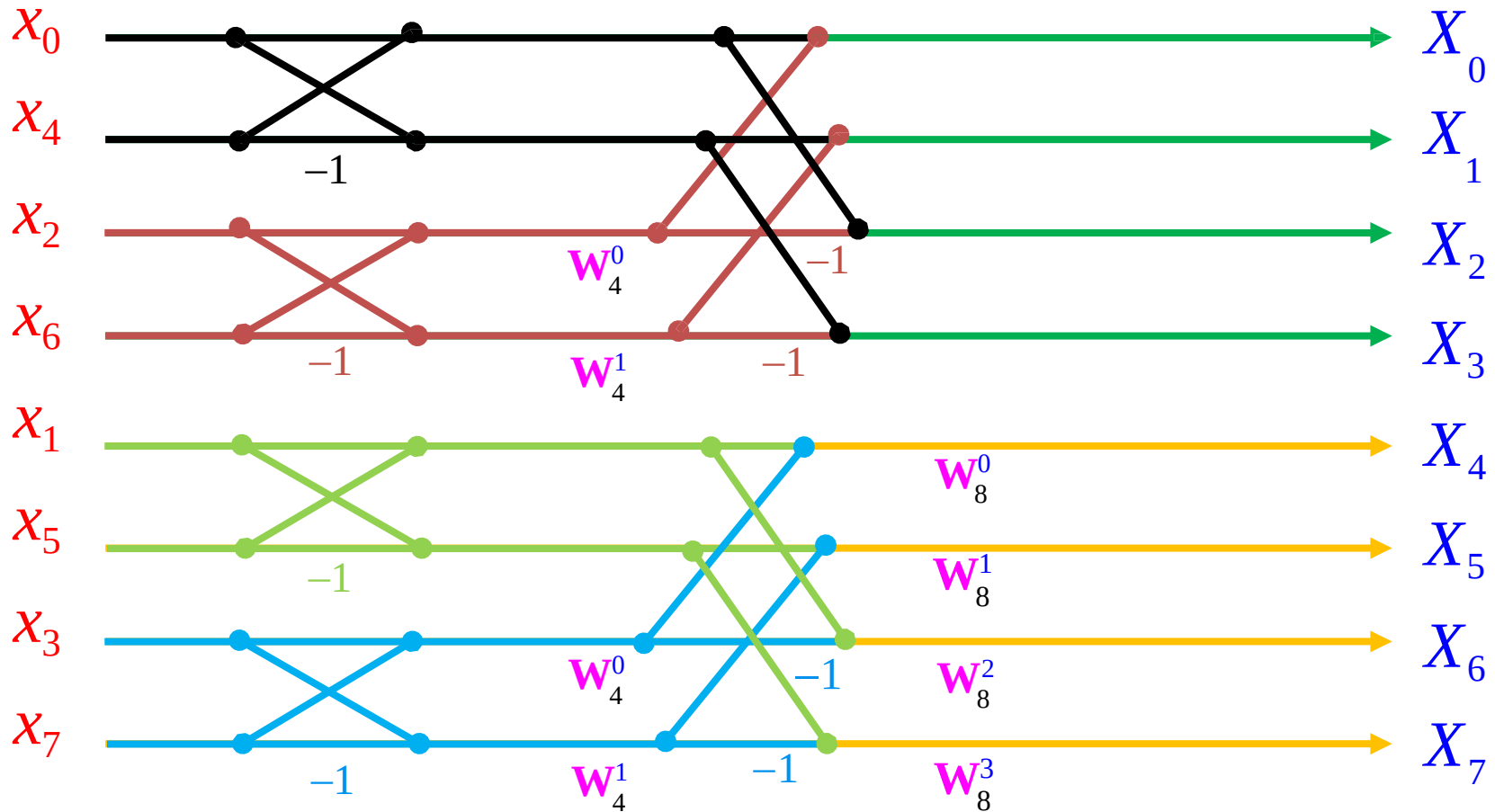
8-point DITFFT $X[k] = G[k] + \mathbf{W}_8^k H[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



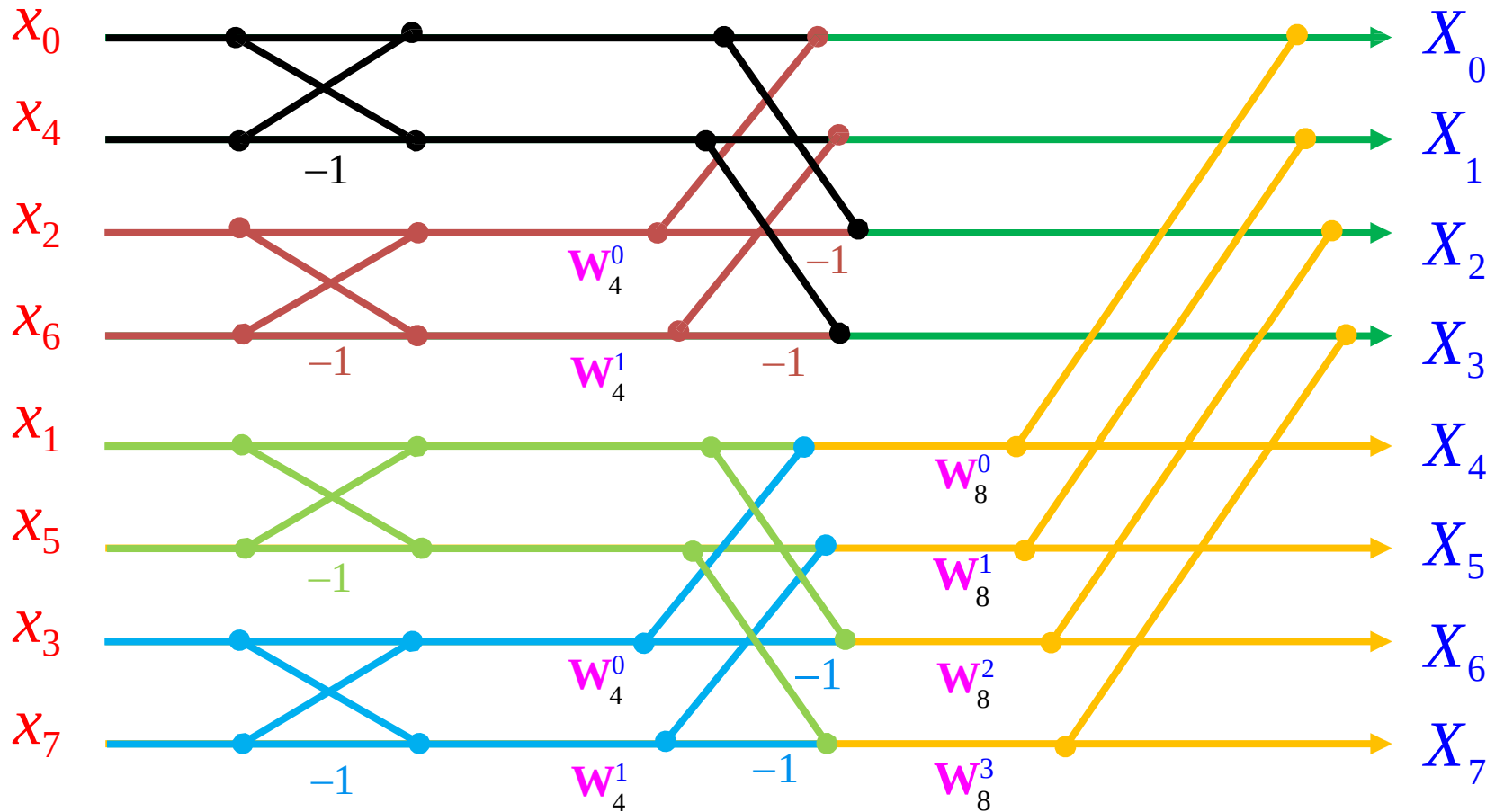
8-point DITFFT $X[k] = G[k] + \mathbf{W}_8^k H[k]$

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



8-point DITFFT $X[k] = G[k] + \mathbf{W}_8^k H[k]$

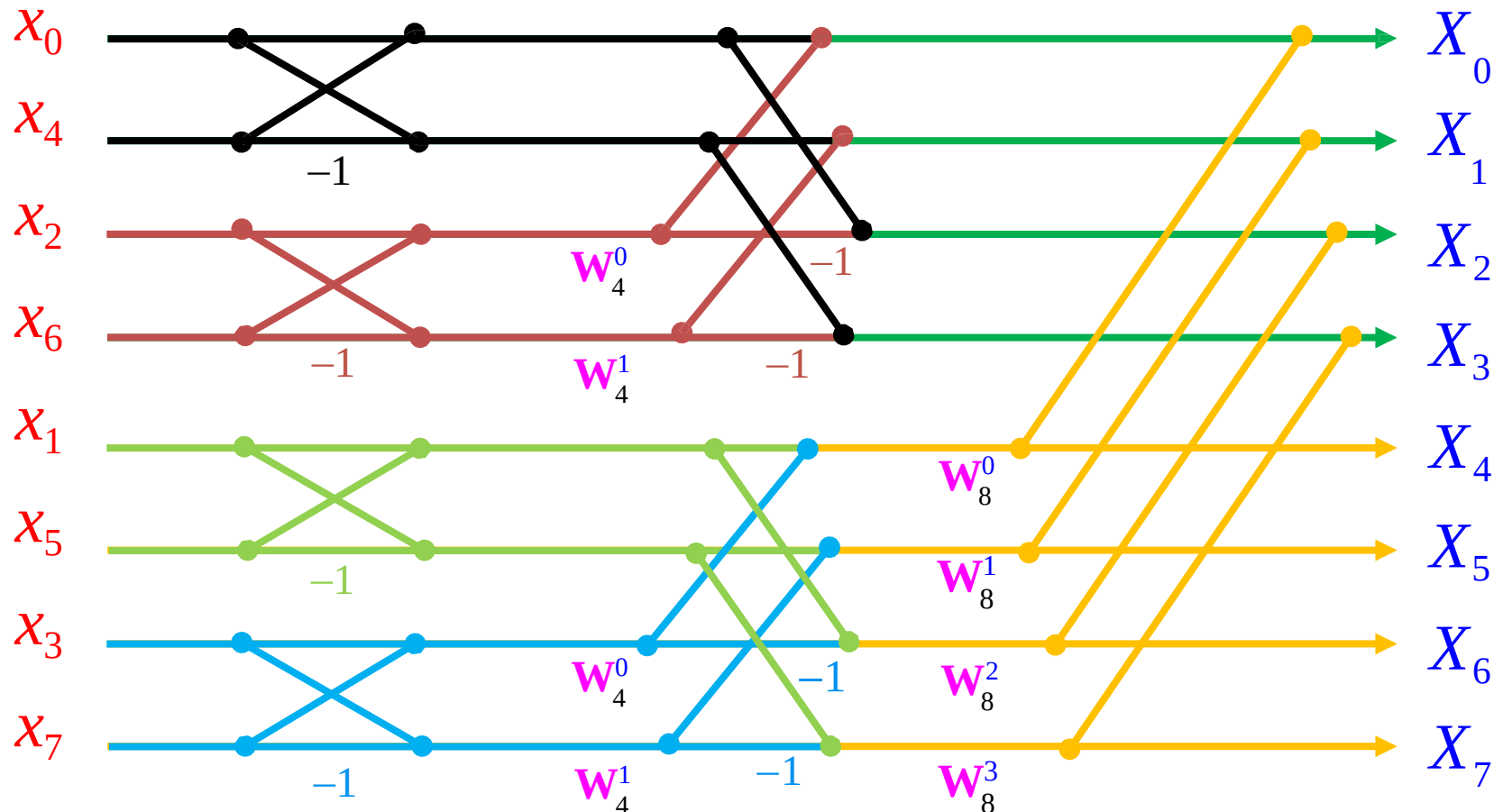
- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



8-point DITFFT

$$X[k] = G[k] - \mathbf{W}_8^k H[k]$$

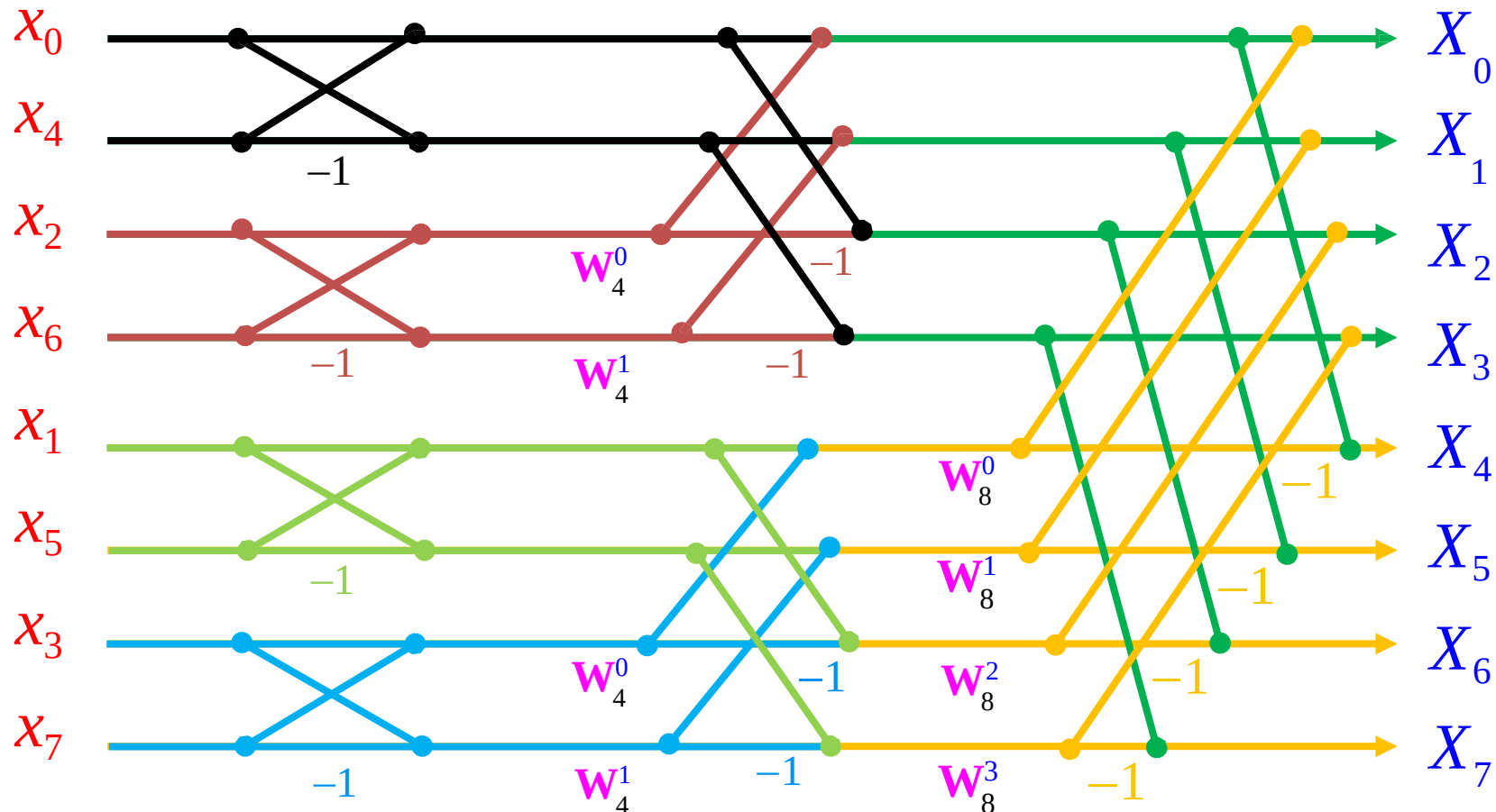
- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



8-point DITFFT

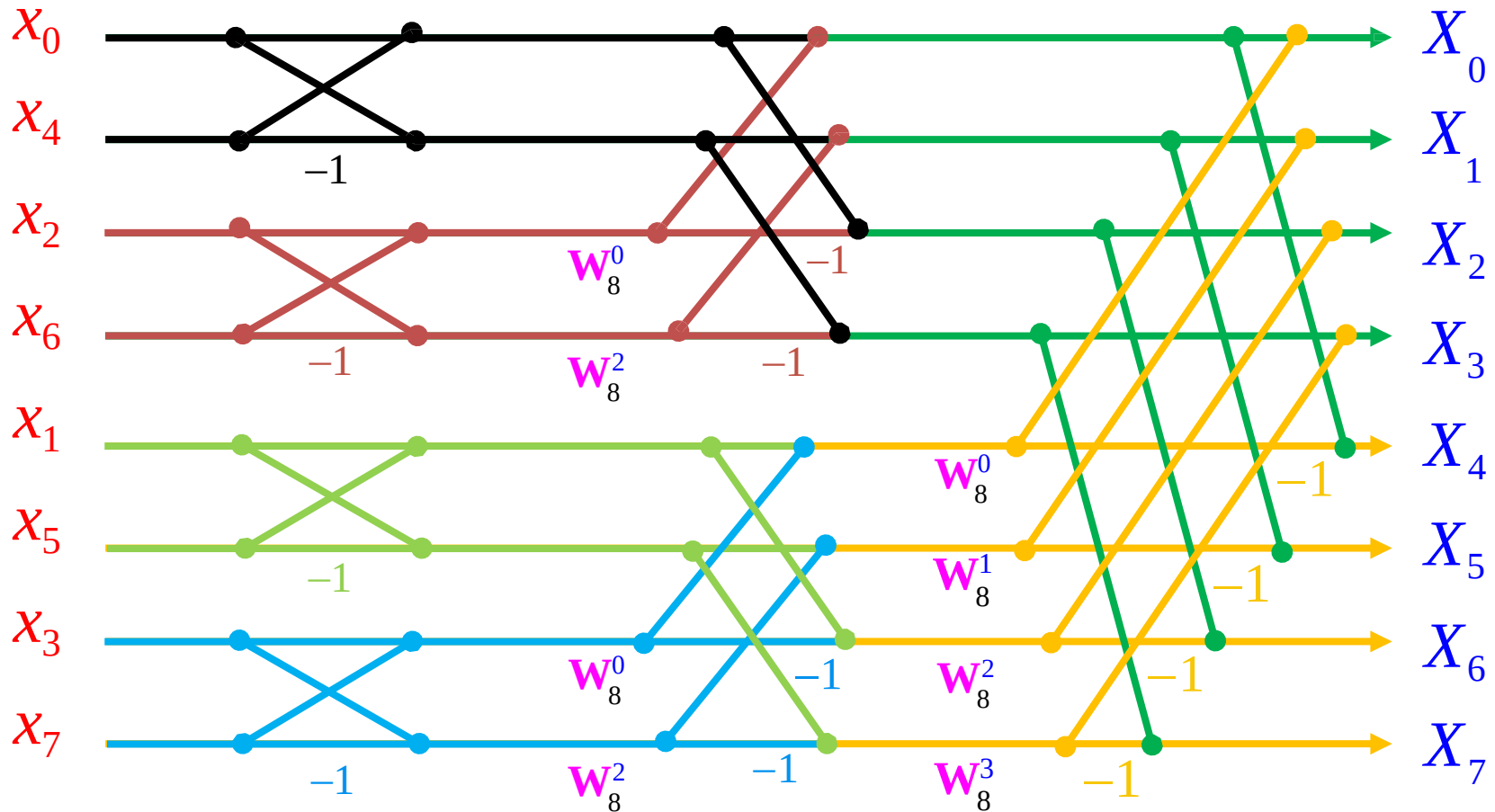
$$X[k] = G[k] - \mathbf{W}_8^k H[k]$$

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



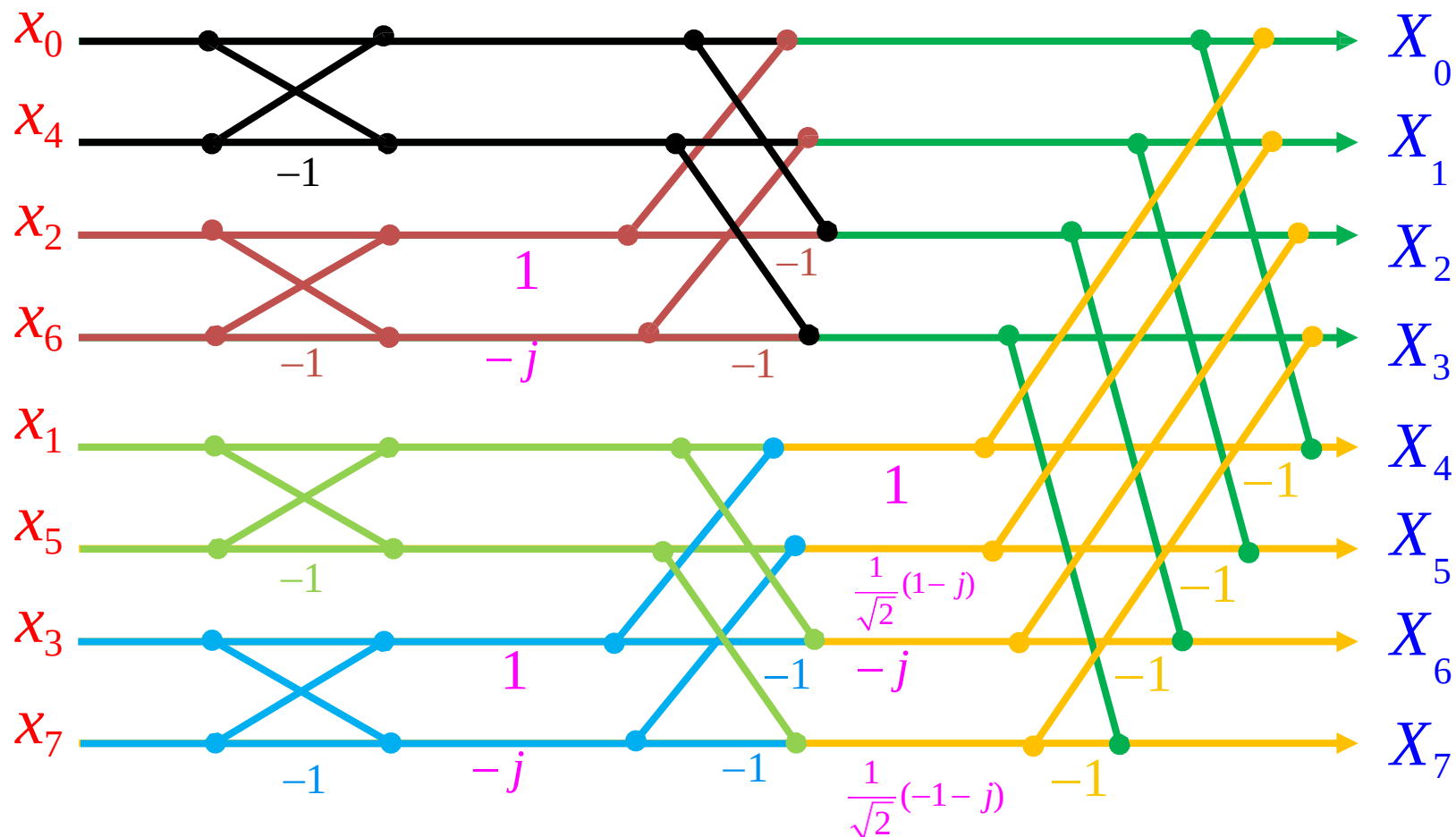
8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



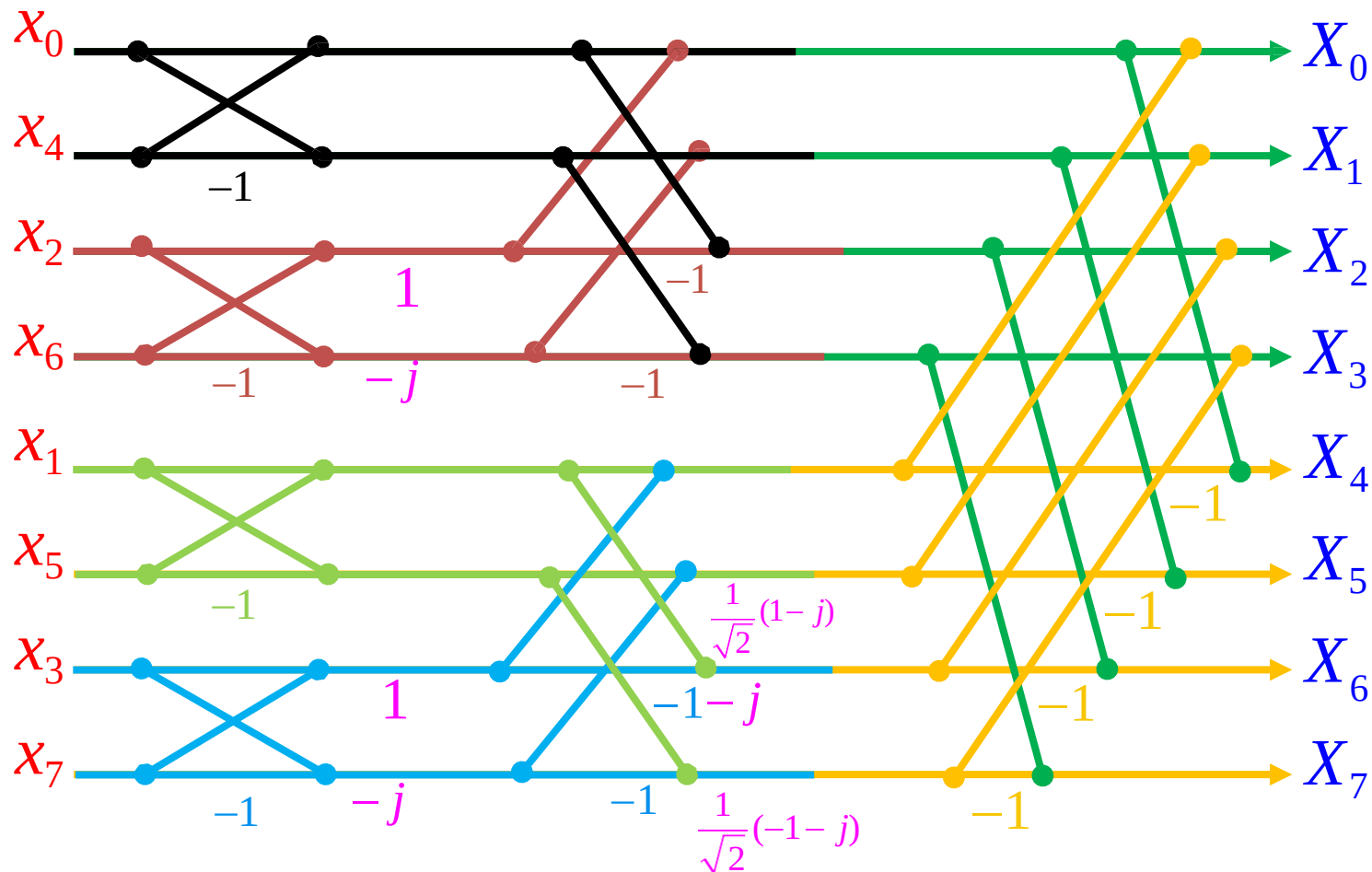
8-point DITFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7\}$



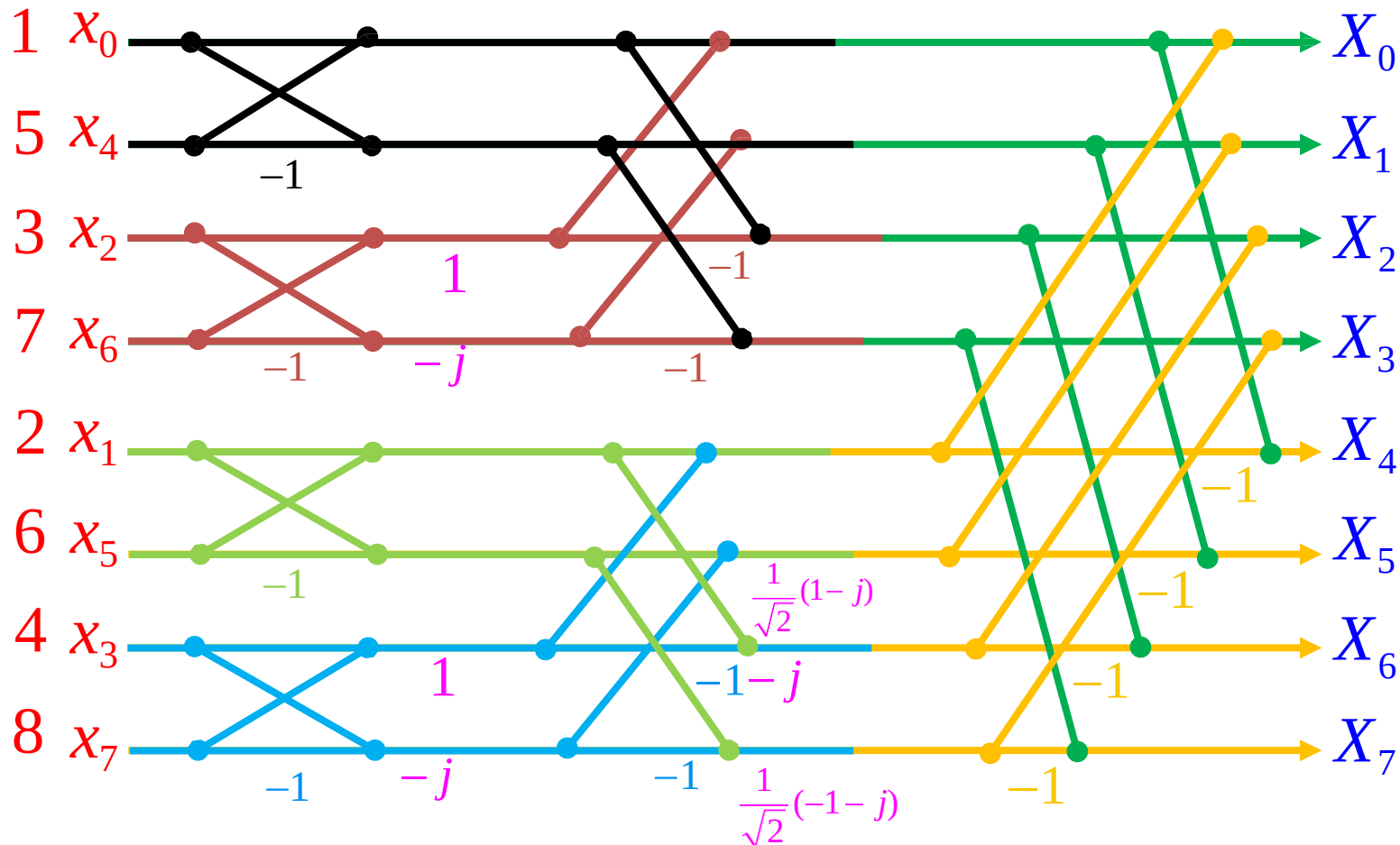
Example-10

Compute DFT of $x[n] = \{1,2,3,4,5,6,7,8\}$ using DITFFT algorithm. (May19,10 marks)



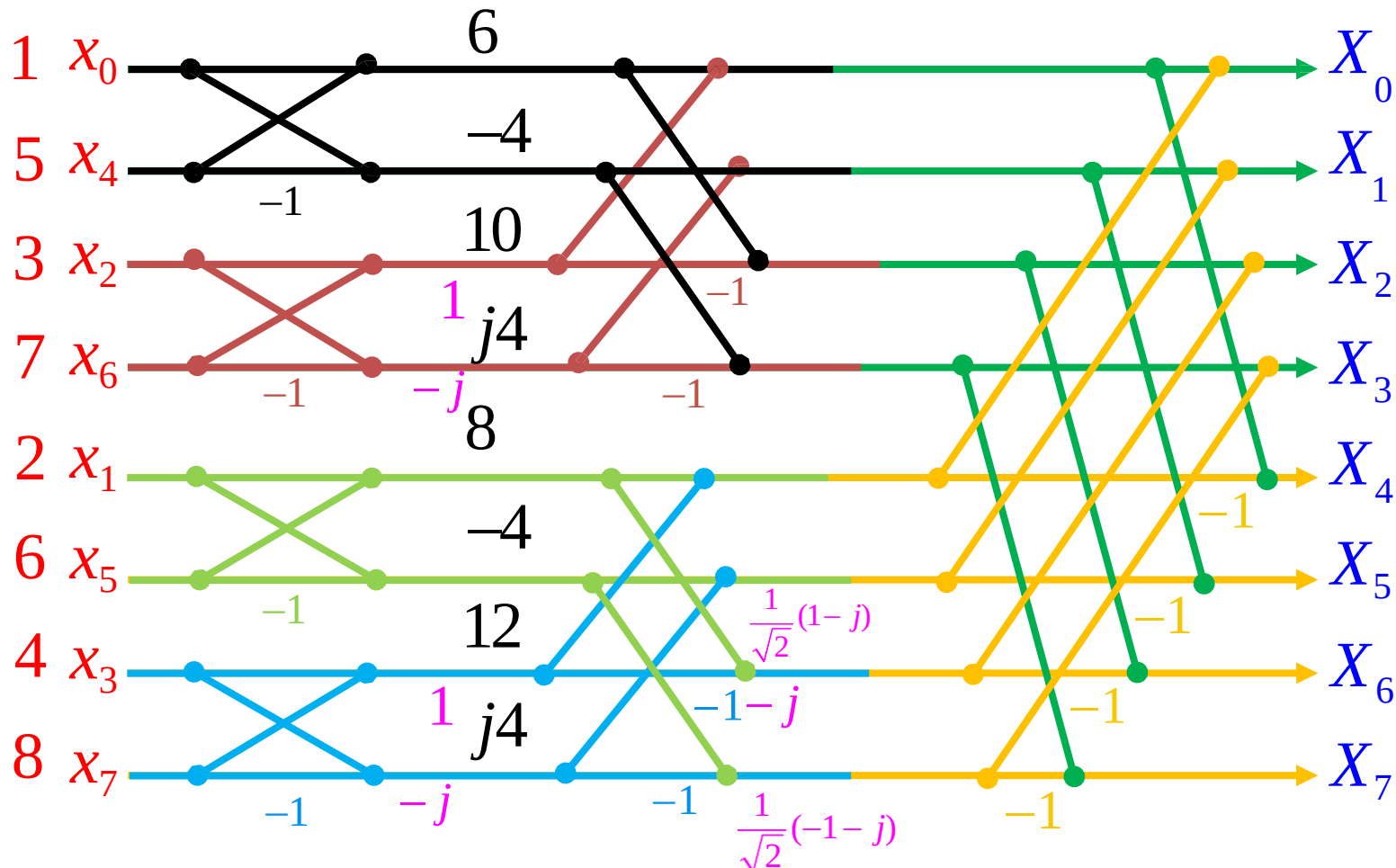
Example-10

Compute DFT of $x[n] = \{1, 2, 3, 4, 5, 6, 7, 8\}$ using DITFFT algorithm. (May19, 10 marks)



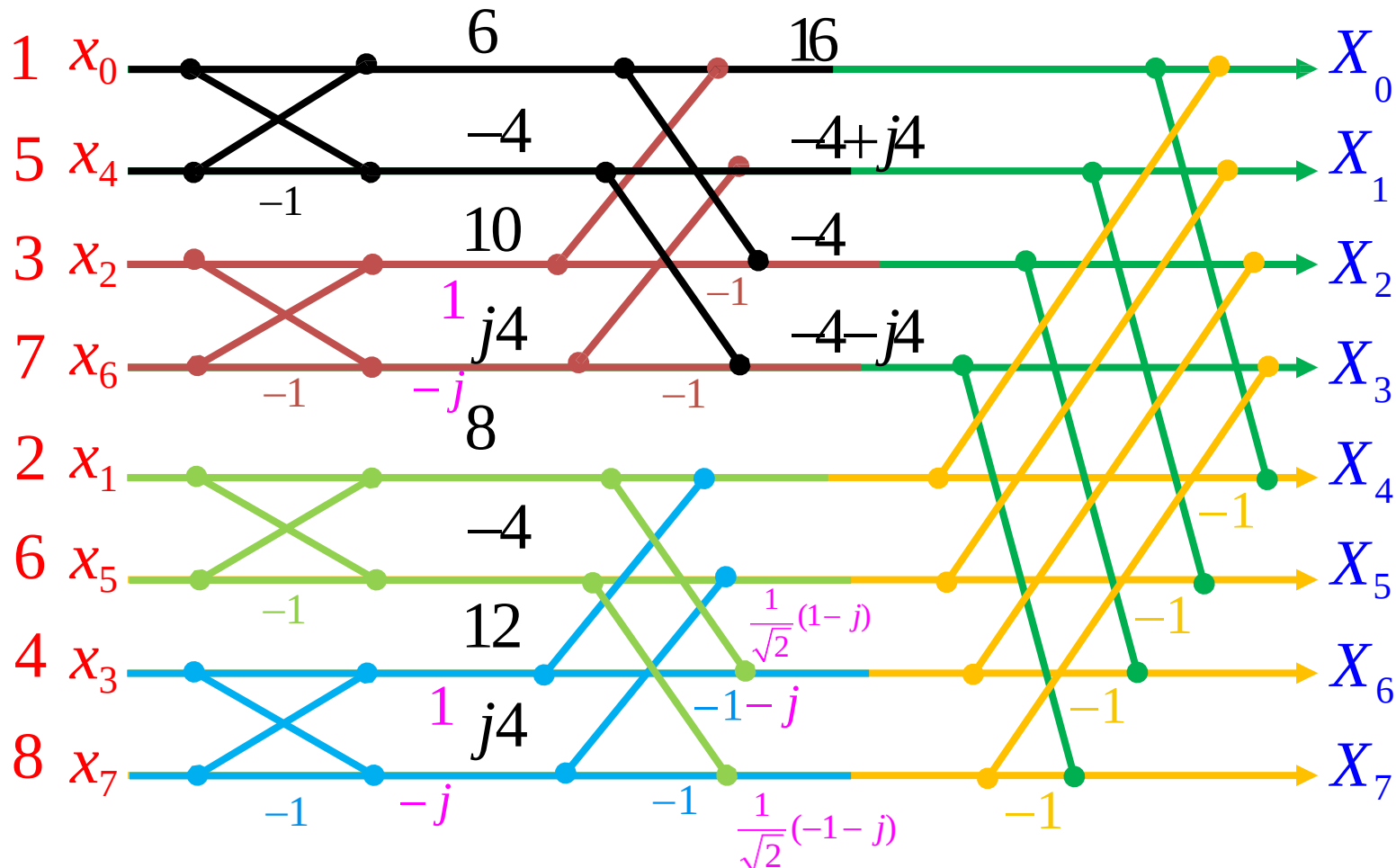
Example-10

Compute DFT of $x[n] = \{1, 2, 3, 4, 5, 6, 7, 8\}$ using DITFFT algorithm. (May19, 10 marks)



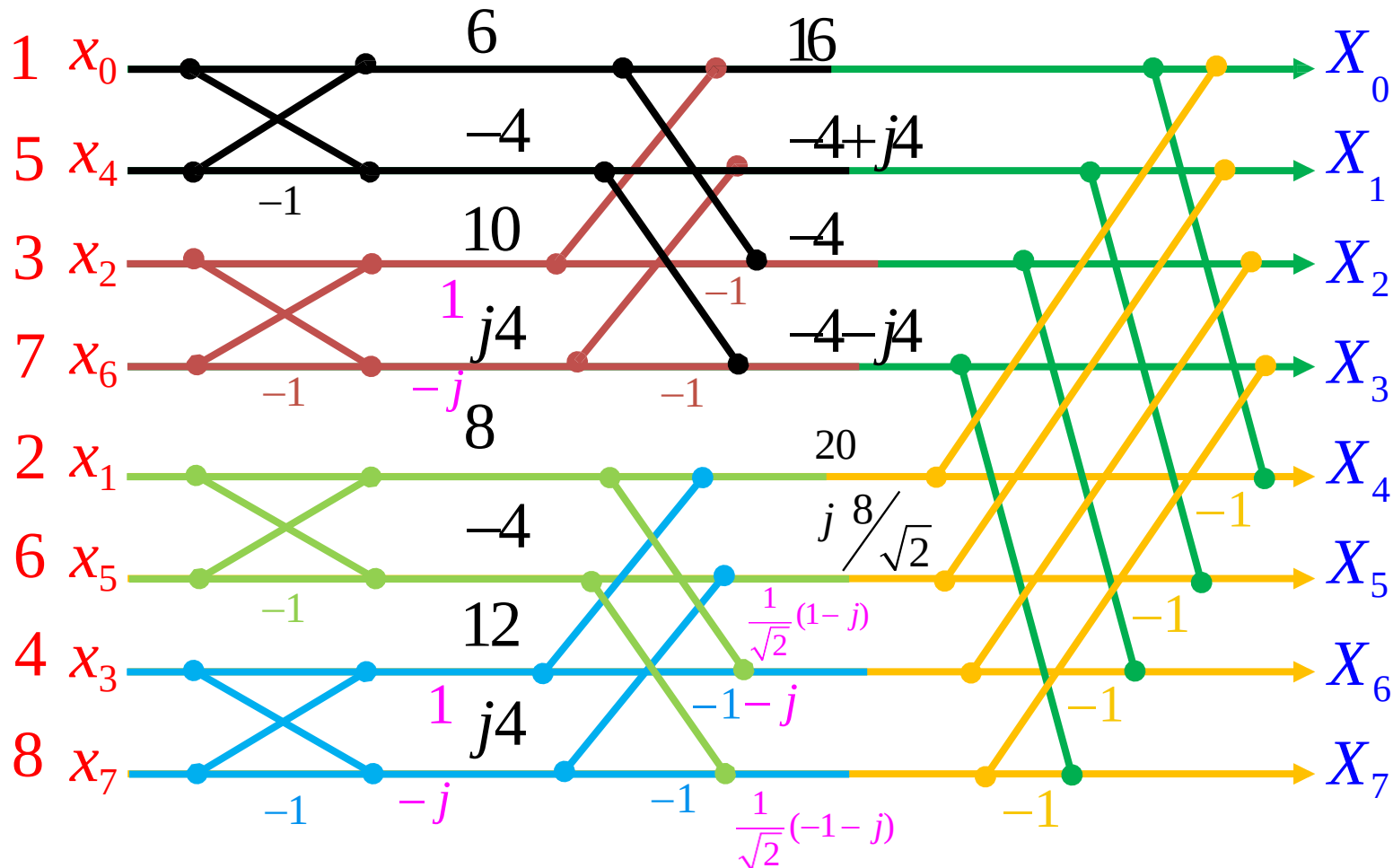
Example-10

Compute DFT of $x[n] = \{1, 2, 3, 4, 5, 6, 7, 8\}$ using DITFFT algorithm. (May19, 10 marks)



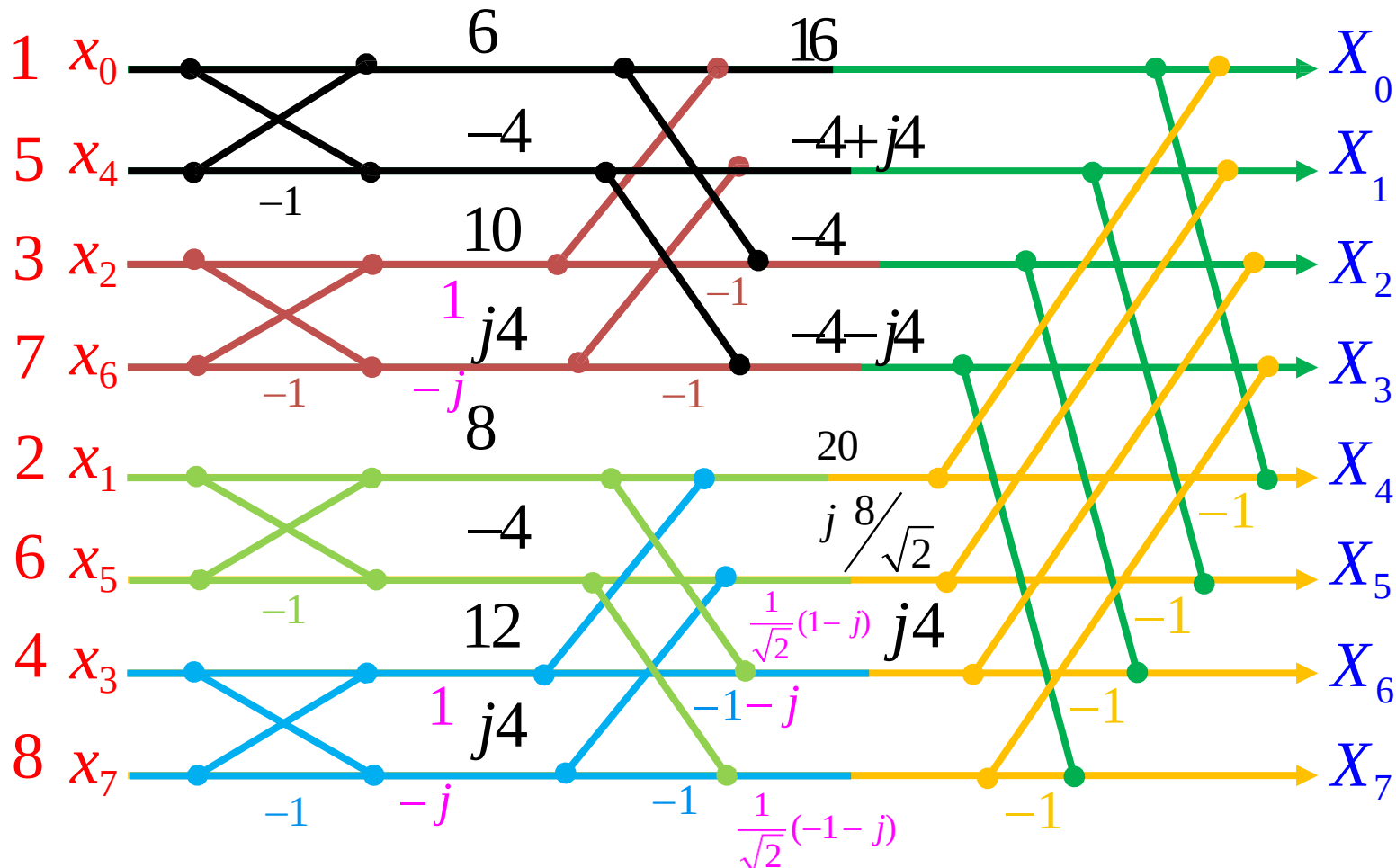
Example-10

Compute DFT of $x[n] = \{1,2,3,4,5,6,7,8\}$ using DITFFT algorithm. (May19,10 marks)



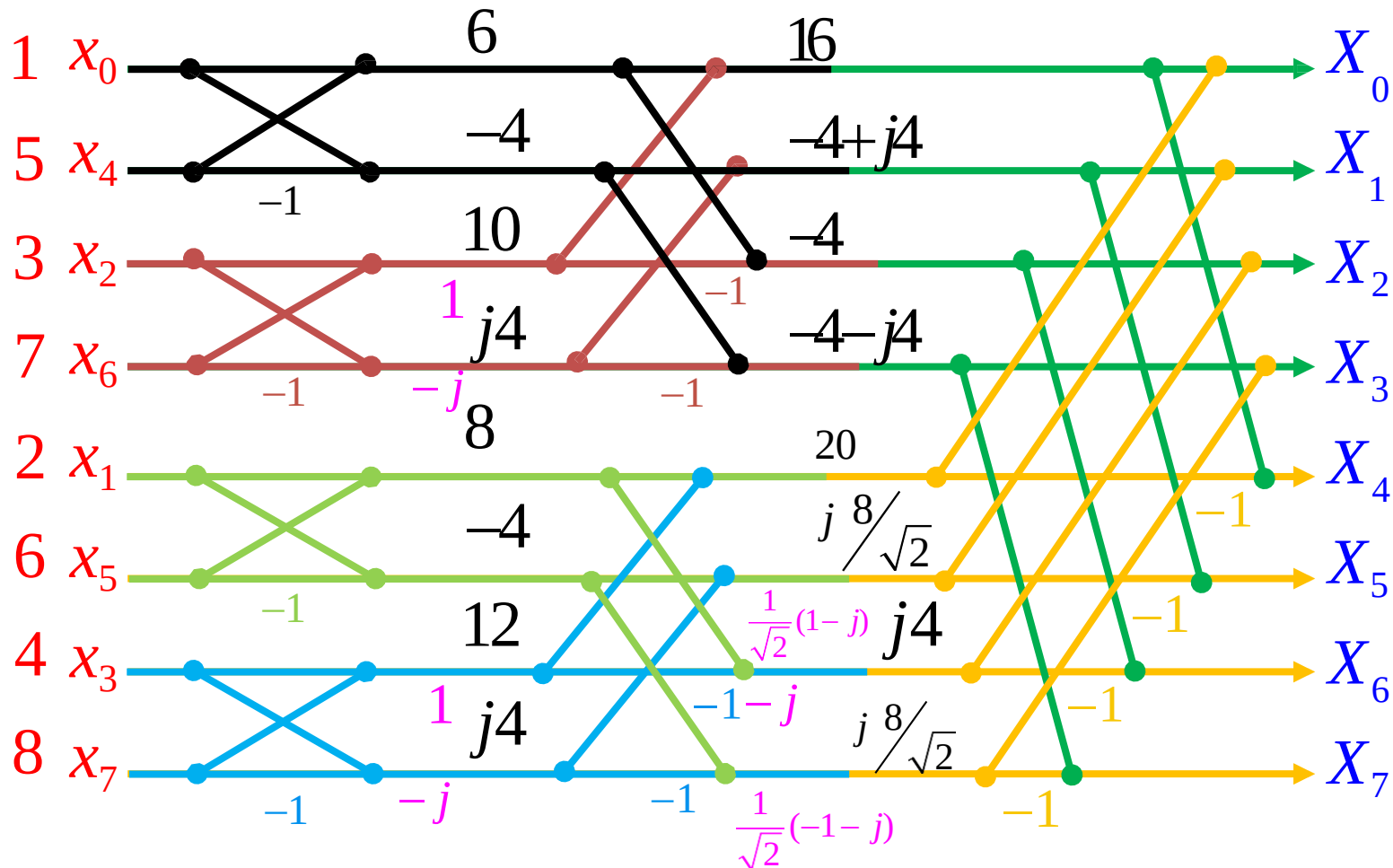
Example-10

Compute DFT of $x[n] = \{1, 2, 3, 4, 5, 6, 7, 8\}$ using DITFFT algorithm. (May19, 10 marks)



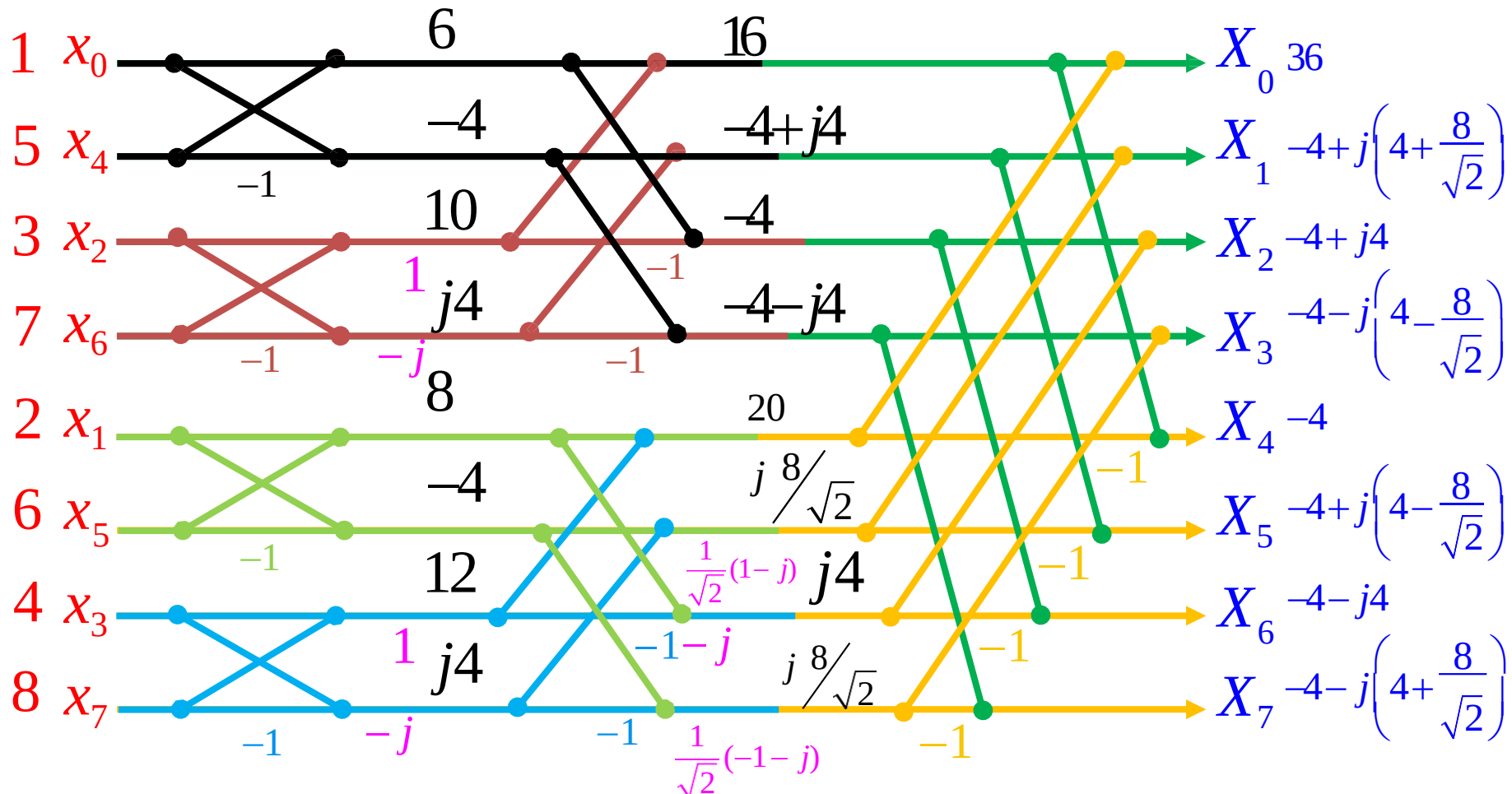
Example-10

Compute DFT of $x[n] = \{1, 2, 3, 4, 5, 6, 7, 8\}$ using DITFFT algorithm. (May19, 10 marks)



Example-10

Compute DFT of $x[n] = \{1, 2, 3, 4, 5, 6, 7, 8\}$ using DITFFT algorithm. (May19, 10 marks)



DIT-FFT

$$F_k = \sum_{n=0}^{N-1} x_n \cdot e^{-\frac{j2\pi kn}{N}}$$

$$\sum_{m=0}^{N/2-1} x_{2m} \cdot e^{-\frac{j2\pi k \cdot 2m}{N}}$$

$$e^{-\frac{j2\pi k}{N}} \sum_{m=0}^{N/2-1} x_{2m+1} \cdot e^{-\frac{j2\pi km}{N/2}}$$

$$\sum_{m=0}^{N/4-1} x_{4m} \cdot e^{-\frac{j2\pi km}{N/4}} \quad e^{-\frac{j4\pi k}{N}} \sum_{m=0}^{N/4-1} x_{4m+2} \cdot e^{-\frac{j2\pi km}{N/4}}$$

$$\sum_{m=0}^{N/4-1} x_{4m+1} \cdot e^{-\frac{j2\pi km}{N/4}} \quad e^{-\frac{j4\pi k}{N}} \sum_{m=0}^{N/4-1} x_{4m+3} \cdot e^{-\frac{j2\pi km}{N/4}}$$

Completed:
 $F_0 = 0$

[FFT Algorithm: Step-by-Step](#)

Decimation In Frequency FFT (DIF-FFT)



Decimation-In-Frequency FFT Algorithm

Decimation-In-Frequency FFT Algorithm

- Sequence $x[n]$ is decomposed into successively smaller subsequences.
- Considering the special case of **N an integer power of 2**, i.e., $N = 2^v$.
- Separating $X[k]$ into two $(N/2)$ -point sequences consisting of the even-indexed points in $X[k]$ and the odd-indexed points in $X[k]$.
- Even indexed $X[2r]$ and odd indexed $X[2r + 1]$ each $N/2$ long

Decimation-In-Frequency FFT Algorithm

- Sequence $x[n]$ is decomposed into successively smaller subsequences.
- Considering the special case of **N an integer power of 2**, i.e., $N = 2^v$.

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn} \quad k, n = 0, 1, \dots, N-1$$

- Separating $X[k]$ into two $(N/2)$ -point sequences consisting of the even-indexed points in $X[k]$ and the odd-indexed points in $X[k]$.
- Even indexed $X[2r]$ and odd indexed $X[2r + 1]$ each $N/2$ long

Decimation-In-Frequency FFT Algorithm

- Sequence $x[n]$ is decomposed into successively smaller subsequences.
- Considering the special case of **N an integer power of 2**, i.e., $N = 2^v$.

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn} \quad k, n = 0, 1, \dots, N-1$$

- Separating $X[k]$ into two $(N/2)$ -point sequences consisting of the even-indexed points in $X[k]$ and the odd-indexed points in $X[k]$.
- Even indexed $X[2r]$ and odd indexed $X[2r + 1]$ each $N/2$ long

Decimation-In-Frequency FFT Algorithm

- Sequence $x[n]$ is decomposed into successively smaller subsequences.
- Considering the special case of **N an integer power of 2**, i.e., $N = 2^v$.

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn} \quad k, n = 0, 1, \dots, N-1$$

- Separating $X[k]$ into two $(N/2)$ -point sequences consisting of the even-indexed points in $X[k]$ and the odd-indexed points in $X[k]$.
- Even indexed $X[2r]$ and odd indexed $X[2r + 1]$ each $N/2$ long

$$X[2r] = \sum_{n=0}^{N-1} x[n] W_N^{2rn} \quad r = 0, 1, \dots, (N/2) - 1$$

Decimation-In-Frequency FFT Algorithm

Decimation-In-Frequency FFT Algorithm

$$X[2r] = \sum_{n=0}^{N/2-1} x[n] W_N^{2rn} + \sum_{n=N/2}^{N-1} x[n] W_N^{2rn}$$

Decimation-In-Frequency FFT Algorithm

$$X[2r] = \sum_{n=0}^{N/2-1} x[n] W_N^{2rn} + \sum_{n=N/2}^{N-1} x[n] W_N^{2rn}$$

$$X[2r] = \sum_{n=0}^{N/2-1} x[n] \left(W_N^2 \right)^{rn} + \sum_{n=0}^{N/2-1} x[n + (N/2)] \left(W_N^2 \right)^{r[n + (N/2)]}$$

Decimation-In-Frequency FFT Algorithm

$$X[2r] = \sum_{n=0}^{N/2-1} x[n] W_N^{2r n} + \sum_{n=N/2}^{N-1} x[n] W_N^{2r n}$$

$$X[2r] = \sum_{n=0}^{N/2-1} x[n] \left(W_N^2 \right)^{r n} + \sum_{n=0}^{N/2-1} x[n + (N/2)] \left(W_N^2 \right)^{r [n + (N/2)]}$$

$$X[2r] = \sum_{n=0}^{N/2-1} (x[n] + x[n + (N/2)]) W_{N/2}^{r n} \quad r = 0, 1, \dots, \left(\frac{N}{2} \right) - 1$$

Decimation-In-Frequency FFT Algorithm

$$X[2r] = \sum_{n=0}^{N/2-1} x[n] W_N^{2rn} + \sum_{n=N/2}^{N-1} x[n] W_N^{2rn}$$

$$X[2r] = \sum_{n=0}^{N/2-1} x[n] \left(W_N^2\right)^{rn} + \sum_{n=0}^{N/2-1} x[n + (N/2)] \left(W_N^2\right)^{r[n+(N/2)]}$$

$$X[2r] = \sum_{n=0}^{N/2-1} (x[n] + x[n + (N/2)]) W_{N/2}^{rn} \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

- $(N/2)$ -point DFT of the $(N/2)$ -point sequence obtained by adding the first half and the last half of the input sequence.
 - Adding the two halves of the input sequence represents **time aliasing**

Decimation-In-Frequency FFT Algorithm

Decimation-In-Frequency FFT Algorithm

$$X[2r+1] = \sum_{n=0}^{N-1} x[n] W_N^{(2r+1)n} \quad r = 0, 1, \dots, (N/2) - 1$$

Decimation-In-Frequency FFT Algorithm

$$X[2r+1] = \sum_{n=0}^{N-1} x[n] W_N^{(2r+1)n} \quad r = 0, 1, \dots, (N/2)-1$$

$$X[2r+1] = \sum_{n=0}^{N/2-1} x[n] W_N^{(2r+1)n} + \sum_{n=N/2}^{N-1} x[n] W_N^{(2r+1)n}$$

Decimation-In-Frequency FFT Algorithm

$$X[2r + 1] = \sum_{n=0}^{N-1} x[n] W_N^{(2r+1)n} \quad r = 0, 1, \dots, (N/2) - 1$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} x[n] W_N^{(2r+1)n} + \sum_{n=N/2}^{N-1} x[n] W_N^{(2r+1)n}$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} x[n] W_N^{(2r+1)n} + \sum_{n=0}^{N/2-1} x[n + (N/2)] W_N^{(2r+1)[n + (N/2)]}$$

Decimation-In-Frequency FFT Algorithm

$$X[2r+1] = \sum_{n=0}^{N-1} x[n] W_N^{(2r+1)n} \quad r = 0, 1, \dots, (N/2) - 1$$

$$X[2r+1] = \sum_{n=0}^{N/2-1} x[n] W_N^{(2r+1)n} + \sum_{n=N/2}^{N-1} x[n] W_N^{(2r+1)n}$$

$$X[2r+1] = \sum_{n=0}^{N/2-1} x[n] W_N^{(2r+1)n} + \sum_{n=0}^{N/2-1} x[n + (N/2)] W_N^{(2r+1)[n + (N/2)]}$$

$$X[2r+1] = \sum_{n=0}^{N/2-1} (x[n] - x[n + (N/2)]) W_N^{(2r+1)n} \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

Decimation-In-Frequency FFT Algorithm

$$X[2r + 1] = \sum_{n=0}^{N-1} x[n] \mathbf{W}_N^{(2r+1)n} \quad r = 0, 1, \dots, (N/2) - 1$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} x[n] \mathbf{W}_N^{(2r+1)n} + \sum_{n=N/2}^{N-1} x[n] \mathbf{W}_N^{(2r+1)n}$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} x[n] \mathbf{W}_N^{(2r+1)n} + \sum_{n=0}^{N/2-1} x[n + (N/2)] \mathbf{W}_N^{(2r+1)[n+(N/2)]}$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} (x[n] - x[n + (N/2)]) \mathbf{W}_N^{(2r+1)n} \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} (x[n] - x[n + (N/2)]) \mathbf{W}_{N/2}^{rn} \mathbf{W}_N^n$$

Decimation-In-Frequency FFT Algorithm

Decimation-In-Frequency FFT Algorithm

$$X[2r] = \sum_{n=0}^{N/2-1} (x[n] + x[n + (N/2)]) w_{N/2}^{rn} \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

Decimation-In-Frequency FFT Algorithm

$$X[2r] = \sum_{n=0}^{N/2-1} (x[n] + x[n + (N/2)]) W_{N/2}^{rn} \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} \left(x[n] - x\left[n + \frac{N}{2}\right] \right) W_{N/2}^{rn} W_N^n \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

Decimation-In-Frequency FFT Algorithm

$$X[2r] = \sum_{n=0}^{N/2-1} (x[n] + x[n + (N/2)]) W_{N/2}^{rn} \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} \left(x[n] - x\left[n + \frac{N}{2}\right] \right) W_{N/2}^{rn} W_N^n \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

- Let two $(N/2)$ -point sequences be generated as,

Decimation-In-Frequency FFT Algorithm

$$X[2r] = \sum_{n=0}^{N/2-1} (x[n] + x[n + (N/2)]) W_{N/2}^{rn} \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} \left(x[n] - x\left[n + \frac{N}{2}\right] \right) W_{N/2}^{rn} W_N^n \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

- Let two $(N/2)$ -point sequences be generated as,

$$g[n] = x[n] + x[n + (N/2)] \quad h[n] = x[n] - x[n + (N/2)]$$

Decimation-In-Frequency FFT Algorithm

$$X[2r] = \sum_{n=0}^{N/2-1} (x[n] + x[n + (N/2)]) W_{N/2}^{rn} \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} \left(x[n] - x\left[n + \frac{N}{2}\right] \right) W_{N/2}^{rn} W_N^n \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

- Let two $(N/2)$ -point sequences be generated as,

$$g[n] = x[n] + x[n + (N/2)] \quad h[n] = x[n] - x[n + (N/2)]$$

- Separating $G[k]$ and $H[k]$ into $(N/2)$ -point sequences consisting of the even-indexed points and the odd-indexed points

Decimation-In-Frequency FFT Algorithm

$$X[2r] = \sum_{n=0}^{N/2-1} (x[n] + x[n + (N/2)]) W_{N/2}^{rn} \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} \left(x[n] - x\left[n + \frac{N}{2}\right] \right) W_{N/2}^{rn} W_N^n \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

- Let two $(N/2)$ -point sequences be generated as,

$$g[n] = x[n] + x[n + (N/2)] \quad h[n] = x[n] - x[n + (N/2)]$$

- Separating $G[k]$ and $H[k]$ into $(N/2)$ -point sequences consisting of the even-indexed points and the odd-indexed points
- Repeating the procedure to calculate smaller FFTs.
 - Repeat until 2-point FFT.

Decimation-In-Frequency FFT Algorithm

$$X[2r] = \sum_{n=0}^{N/2-1} (x[n] + x[n + (N/2)]) W_{N/2}^{rn} \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} \left(x[n] - x\left[n + \frac{N}{2}\right] \right) W_{N/2}^{rn} W_N^n \quad r = 0, 1, \dots, \left(\frac{N}{2}\right) - 1$$

- Let two $(N/2)$ -point sequences be generated as,

$$g[n] = x[n] + x[n + (N/2)] \quad h[n] = x[n] - x[n + (N/2)]$$

- Separating $G[k]$ and $H[k]$ into $(N/2)$ -point sequences consisting of the even-indexed points and the odd-indexed points
- Repeating the procedure to calculate smaller FFTs.
 - Repeat until 2-point FFT.

4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

X_0

X_1

X_2

X_3

4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

x_0

X_0

x_1

X_2

x_2

X_1

x_3

X_3

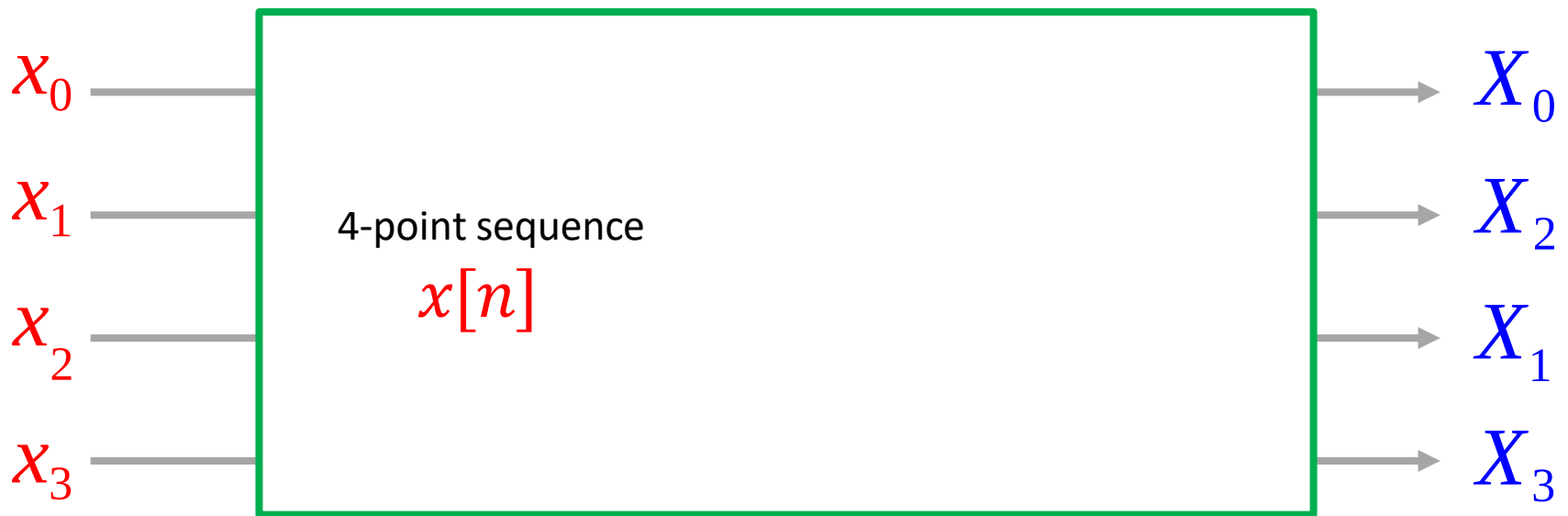
4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$



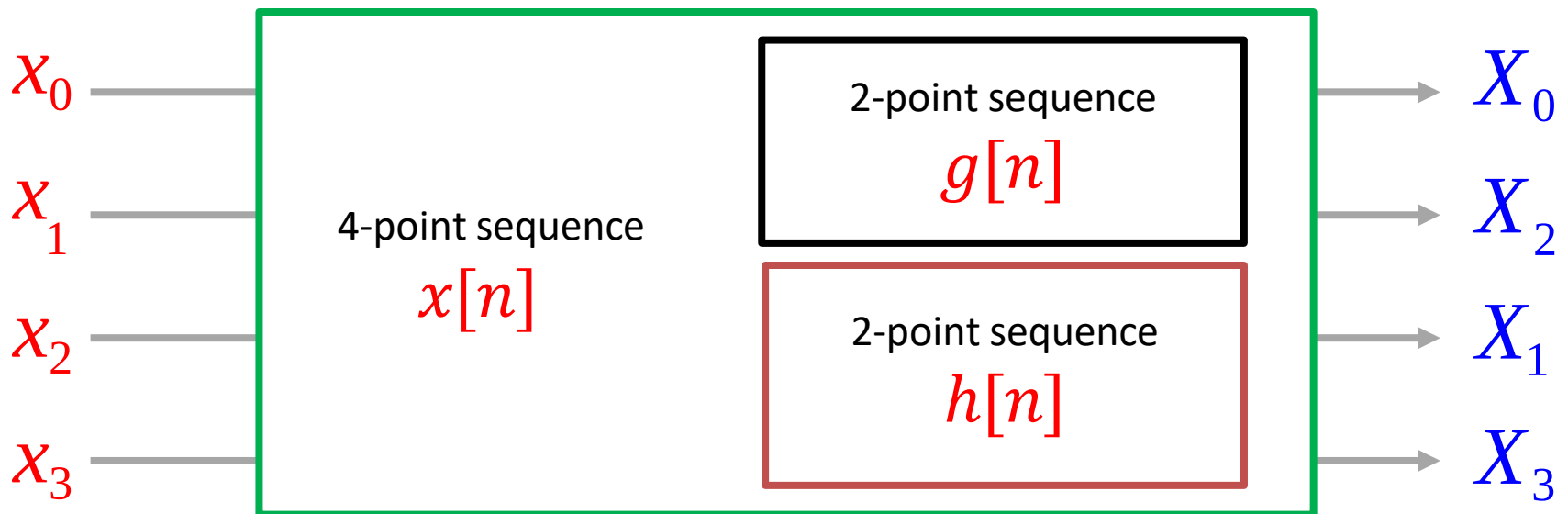
4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$



4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$



4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

X_0

X_1

X_2

X_3

4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

x_0

X_0

x_1

X_2

x_2

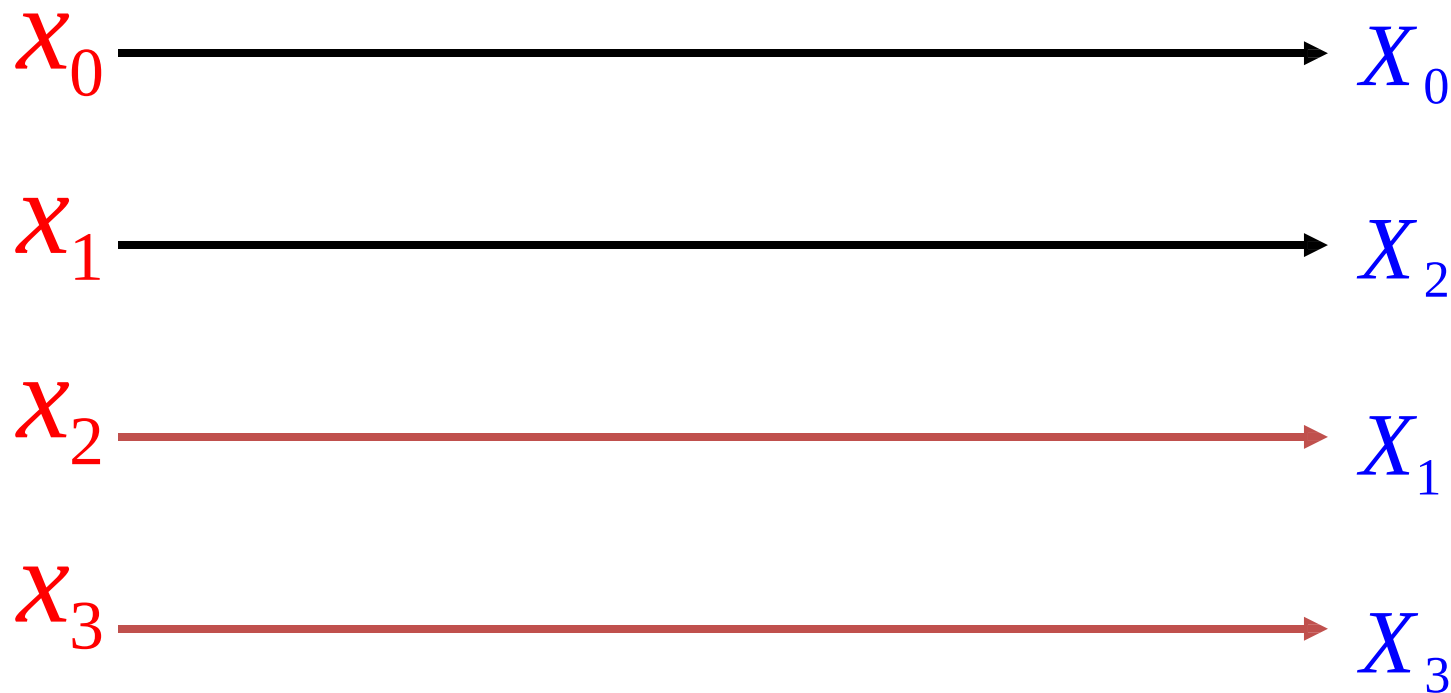
X_1

x_3

X_3

4-point DIFFFT

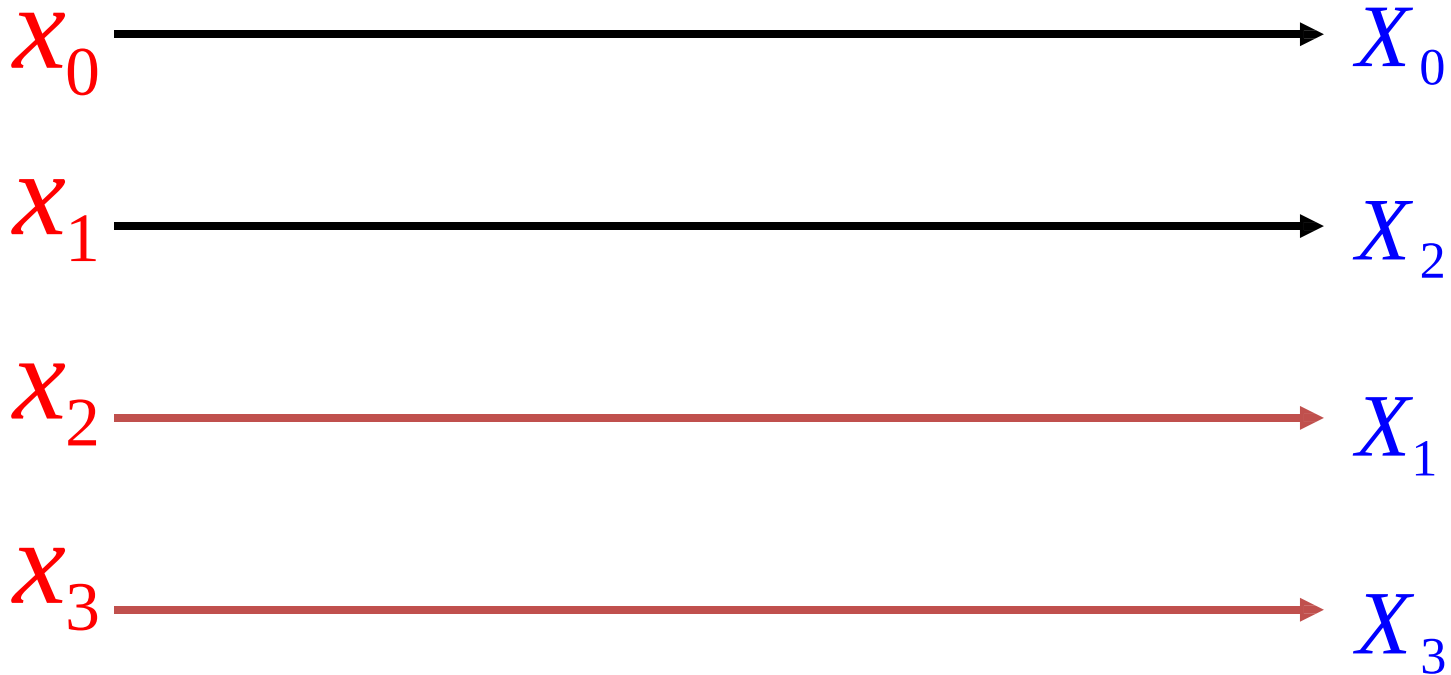
- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$



4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r] = \sum_{n=0}^1 (x[n] + x[n+2]) w_2^{rn} \quad r = 0, 1$$



4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r] = \sum_{n=0}^1 (x[n] + x[n+2]) w_2^{rn} \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$

x_0 $\longrightarrow$ X_0

x_1 $\longrightarrow$ X_2

x_2 $\longrightarrow$ X_1

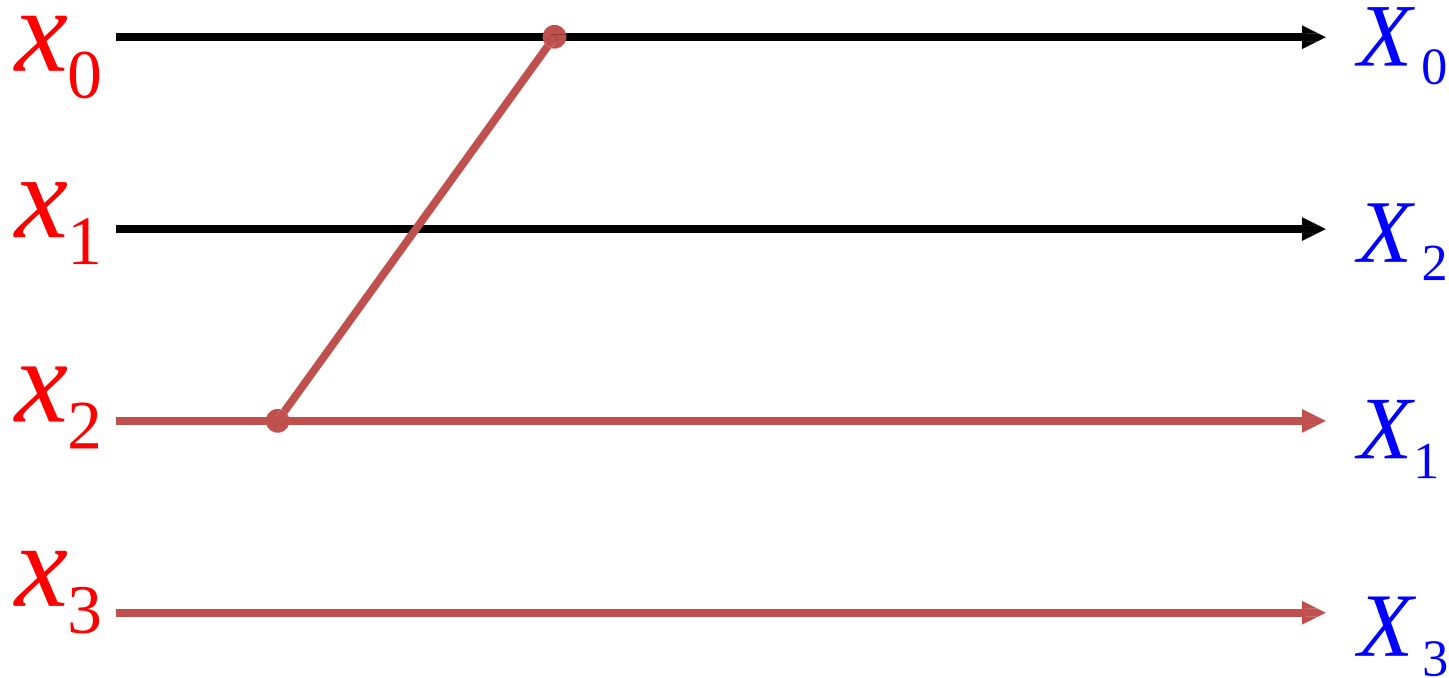
x_3 $\longrightarrow$ X_3

4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r] = \sum_{n=0}^1 (x[n] + x[n+2]) w_2^{rn} \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$

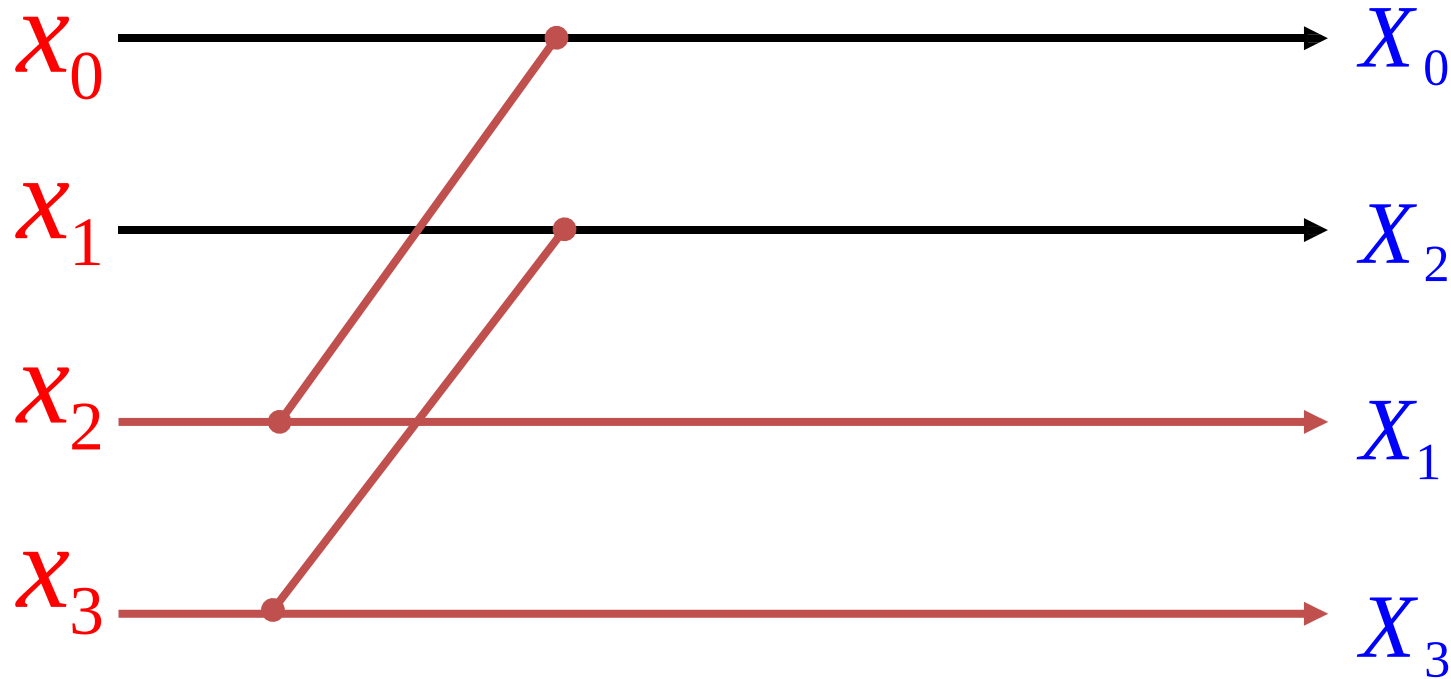


4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r] = \sum_{n=0}^1 (x[n] + x[n+2]) w_2^{rn} \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$

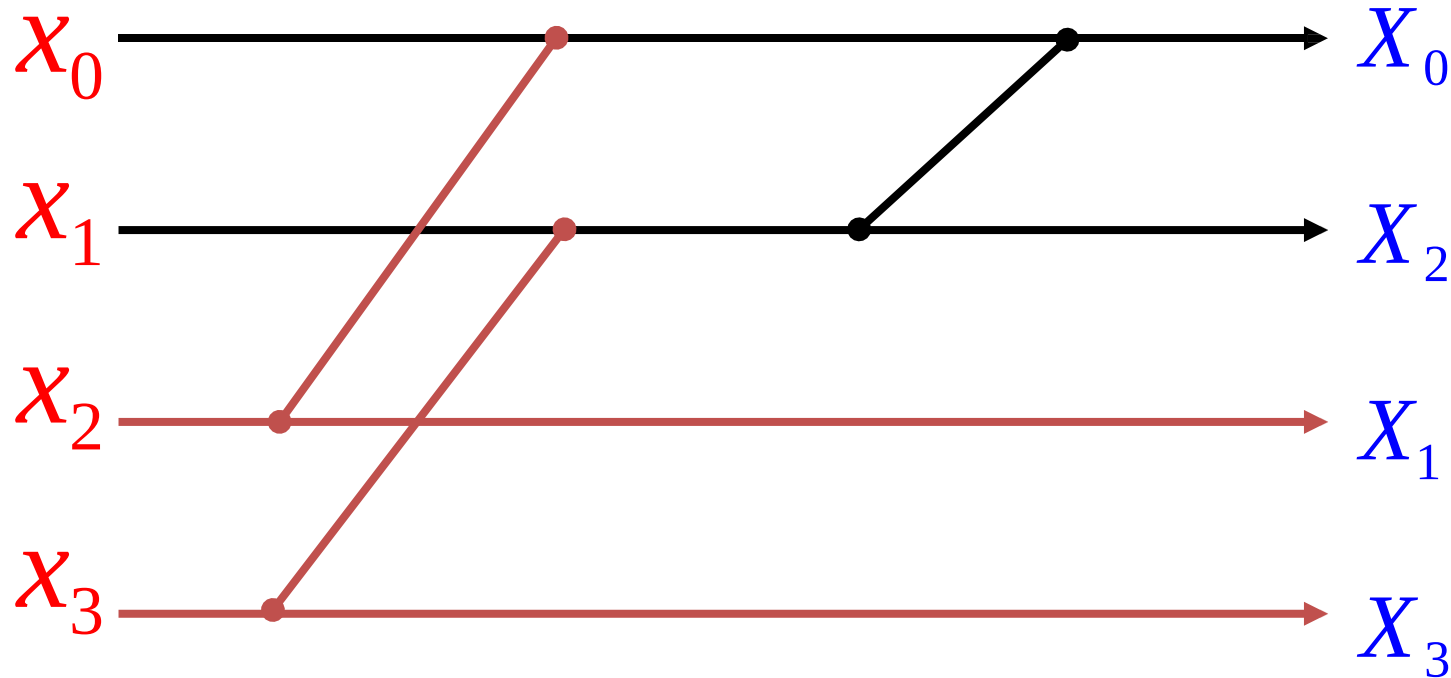


4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r] = \sum_{n=0}^1 (x[n] + x[n+2]) w_2^{rn} \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$

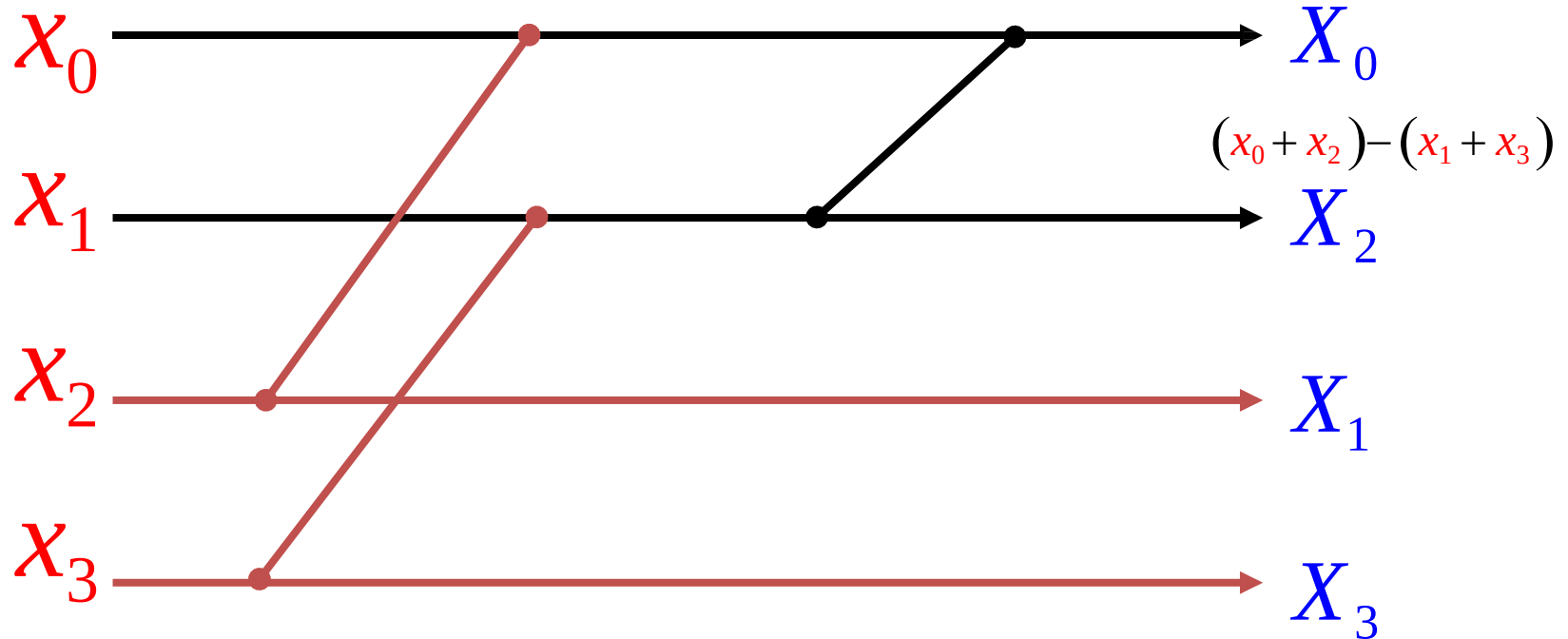


4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r] = \sum_{n=0}^1 (x[n] + x[n+2]) w_2^{rn} \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$
 $(x_0 + x_2) - (x_1 + x_3)$

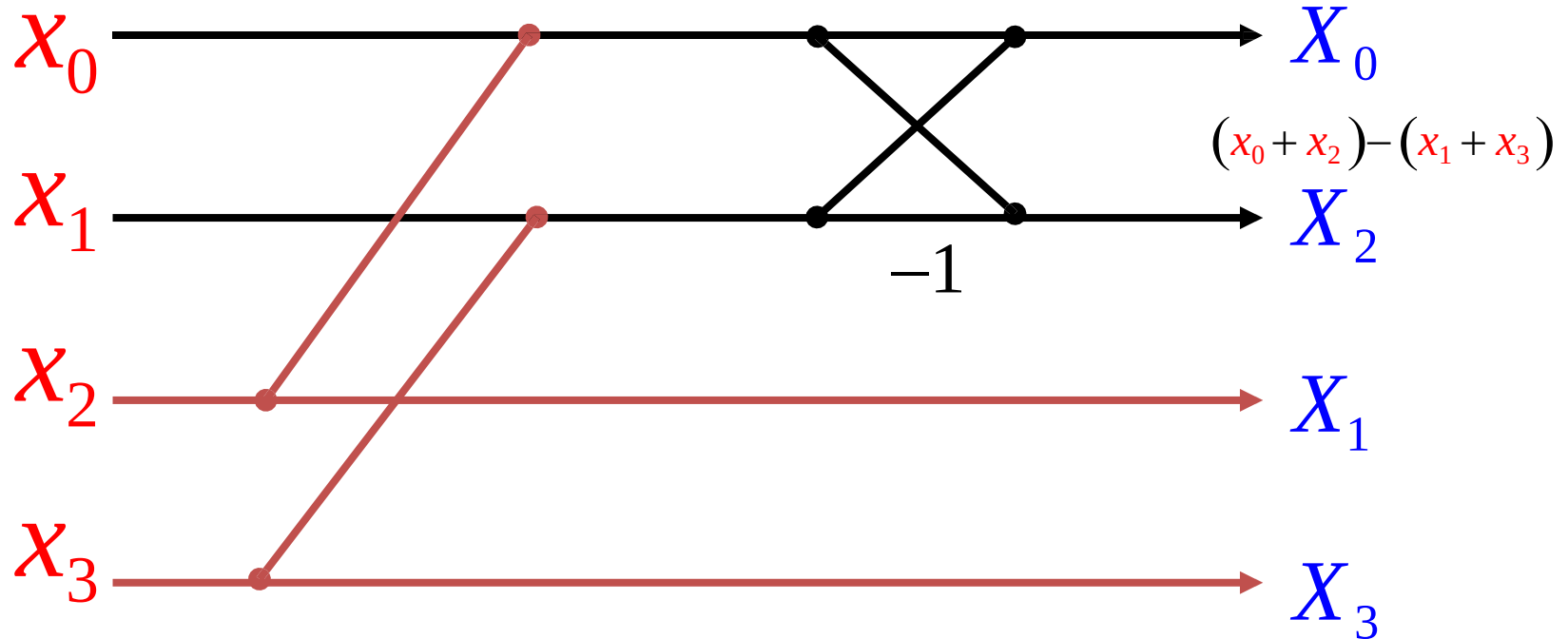


4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r] = \sum_{n=0}^1 (x[n] + x[n+2]) w_2^{rn} \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$
 $(x_0 + x_2) - (x_1 + x_3)$

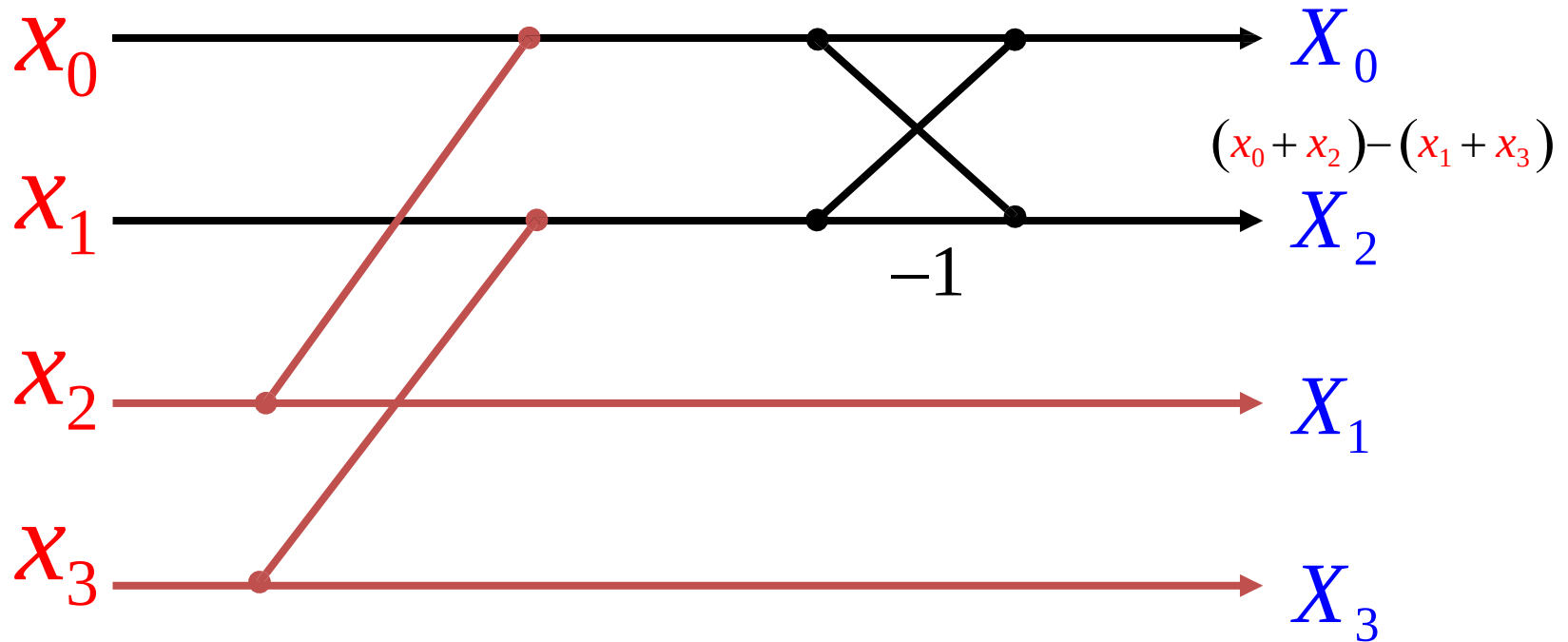


4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r+1] = \sum_{n=0}^1 (x[n] - x[n+2]) w_2^{rn} w_4^n \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$
 $(x_0 + x_2) - (x_1 + x_3)$

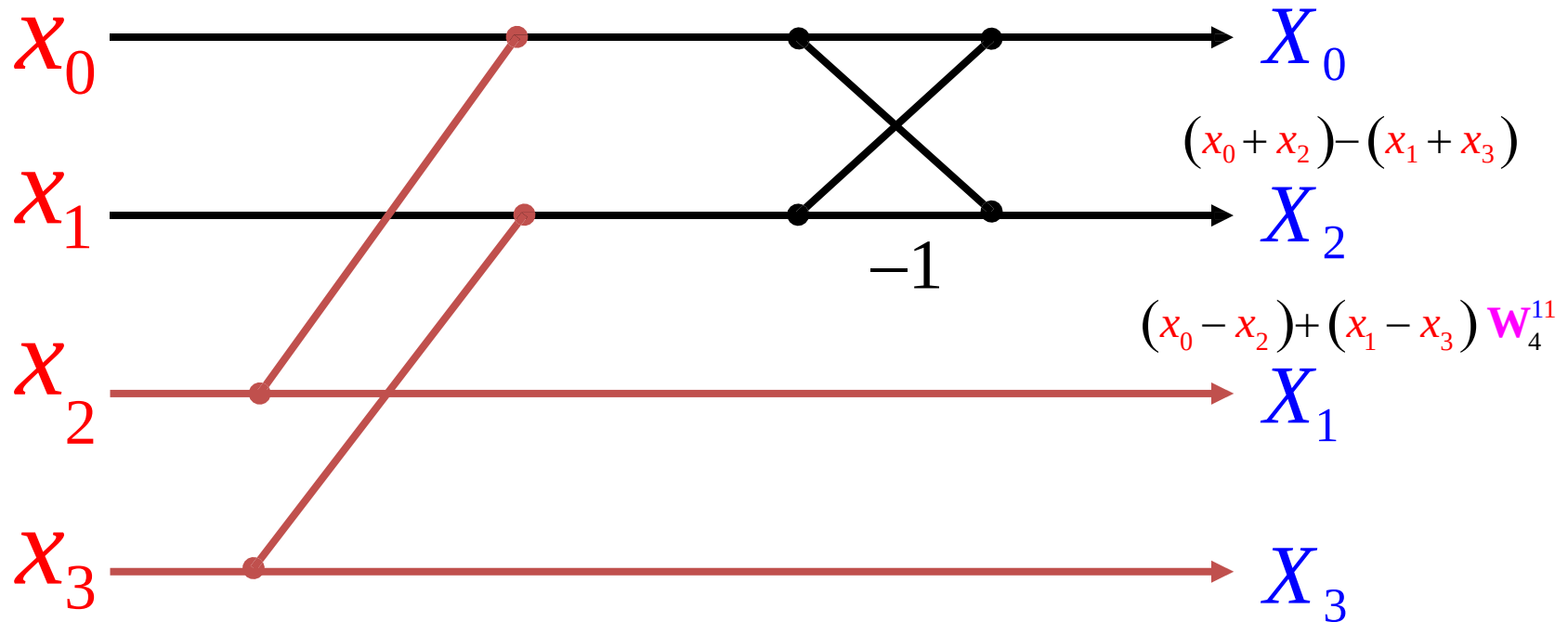


4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r+1] = \sum_{n=0}^1 (x[n] - x[n+2]) w_2^{rn} w_4^n \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$
 $(x_0 + x_2) - (x_1 + x_3)$

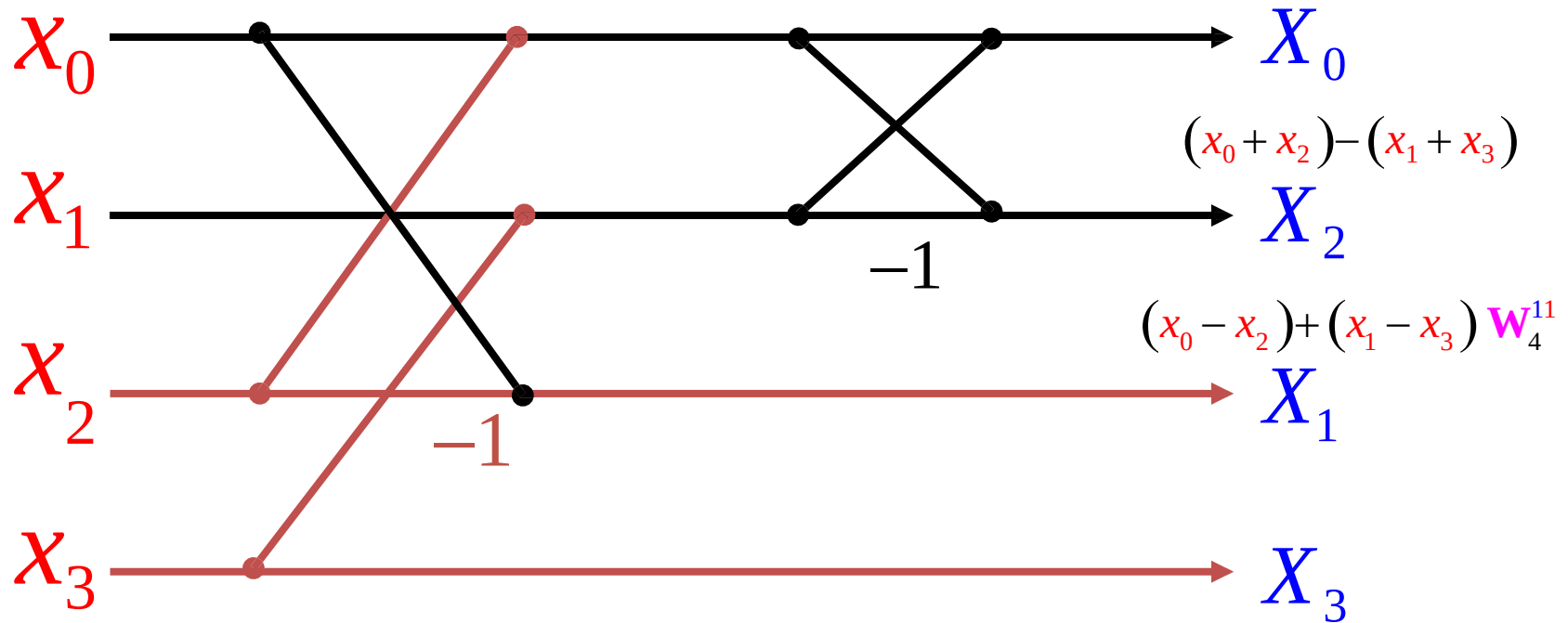


4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r+1] = \sum_{n=0}^1 (x[n] - x[n+2]) w_2^{rn} w_4^n \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$
 $(x_0 + x_2) - (x_1 + x_3)$

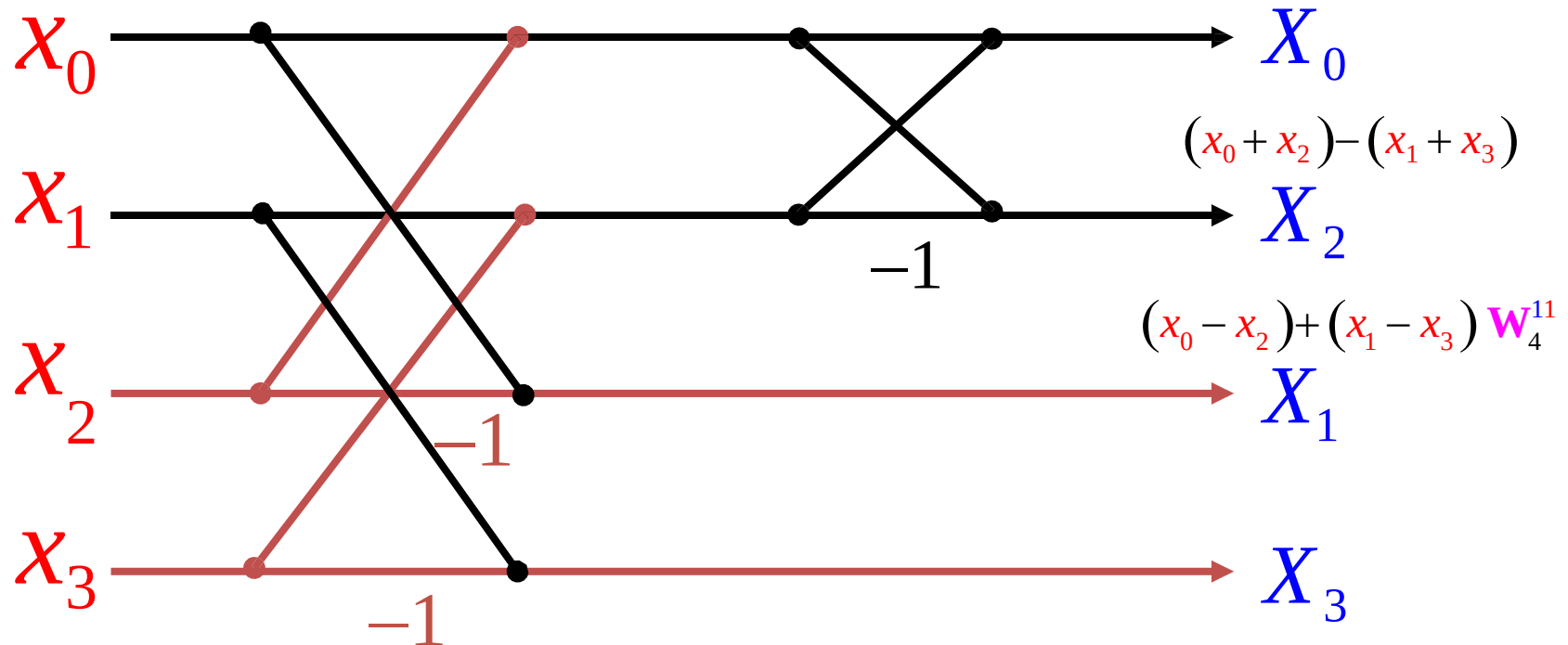


4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r+1] = \sum_{n=0}^1 (x[n] - x[n+2]) w_2^{rn} w_4^n \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$
 $(x_0 + x_2) - (x_1 + x_3)$

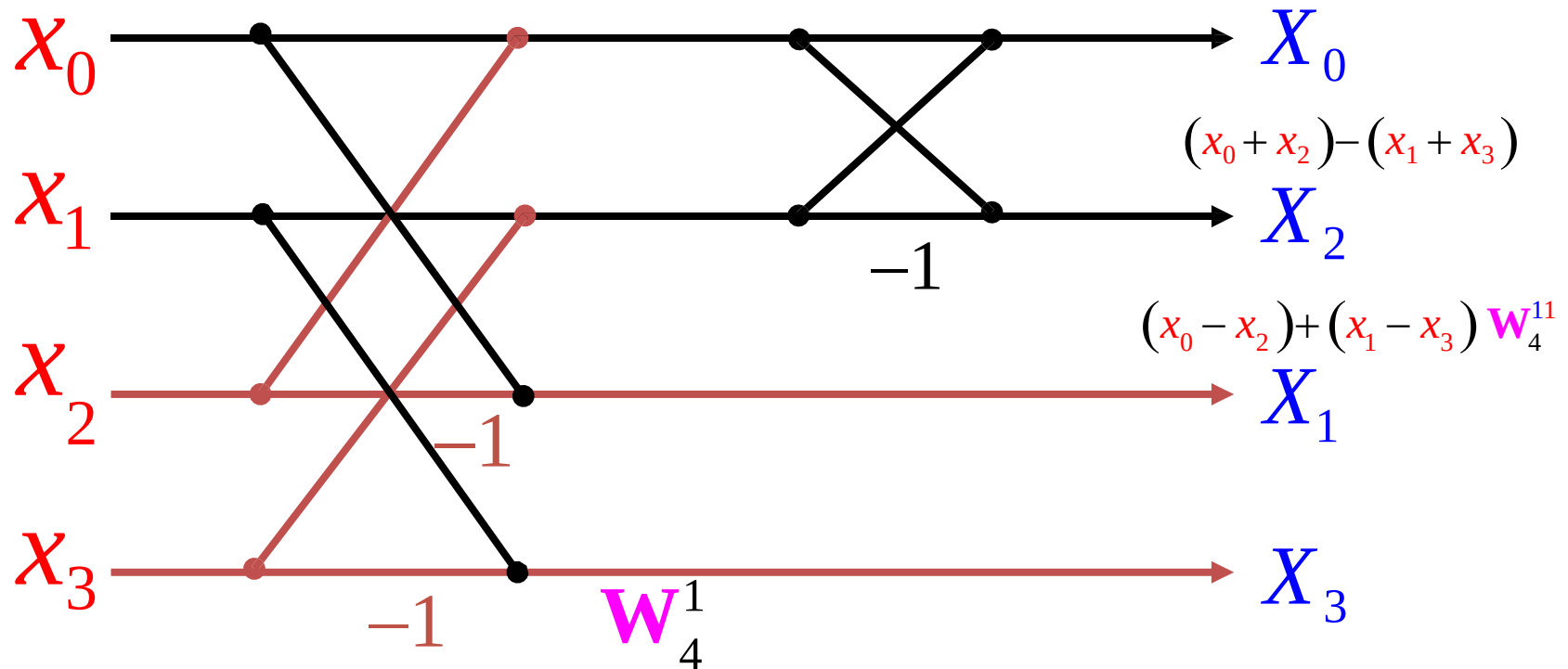


4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r+1] = \sum_{n=0}^1 (x[n] - x[n+2]) w_2^{rn} w_4^n \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$
 $(x_0 + x_2) - (x_1 + x_3)$

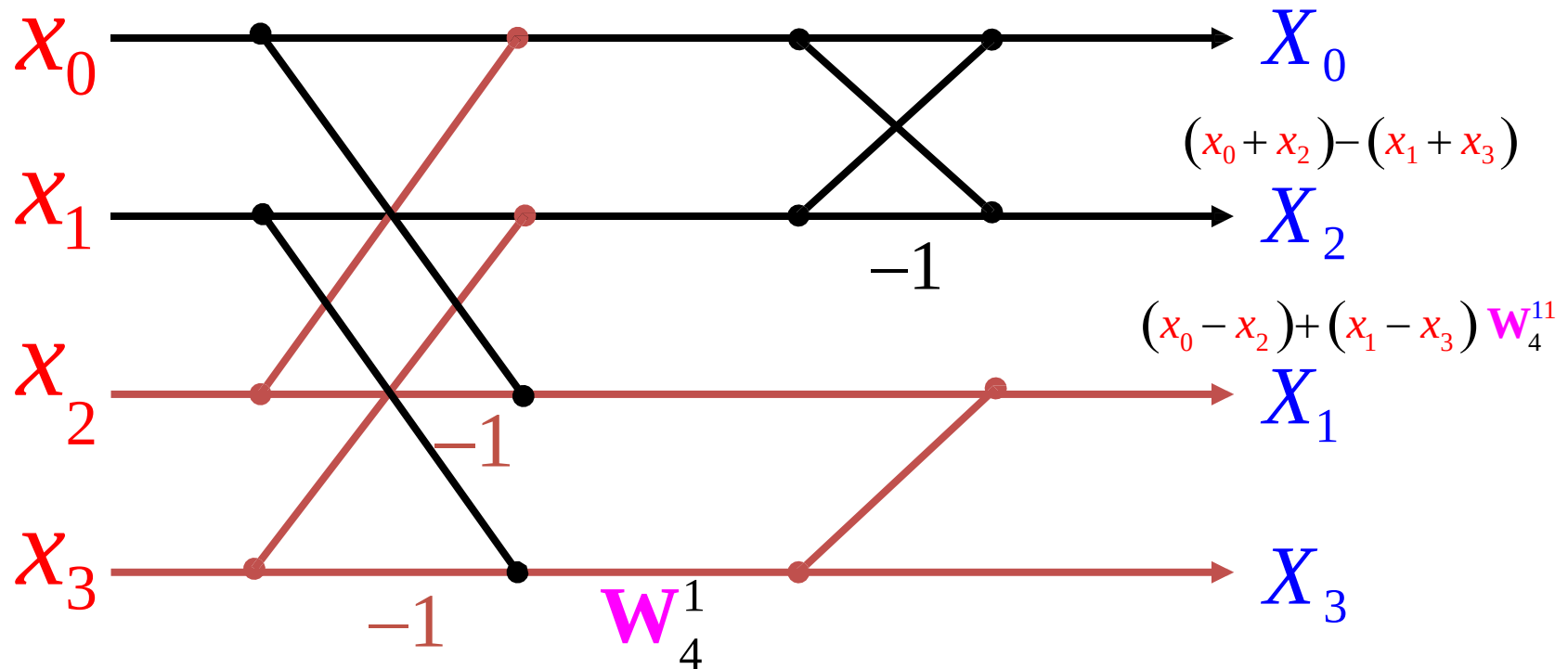


4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r+1] = \sum_{n=0}^1 (x[n] - x[n+2]) w_2^{rn} w_4^n \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$
 $(x_0 + x_2) - (x_1 + x_3)$

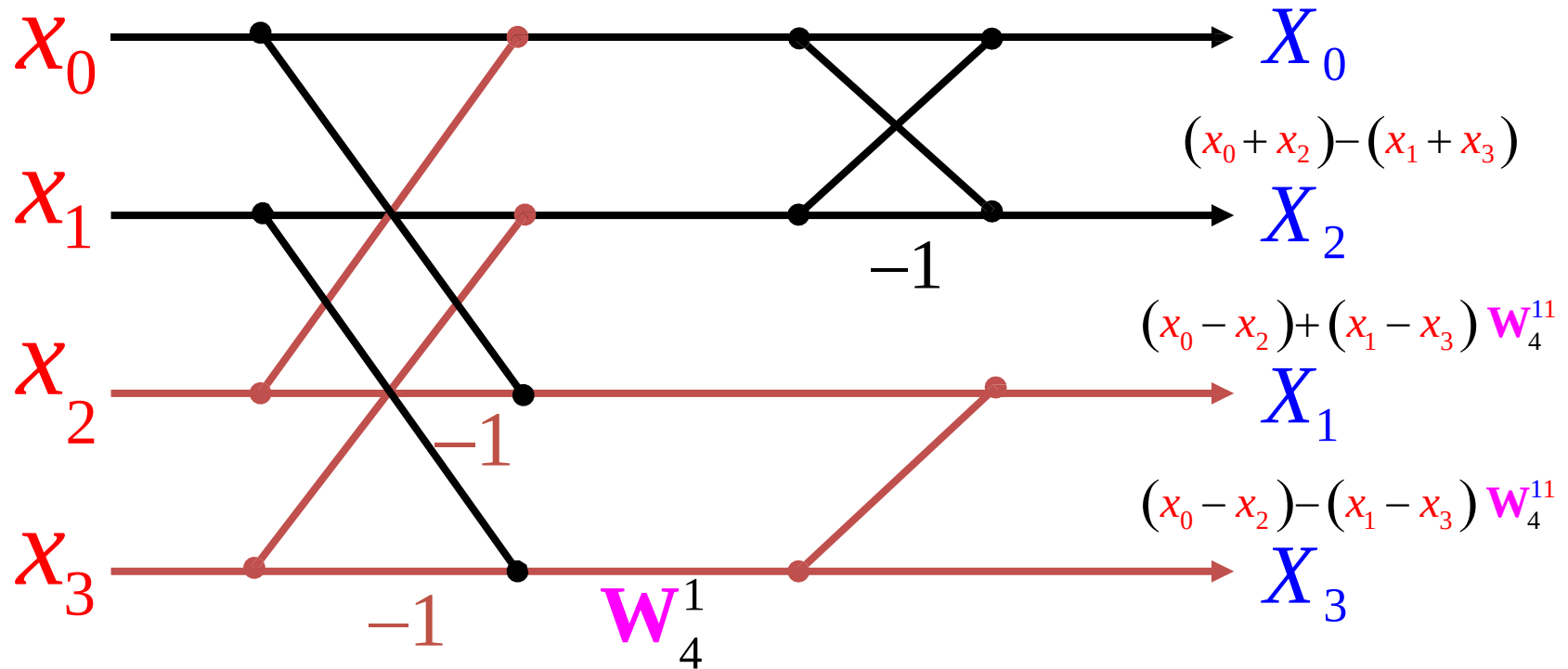


4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

$$X[2r+1] = \sum_{n=0}^1 (x[n] - x[n+2]) w_2^{rn} w_4^n \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$
 $(x_0 + x_2) - (x_1 + x_3)$

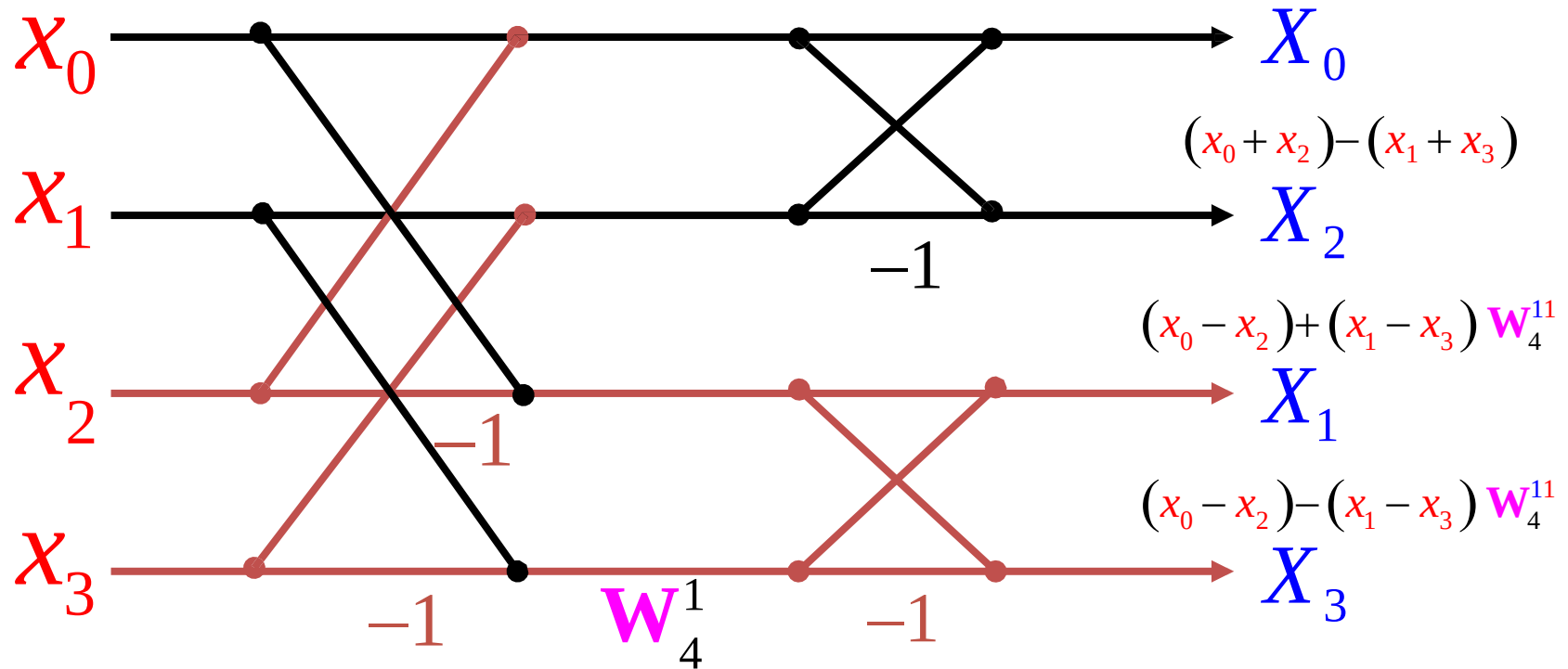


4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$

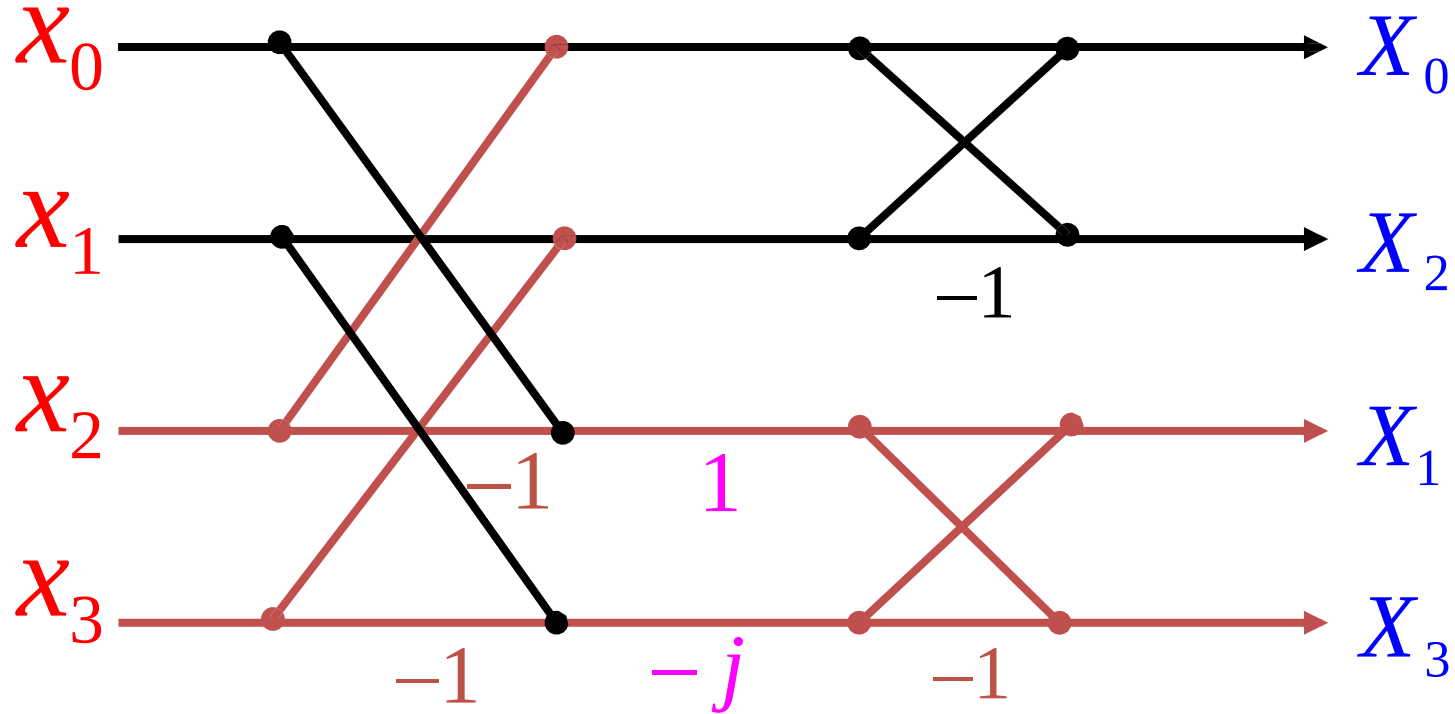
$$X[2r+1] = \sum_{n=0}^1 (x[n] - x[n+2]) w_2^{rn} w_4^n \quad r = 0, 1$$

$(x_0 + x_2) + (x_1 + x_3)$
 $(x_0 + x_2) - (x_1 + x_3)$



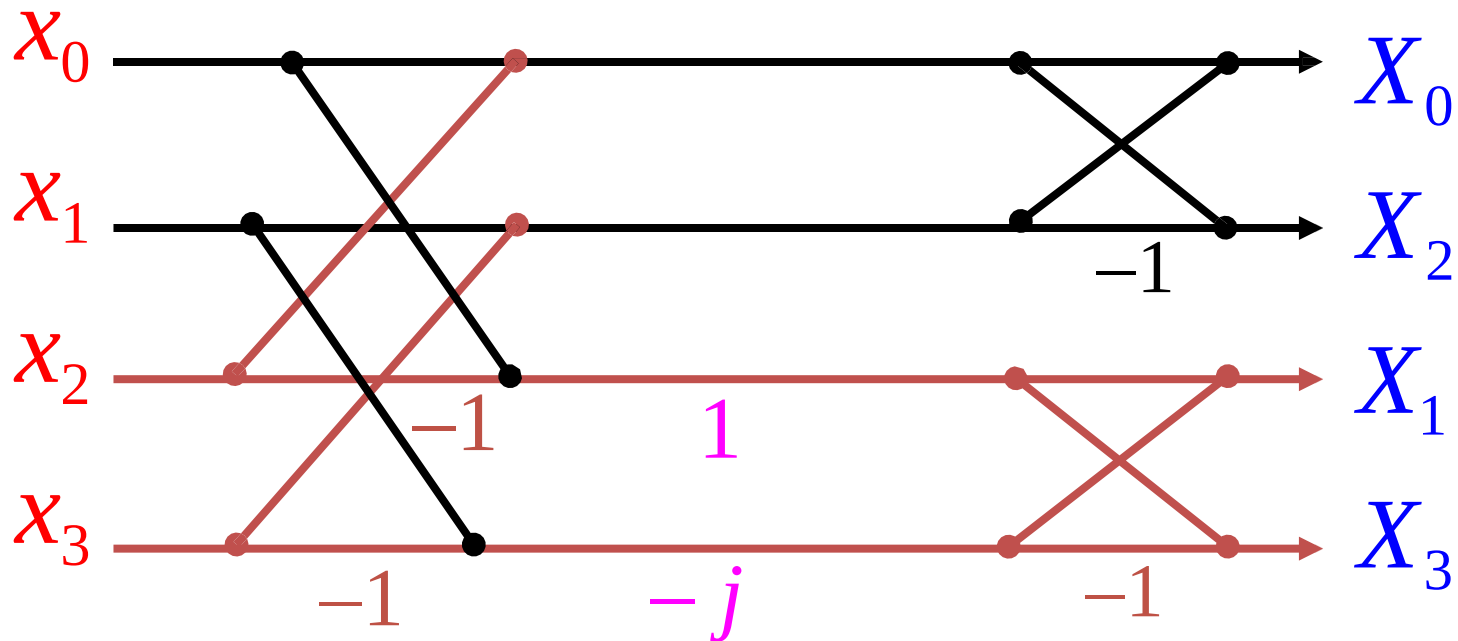
4-point DIFFFT

- Let $x[n] = \{x_0, x_1, x_2, x_3\} \xrightarrow{DFT} X(k) = \{X_0, X_1, X_2, X_3\}$



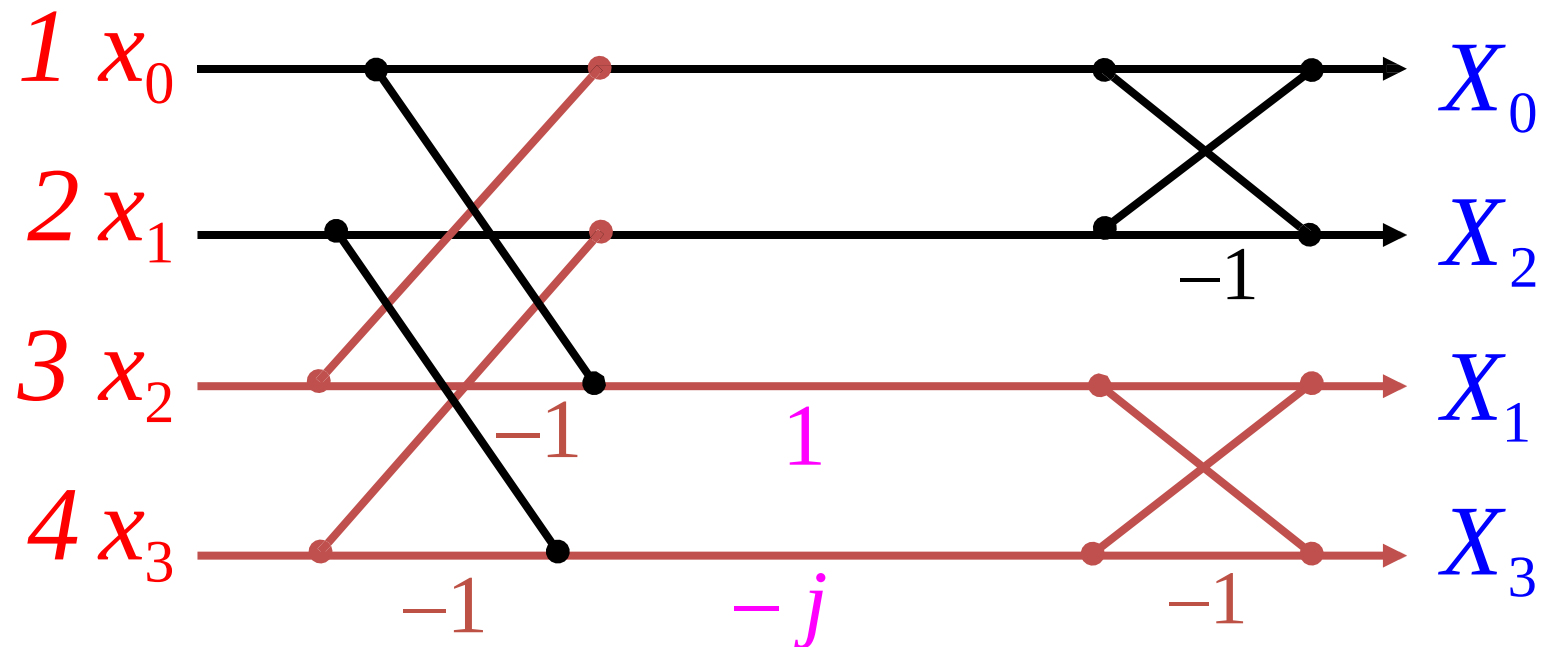
Example-11

Compute DFT of $x[n] = \{1, 2, 1, 2\}$ using DIFFFT algorithm. (May19, 05 marks)



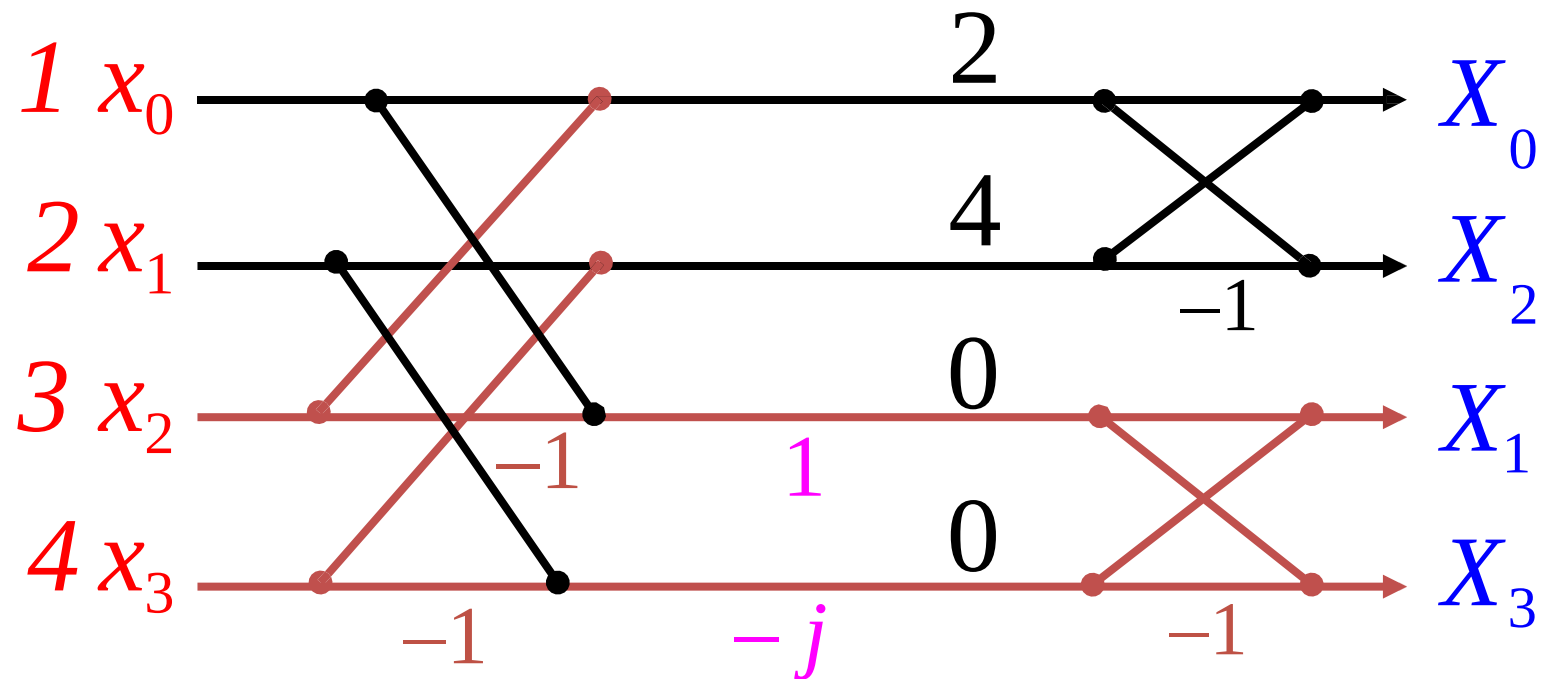
Example-11

Compute DFT of $x[n] = \{1, 2, 1, 2\}$ using DIFFFT algorithm. (May19, 05 marks)



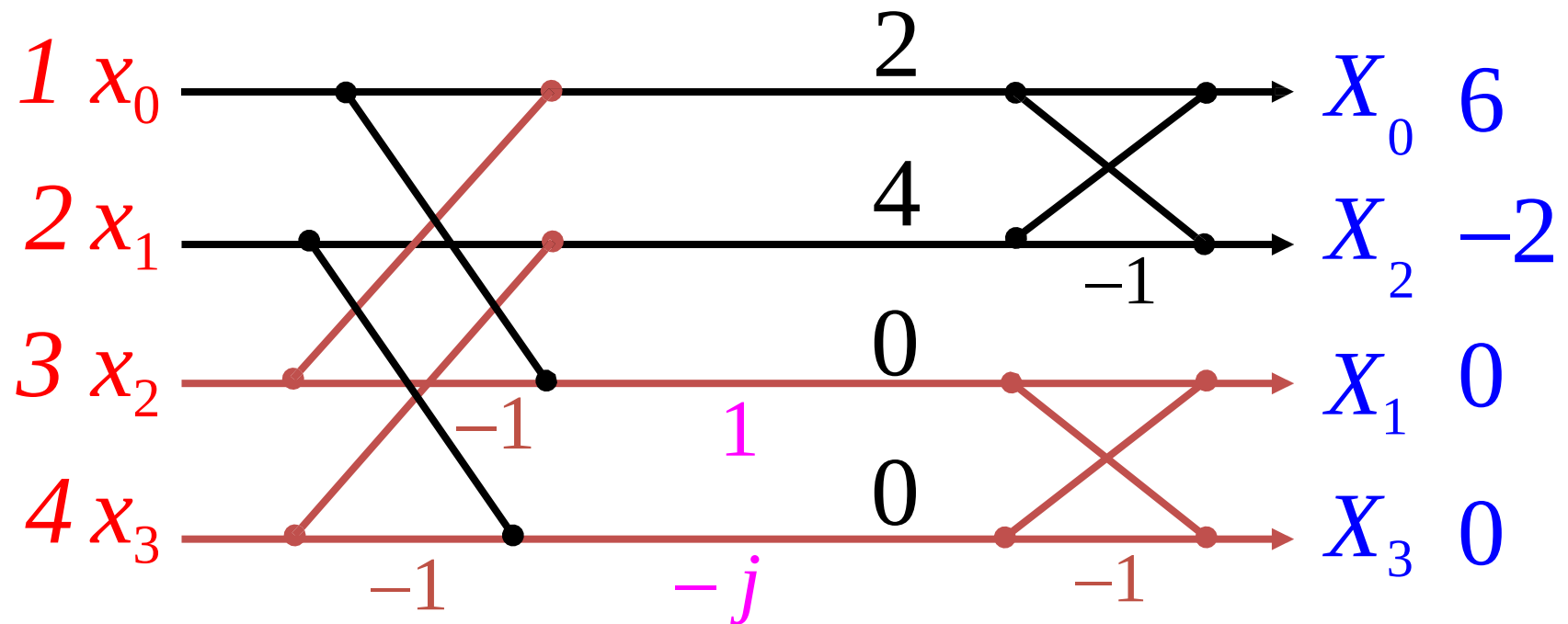
Example-11

Compute DFT of $x[n] = \{1, 2, 1, 2\}$ using DIFFFT algorithm. (May19, 05 marks)



Example-11

Compute DFT of $x[n] = \{1, 2, 1, 2\}$ using DIFFFT algorithm. (May19, 05 marks)



Thank You